

Αναφορά

Επεξεργασία και Ανάλυση Εικόνας

ΜΕΡΟΣ Β

Θέμα 3: Βελτίωση και αποκατάσταση εικόνων χαμηλού φωτισμού

Όνομα: Γεώργιος Λαλάς

Αριθμός μητρώου: 1093406

ΠΕΡΙΕΧΟΜΕΝΑ

- 1) Περίληψη
- 2) Αρχεία εικόνων
- 3) Μέρος Α - Ανάλυση εικόνων χαμηλού φωτισμού
- 4) Μέρος Β - Γραμμικοί και μη γραμμικοί μετασχηματισμοί φωτεινότητας
- 5) Μέρος Γ - Ισοστάθμιση ιστογράμματος
- 6) Μέρος Δ - Αποθορυβοποίηση εικόνας
- 7) Μέρος Ε - Όξυνση εικόνα
- 8) Μέρος ΣΤ - Σύγκριση πλήρων pipelines
- 9) Μέρος Ζ - Ποσοτική αξιολόγηση
- 10) Μέρος Η - Επίδραση των μεθόδων σε ακμές και χαρακτηριστικά
- 11) Γενικά συμπεράσματα
- 12) Κώδικας MATLAB

Για σωστή λειτουργία πρέπει να τροποποιηθεί η μεταβλητή `projectDir` στην αρχή του κάθε κώδικα MATLAB, ώστε να δείχνει στον τοπικό φάκελο όπου βρίσκονται οι εικόνες και επίσης θα δημιουργηθούν τα αποτελέσματα.

Οι εικόνες εισόδου πρέπει να είναι με μορφή `αριθμός_low.png` και `αριθμός_high.png`.

Οι φάκελοι αποτελεσμάτων δημιουργούνται αυτόματα από τα `scripts` και τα πλήρη αποτελέσματα είναι χωρισμένα ανά εικόνα και ανά μέρος της άσκησης.

1. Περίληψη

Η εργασία εξετάζει την ανάλυση, βελτίωση και αποκατάσταση εικόνων χαμηλού φωτισμού με κλασικές τεχνικές επεξεργασίας εικόνων. Η μελέτη ξεκινά από απλή στατιστική ανάλυση των `low-light` εικόνων και συνεχίζει με μετασχηματισμούς φωτεινότητας, ισοστάθμιση ιστογράμματος, αποθορυβοποίηση, όξυνση, ολοκληρωμένα pipelines, ποσοτική αξιολόγηση και επίδραση στις ακμές.

Η τελική υλοποίηση οργανώνει τα αποτελέσματα ανά μέρος της άσκησης. Στην αναφορά παρουσιάζονται αντιπροσωπευτικά figures και συγκεντρωτικοί πίνακες, ενώ τα πλήρη PNG, CSV και TXT αποτελέσματα βρίσκονται σε GitHub repository με link:

<https://github.com/GEOLAL03/Image-Processing-2>

2. Αρχεία εικόνων

Χρησιμοποιήθηκαν 15 ζεύγη εικόνων. Η low-light εικόνα έχει μορφή αριθμός_low.png (πχ 22_low.png) και η αντίστοιχη φωτεινή εικόνα αναφοράς έχει όνομα αριθμός_high.png (πχ 22_high.png).

Τα IDs των εικόνων είναι: 1, 22, 23, 55, 79, 111, 146, 179, 493, 547, 665, 669, 748, 778, 780.

Οι high χρησιμοποιούνται ως εικόνες αναφοράς χρησιμοποιούνται όπου ζητείται ποσοτική σύγκριση, κυρίως στα Μέρη Ε, Ζ και Η. Οι υπολογισμοί γίνονται σε double μορφή με τιμές κανονικοποιημένες στο $[0,1]$, ενώ οι αποθηκευμένες εικόνες γράφονται ξανά ως PNG.

Όπως αναφέρθηκε πριν, η αναφορά δεν έχει όλα τα παραγόμενα αρχεία, επειδή είναι πολύ μεγάλος όγκος. Συγκεκριμένα, παρουσιάζονται οι εκτελέσεις στην εικόνα 146, γενικά graphs και συνοπτικές μετρικές για αποφυγή overbloating της αναφοράς με επαναλαμβανόμενες εκτελέσεις.

Τα αποτελέσματα ολοκληρωμένων εκτελέσεων όλων των εικόνων βρίσκονται στο Github.

3. Μέρος Α - Ανάλυση εικόνων χαμηλού φωτισμού

Η ανάλυση μιας εικόνας χαμηλού φωτισμού ξεκινά από τη μελέτη της κατανομής των τιμών έντασης. Για τον λόγο αυτό χρησιμοποιείται η grayscale εκδοχή της εικόνας, ώστε η φωτεινότητα να εξετάζεται ανεξάρτητα από το χρώμα. Το ιστόγραμμα δείχνει πόσα pixels αντιστοιχούν σε κάθε στάθμη έντασης. Σε μία σχετικά πιο σκοτεινή εικόνα αναμένεται οι περισσότερες τιμές να συγκεντρώνονται κοντά στο μηδέν, ενώ οι υψηλές στάθμες να εμφανίζονται σπάνια. Τα RGB ιστογράμματα χρησιμοποιούνται συμπληρωματικά, επειδή μπορούν να δείξουν αν κάποιο χρωματικό κανάλι είναι περισσότερο καταπιεσμένο λόγω του χαμηλού φωτισμού.

Η μέση φωτεινότητα περιγράφει το συνολικό επίπεδο φωτισμού της εικόνας, ενώ η διασπορά και η τυπική απόκλιση δείχνουν πόσο απλωμένες είναι οι εντάσεις γύρω από τη μέση τιμή. Μία χαμηλή μέση τιμή δείχνει ότι η εικόνα είναι σκοτεινή, δεν αρκεί μόνο αυτή για να περιγράψει η αντίθεση. Όταν και η διασπορά είναι μικρή, οι περισσότερες εντάσεις βρίσκονται σε στενό εύρος, με αποτέλεσμα διαφορετικά αντικείμενα να διακρίνονται δύσκολα. Αντίθετα, μία μεγαλύτερη διασπορά μπορεί να οφείλεται είτε σε καλύτερη αντίθεση είτε στην παρουσία λίγων πηγών φωτός μέσα σε μια σκοτεινή εικόνα.

Στο πρώτο μέρος φορτώνονται οι low-light εικόνες, παράγεται grayscale εκδοχή και υπολογίζονται ιστογράμματα φωτεινότητας. Για κάθε εικόνα υπολογίζονται η μέση φωτεινότητα μ , η διασπορά σ^2 , η τυπική απόκλιση ως δείκτης αντίθεσης, τα ποσοστά σκοτεινών/φωτεινών pixels και η Michelson contrast όπου ορίζεται.

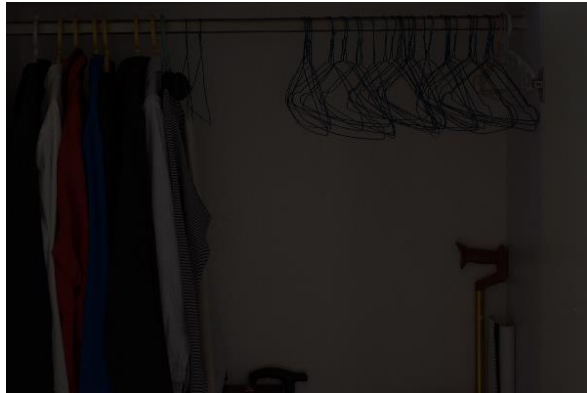
Η μέση φωτεινότητα των 15 εικόνων είναι πολύ χαμηλή. Ο μέσος όρος της MeanBrightness είναι 0.0598 και το μέσο ποσοστό σκοτεινών pixels είναι 99.00%. Αυτό επιβεβαιώνει ότι οι τιμές του ιστογράμματος συγκεντρώνονται κυρίως στις χαμηλές στάθμες φωτεινότητας.

Συγκεντρωτικά στατιστικά low-light εικόνων

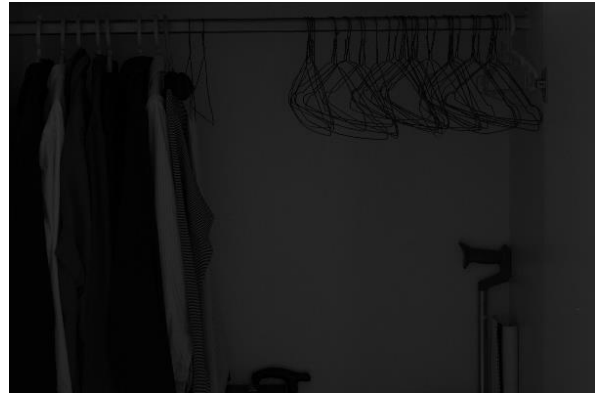
Μετρική	Μέση τιμή	Ελάχιστο	Μέγιστο
Mean brightness	0.0598	0.0252	0.0966
Intensity variance	0.0018	0.0004	0.0083
Std contrast	0.0395	0.0192	0.0914
Dark pixels	99.00%	87.15%	100.00%
Bright pixels	0.00%	0.00%	0.02%

Η σκοτεινότερη εικόνα με βάση τη μέση φωτεινότητα είναι η 493_low.png με MeanBrightness 0.0252, ενώ η φωτεινότερη low-light εικόνα είναι η 1_low.png με MeanBrightness 0.0966. Η μικρή τυπική απόκλιση στις περισσότερες εικόνες συνδέεται με χαμηλή αντίθεση και περιορισμένη διακριτικότητα λεπτομερειών.

Ακολουθεί ένα παράδειγμα Grayscale:

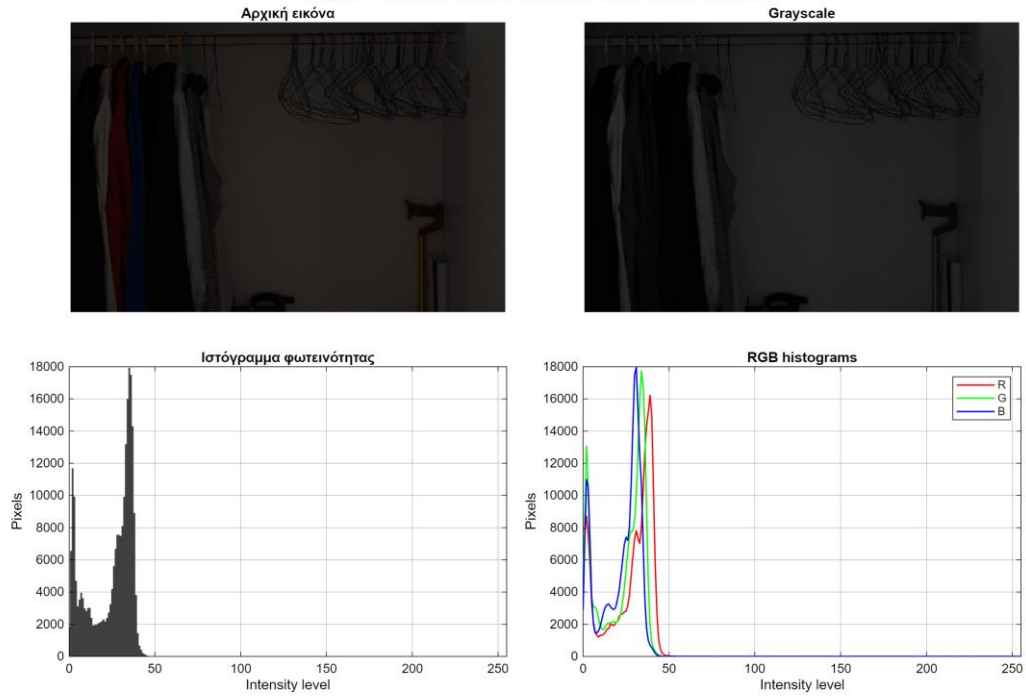


Αρχική low-light εικόνα 146_low.png.

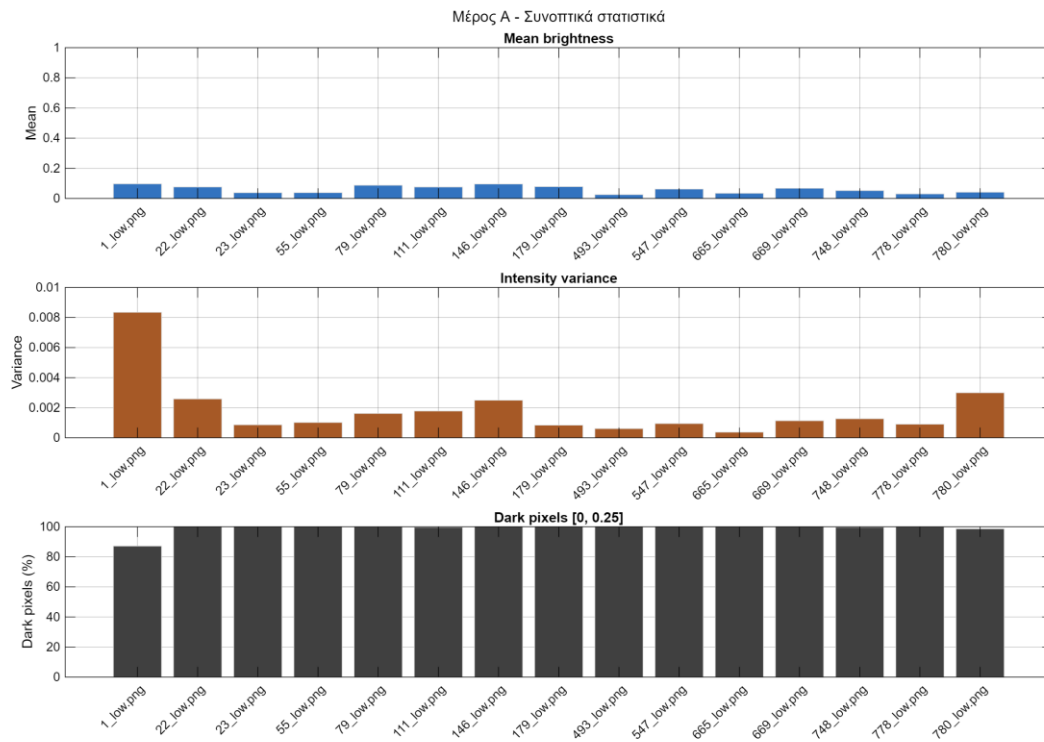


Grayscale εκδοχή της 146_low.png.

Μέρος A - 146_low.png | μέση φωτεινότητα 0.0956, διασπορά 0.00249



Ιστόγραμμα φωτεινότητας και RGB για την 146_low.png.



Συνοπτικό διάγραμμα στατιστικών του Μέρους A για όλες τις low-light εικόνες.

Τα αποτελέσματα δείχνουν ότι το σύνολο εικόνων είναι πράγματι αρκετά σκοτεινό, καθώς τα ιστογράμματα συγκεντρώνονται κυρίως στις χαμηλές στάθμες και το ποσοστό σκοτεινών pixels είναι πολύ υψηλό.

4. Μέρος Β - Γραμμικοί και μη γραμμικοί μετασχηματισμοί φωτεινότητας

Στο Μέρος Β εφαρμόζονται τρεις βασικές τεχνικές για να αυξηθεί η φωτεινότητα των low-light εικόνων:

Γραμμική επέκταση δυναμικού εύρους, η μέθοδος αυτή απλώνει τις τιμές των pixels ώστε να χρησιμοποιείται μεγαλύτερο μέρος του διαθέσιμου εύρους φωτεινότητας. Έτσι αυξάνεται η αντίθεση, αλλά μπορεί να γίνει πιο εμφανής και ο θόρυβος.

Gamma correction, σε αυτή με τιμές $\gamma < 1$ φωτίζονται κυρίως τα σκοτεινά pixels, ενώ για $\gamma > 1$ η εικόνα σκοτεινιάζει. Όσο μικρότερη είναι η τιμή του γ , τόσο πιο έντονη είναι η ενίσχυση.

Λογαριθμικός μετασχηματισμός, ενισχύει περισσότερο τις χαμηλές εντάσεις και συμπιέζει τις υψηλές. Συνήθως δίνει πιο ήπια φωτεινότητα από μια πολύ μικρή τιμή gamma.

Σε αυτό το ερώτημα εφαρμόστηκαν αυτές οι μέθοδοι στην πράξη. Όπως αναφέρθηκε και πριν για gamma < 1 οι σκοτεινές περιοχές φωτίζονται περισσότερο, ενώ πάλι για gamma > 1 η εικόνα σκοτεινιάζει.

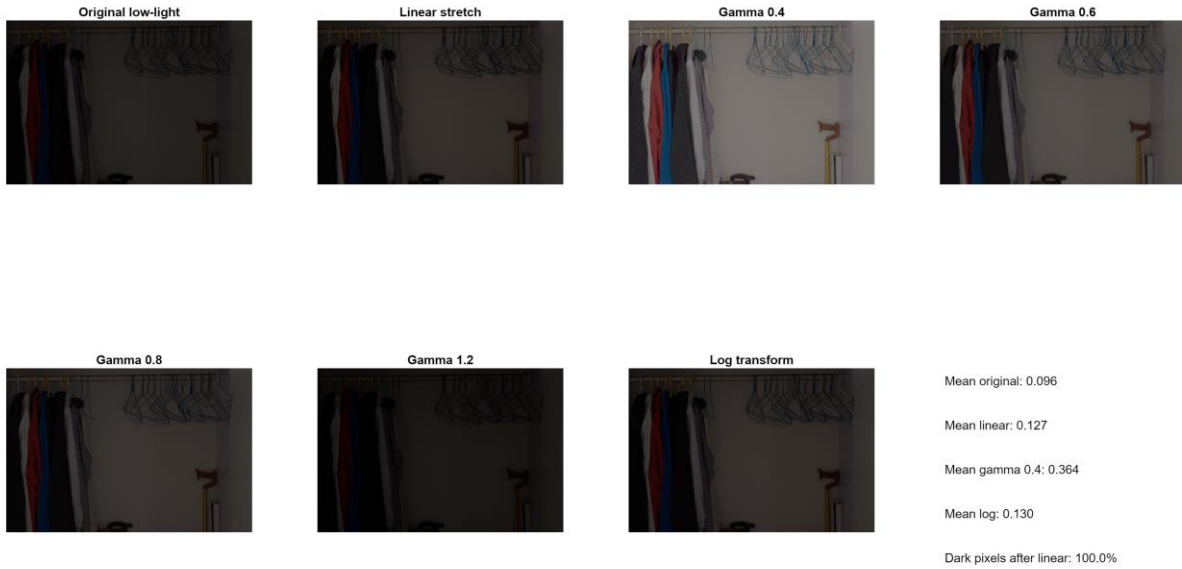
Το linear stretch ναι μεν ανοίγει το δυναμικό εύρος αλλά μπορεί να κάνει πιο ορατό τον θόρυβο όταν η αρχική εικόνα είναι πολύ σκοτεινή.

Όπως φαίνεται και στο κάτω διάγραμμα, στα αποτελέσματα η gamma 0.4 δίνει τη μεγαλύτερη αύξηση φωτεινότητας, με mean 0.2890 και μείωση των dark pixels σε 33.64%. Η gamma 0.6 είναι πιο ήπια. Η gamma 1.2 μειώνει ακόμη περισσότερο τη φωτεινότητα και δεν είναι κατάλληλη για βελτίωση low-light εικόνων.

Μέσες μετρικές μετασχηματισμών φωτεινότητας

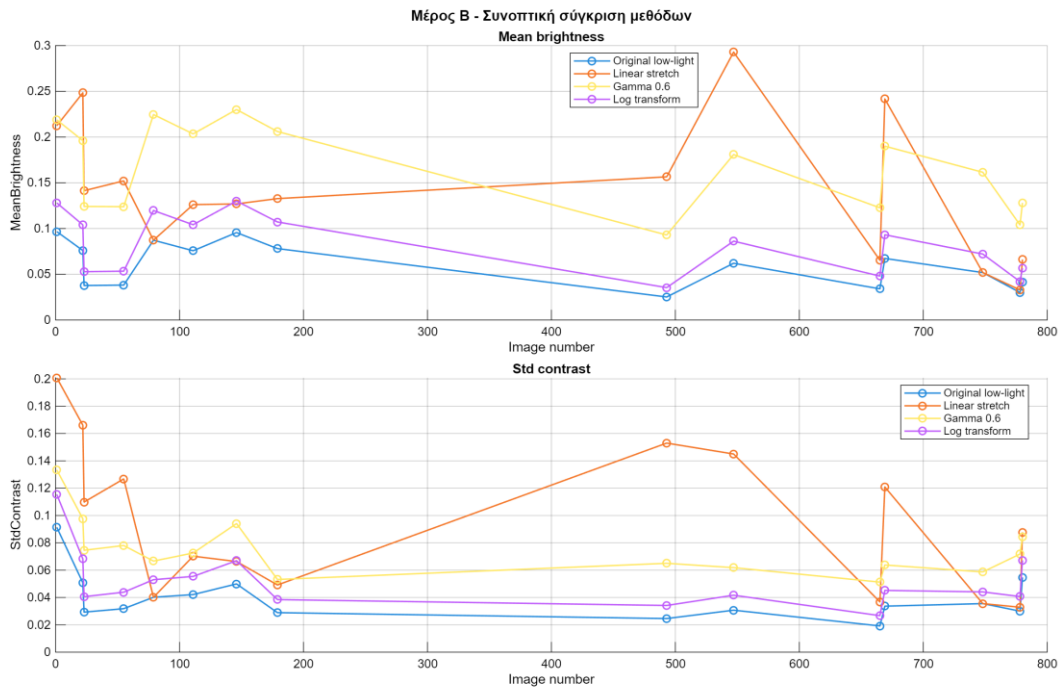
Μέθοδος	Mean	Std	Dark pixels	Saturated
Original low-light	0.0598	0.0395	99.00%	0.00%
Linear stretch	0.1423	0.0961	82.17%	0.00%
Gamma 0.4	0.2890	0.0990	33.64%	0.00%
Gamma 0.6	0.1671	0.0751	82.45%	0.00%
Gamma 0.8	0.0989	0.0549	98.04%	0.00%
Gamma 1.2	0.0368	0.0285	99.89%	0.00%
Log	0.0822	0.0521	98.49%	0.00%

Μέρος Β - Σύγκριση μετασχηματισμών φωτεινότητας: 146_low.png



Σύγκριση linear, gamma και log μετασχηματισμών για την 146_low.png.

Οπτικά η Gamma 0.4 παρουσιάζει το καλύτερο αποτέλεσμα επιβεβαιώνοντας τα στατιστικά.



Συνοπτικό διάγραμμα στατιστικών για mean brightness, STD contrast για όλες τις εικόνες (με βάση αριθμό εικόνας).

Στα αποτελέσματα, η gamma 0.4 έδωσε τη μεγαλύτερη αύξηση φωτεινότητας και μείωσε αρκετά το ποσοστό των σκοτεινών pixels. Παρ' όλα αυτά, είναι και η πιο έντονη επέμβαση και μπορεί να κάνει την εικόνα λιγότερο φυσική. Η gamma 0.6 δίνει πιο ισορροπημένο αποτέλεσμα, ενώ η gamma 1.2 δεν βοηθά στην ενίσχυση, αφού σκοτεινιάζει ακόμη περισσότερο την εικόνα. Ο λογαριθμικός μετασχηματισμός βελτιώνει ελαφρά τις σκοτεινές περιοχές, αλλά

στο συγκεκριμένο σύνολο ήταν πιο αδύναμος από το gamma correction.

5. Μέρος Γ - Ισοστάθμιση ιστογράμματος

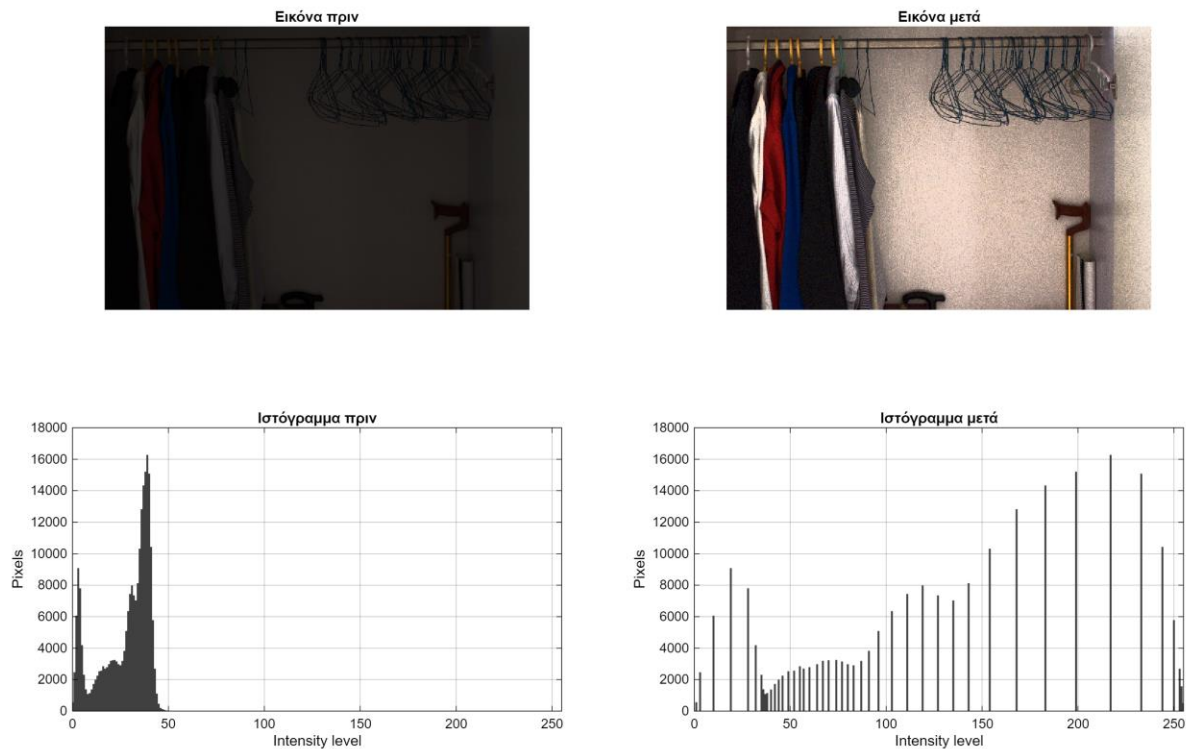
Η global ισοστάθμιση χρησιμοποιεί την CDF για να ανακατανείμει τις τιμές των pixels σε μεγαλύτερο εύρος φωτεινότητας. Έτσι αυξάνει συνολικά την αντίθεση, αλλά μπορεί να υπερενισχύσει τον θόρυβο και τις ήδη φωτεινές περιοχές

Η CLAHE (Contrast Limited Adaptive Histogram Equalization, δηλαδή Προσαρμοστική ισοστάθμιση ιστογράμματος με περιορισμό αντίθεσης) χωρίζει την εικόνα σε μικρές περιοχές, τα λεγόμενα tiles, και η ισοστάθμιση γίνεται ξεχωριστά σε καθεμία. Το ClipLimit περιορίζει την υπερβολική ενίσχυση της αντίθεσης, ώστε να μη γίνονται πολύ έντονα ο θόρυβος και οι τοπικές διαφορές φωτεινότητας.

Μέσες μετρικές ισοστάθμισης ιστογράμματος

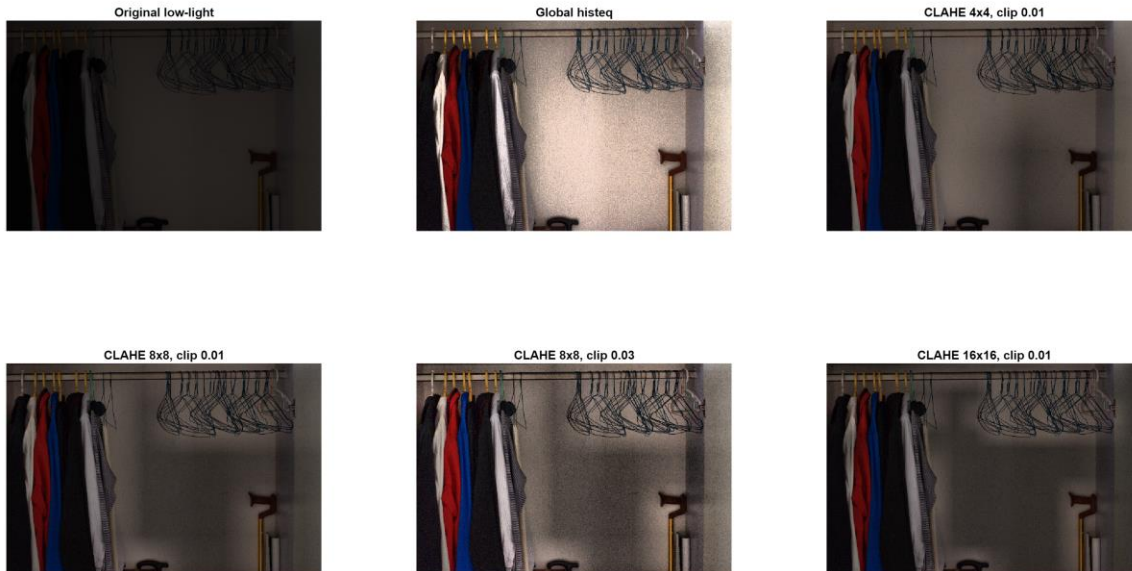
Μέθοδος	Mean	Std	Entropy	Dark pixels	Saturated
Original	0.0726	0.0454	4.999	98.83%	0.01%
Global HE CDF	0.5199	0.2852	4.964	23.30%	2.52%
CLAHE 4x4 clip 0.01	0.1988	0.1126	6.452	68.86%	0.01%
CLAHE 8x8 clip 0.01	0.1811	0.1036	6.386	75.58%	0.01%
CLAHE 8x8 clip 0.03	0.3039	0.1681	7.222	43.32%	0.01%
CLAHE 16x16 clip 0.01	0.1655	0.0933	6.248	82.11%	0.01%

Μέρος Γ1 - Global histogram equalization: 146_low.png

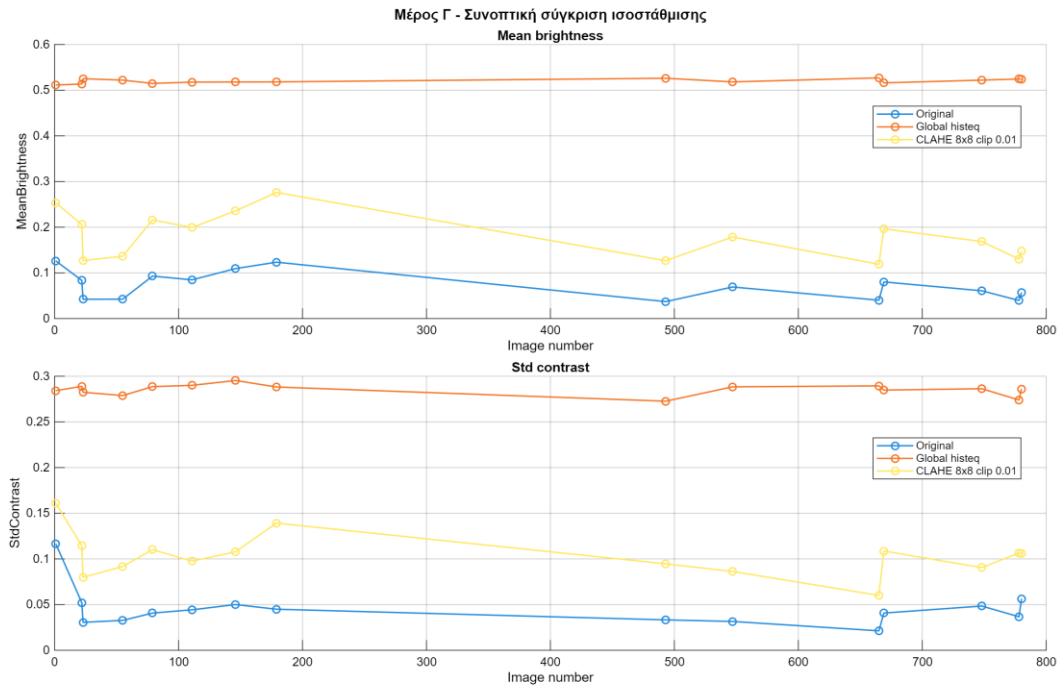


Global histogram equalization και ιστογράμματα πριν/μετά για την 146_low.png.

Μέρος Γ2 - Adaptive histogram equalization: 146_low.png



Σύγκριση CLAHE ρυθμίσεων για την 146_low.png.



Συνοπτικό διάγραμμα στατιστικών του Μέρους Γ για όλες τις εικόνες (πάλι ανά αριθμό εικόνας).

Στα αποτελέσματα, η global ισοστάθμιση αύξησε περισσότερο τη συνολική φωτεινότητα και αντίθεση, αλλά έδωσε πιο έντονο και λιγότερο ελεγχόμενο αποτέλεσμα. Η CLAHE αποκάλυψε καλύτερα τοπικές λεπτομέρειες, ενώ η ρύθμιση 8x8 με ClipLimit 0.03 έδωσε την ισχυρότερη ενίσχυση, με μεγαλύτερο όμως κίνδυνο να γίνει εμφανής η παρουσία θορύβου.

Συμπερασματικά, η global ισοστάθμιση είναι πιο κατάλληλη όταν θέλουμε έντονη συνολική αντίθεση. Η CLAHE είναι πιο ελεγχόμενη και αναδεικνύει τοπικές λεπτομέρειες, αλλά χρειάζεται προσοχή σε ClipLimit και μέγεθος tile.

6. Μέρος Δ - Αποθορυβοποίηση εικόνας

Στο Μέρος Δ προστέθηκε Gaussian θόρυβος και Salt & Pepper θόρυβος στις low-light εικόνες. Στη συνέχεια εφαρμόστηκαν mean, Gaussian, median και Wiener φίλτρα. Επιπλέον εξετάστηκαν δύο σειρές επεξεργασίας: πρώτα αποθορυβοποίηση και μετά ενίσχυση, ή πρώτα ενίσχυση και μετά αποθορυβοποίηση.

Η αποθορυβοποίηση δεν πρέπει να αξιολογείται μόνο από το πόσο μειώνεται ο θόρυβος, αλλά και από το πόσο καλά διατηρούνται οι ακμές και οι υφές. Ένα πολύ ισχυρό φίλτρο μπορεί να βελτιώσει το MSE ή το PSNR, αλλά ταυτόχρονα να θολώσει σημαντικές λεπτομέρειες. Για τον λόγο αυτό χρησιμοποιούνται συμπληρωματικά το gradient ratio και το edge correlation, ώστε να εξετάζεται αν η εικόνα παραμένει καθαρή χωρίς να χάνει τη δομή της.

Μέσες μετρικές φίλτρων αποθορυβοποίησης

Θόρυβος	Φίλτρο	MSE	PSNR	MAE	Gradient ratio	Edge corr.
Gaussian noise	Mean filter	0.0007	31.68	0.0208	1.802	0.435
Gaussian noise	Gaussian filter	0.0006	32.38	0.0194	1.599	0.464
Gaussian noise	Median filter	0.0008	31.00	0.0216	2.384	0.380
Gaussian noise	Wiener filter	0.0006	32.28	0.0194	1.263	0.470
Salt & Pepper noise	Mean filter	0.0030	25.19	0.0285	3.980	0.166
Salt & Pepper noise	Gaussian filter	0.0024	26.20	0.0267	3.502	0.181
Salt & Pepper noise	Median filter	0.0001	40.89	0.0050	0.690	0.880
Salt & Pepper noise	Wiener filter	0.0079	21.05	0.0271	4.527	0.082

Για Gaussian θόρυβο, το Gaussian φίλτρο και το Wiener φίλτρο δίνουν τις καλύτερες μέσες τιμές PSNR, περίπου 32.38 dB και 32.28 dB αντίστοιχα. Για Salt & Pepper θόρυβο, το median φίλτρο είναι καθαρά καλύτερο με μέσο PSNR 40.89 dB και edge correlation 0.880, επειδή απορρίπτει τις ακραίες τιμές χωρίς να τις διαχέει.

Πράγμα που ισχύει με βάση τη θεωρία καθώς gaussian καλύπτει με gaussian κατανομές και το wiener δίνει επίσης καλά αποτελέσματα λόγω του Minimum MSE κριτηρίου του. Το median λειτουργεί για salt pepper επειδή είναι ακραίες τιμές (πχ 0, 255) και τις αντικαθιστά με τη διάμεσο των γειτονικών του πίξελ.

Σύγκριση σειράς αποθορυβοποίησης και ενίσχυσης

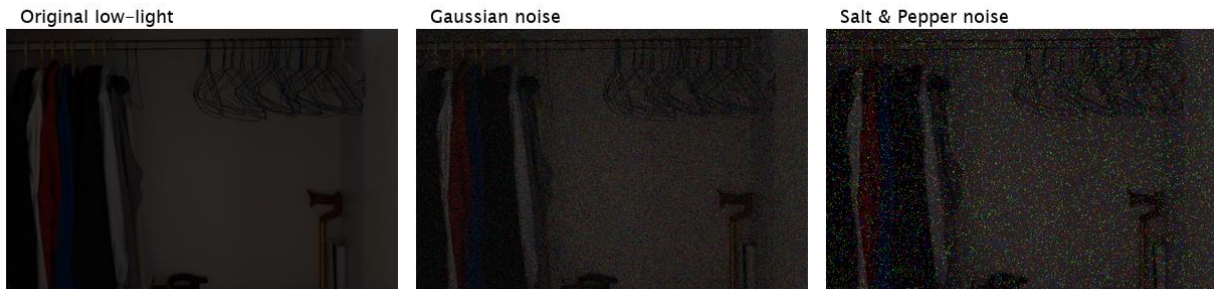
Θόρυβος	Φίλτρο	Σειρά	MSE	PSNR	MAE	Gradient ratio	Edge corr.
Gaussian noise	Gaussian filter	denoise -> enhance	0.0218	18.64	0.1132	1.624	0.408
Gaussian noise	Gaussian filter	enhance -> denoise	0.0120	21.39	0.0819	1.210	0.407
Gaussian noise	Median filter	denoise -> enhance	0.0209	17.86	0.1104	2.691	0.321
Gaussian noise	Median filter	enhance -> denoise	0.0151	19.61	0.0920	1.985	0.321
Salt & Pepper noise	Gaussian filter	denoise -> enhance	0.0191	17.72	0.1011	2.103	0.219
Salt & Pepper noise	Gaussian filter	enhance -> denoise	0.0236	19.36	0.1108	1.035	0.406
Salt & Pepper noise	Median filter	denoise -> enhance	0.0324	19.36	0.1286	0.607	0.880
Salt & Pepper noise	Median filter	enhance -> denoise	0.0248	20.59	0.1118	0.448	0.880

Η σειρά των δύο βημάτων επηρεάζει και τη μορφή του θορύβου. Όταν προηγείται η ενίσχυση φωτεινότητας, οι μικρές διακυμάνσεις των σκοτεινών pixels μπορεί να γίνουν πιο έντονες και το φίλτρο καλείται στη συνέχεια να τις

περιορίσει. Αντίθετα, όταν προηγείται η αποθορυβοποίηση, μειώνεται πρώτα ο θόρυβος, αλλά υπάρχει κίνδυνος να χαθούν λεπτομέρειες πριν αυτές ενισχυθούν.

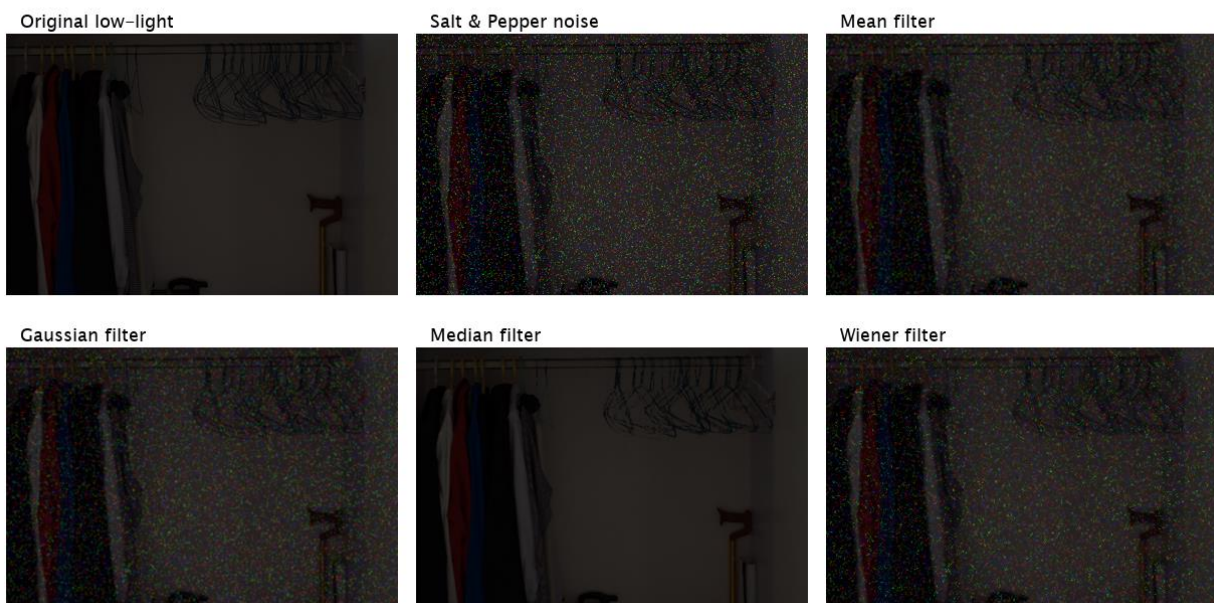
Η σειρά επεξεργασίας δεν έχει μία απόλυτη απάντηση για όλες τις περιπτώσεις. Με βάση το PSNR, στις δοκιμές αυτές η σειρά enhance -> denoise είναι συχνά καλύτερη στις τελικές εικόνες, επειδή το φίλτρο εφαρμόζεται μετά την ανάδειξη του θορύβου. Όμως για διατήρηση δομών, ειδικά στον Salt & Pepper θόρυβο, το median φίλτρο παραμένει κρίσιμο ανεξάρτητα από τη σειρά.

Part D – Noise: 146_low.png



Αρχική εικόνα, Gaussian θόρυβος και Salt & Pepper θόρυβος για την 146_low.png.

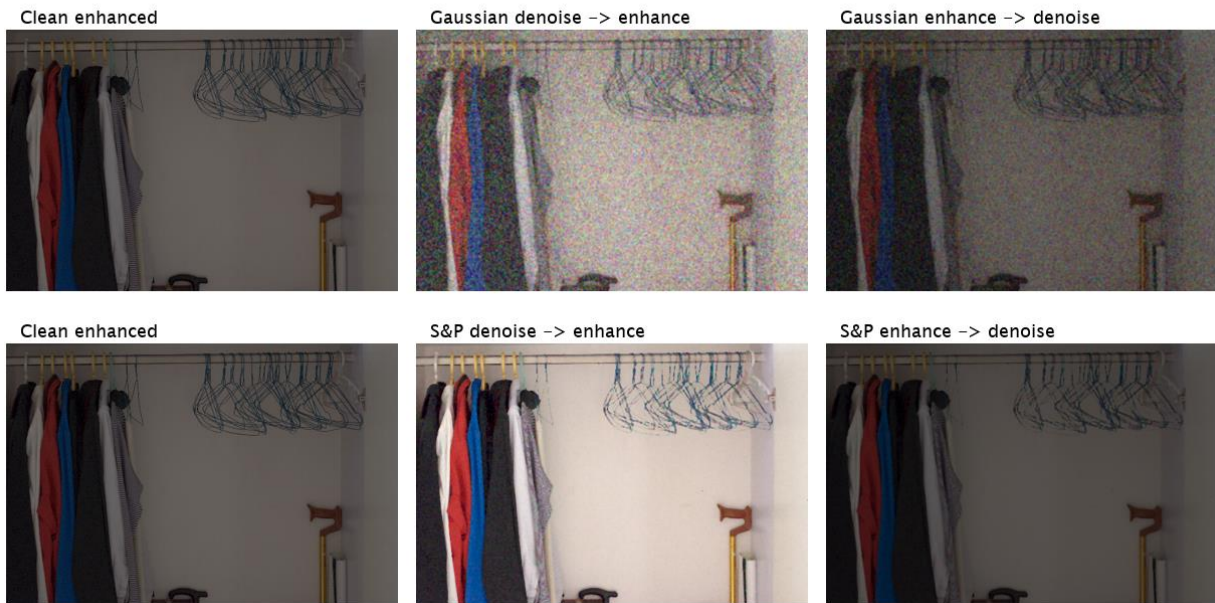
Part D – Filters: 146_low.png – Salt & Pepper noise



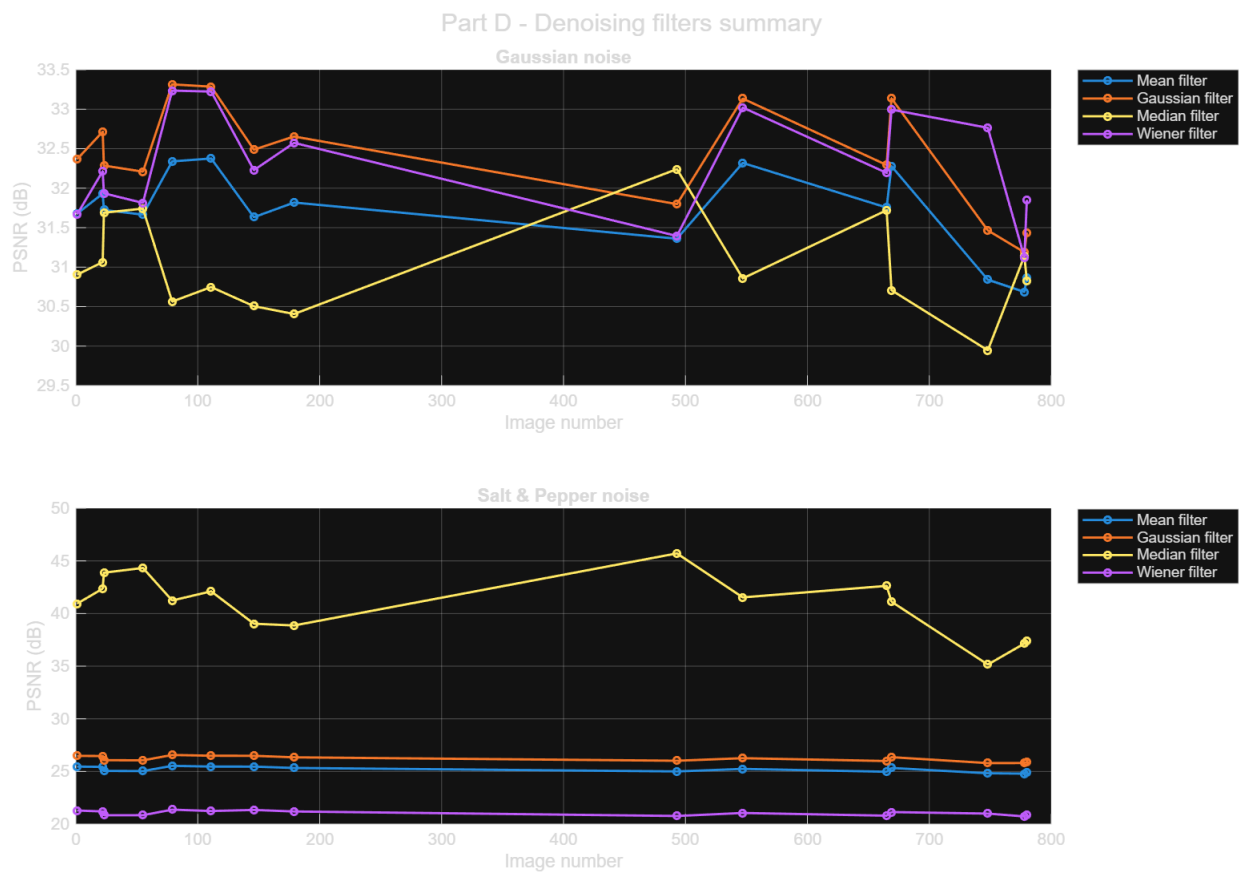
Σύγκριση φίλτρων σε Salt & Pepper θόρυβο για την 146_low.png.

Εδώ φαίνεται και στην πράξη πως το median λειτουργεί καλύτερα για salt pepper noise

Part D – Processing order: 146_low.png



Σύγκριση των δύο σειρών επεξεργασίας για την 146_low.png.



Συνοπτικό διάγραμμα στατιστικών του Μέρους Δ για όλες τις εικόνες.

Οπτικά, το Gaussian και το Wiener φίλτρο δίνουν πιο ομαλή εικόνα στον Gaussian θόρυβο, χωρίς όμως να διατηρούν όλες τις μικρές λεπτομέρειες το ίδιο καλά. Στον Salt & Pepper θόρυβο, το median φίλτρο ξεχωρίζει καθαρά, επειδή αφαιρεί τις έντονες λευκές και μαύρες κουκκίδες χωρίς να θολώνει τόσο τις ακμές. Στη σύγκριση της σειράς επεξεργασίας, το enhance → denoise φαίνεται πιο καθαρό στις περισσότερες περιπτώσεις, επειδή το φίλτρο περιορίζει τον θόρυβο αφού αυτός έχει γίνει πιο εμφανής.

7. Μέρος Ε - Όξυνση εικόνας

Μετά από βασική ενίσχυση και αποθορυβοποίηση εξετάστηκαν Laplacian sharpening και unsharp masking.

Το Laplacian sharpening βασίζεται στη δεύτερη παράγωγο της εικόνας και εντοπίζει περιοχές όπου οι τιμές των pixels αλλάζουν απότομα. Με την προσθήκη αυτής της πληροφορίας στην αρχική εικόνα, οι ακμές γίνονται πιο έντονες, αλλά μπορεί να ενισχυθούν και μικρές διακυμάνσεις που οφείλονται σε θόρυβο.

Στο unsharp masking δημιουργείται πρώτα μία θολωμένη εκδοχή της εικόνας και στη συνέχεια αφαιρείται από την αρχική, ώστε να προκύψει ένα 'mask' λεπτομερειών. Η παράμετρος k καθορίζει πόσο έντονα προστίθεται αυτό το mask: μικρές τιμές δίνουν πιο ήπιο αποτέλεσμα, ενώ μεγάλες τιμές κάνουν τις ακμές πιο έντονες αλλά αυξάνουν τον κίνδυνο halos.

Γενικά η όξυνση αυξάνει τις τοπικές μεταβολές και μπορεί να κάνει πιο εμφανείς ακμές και λεπτομέρειες, αλλά αν είναι υπερβολική μπορεί να εισάγει halos, κορεσμό ή οπτικό θόρυβο.

Μέσες μετρικές όξυνσης

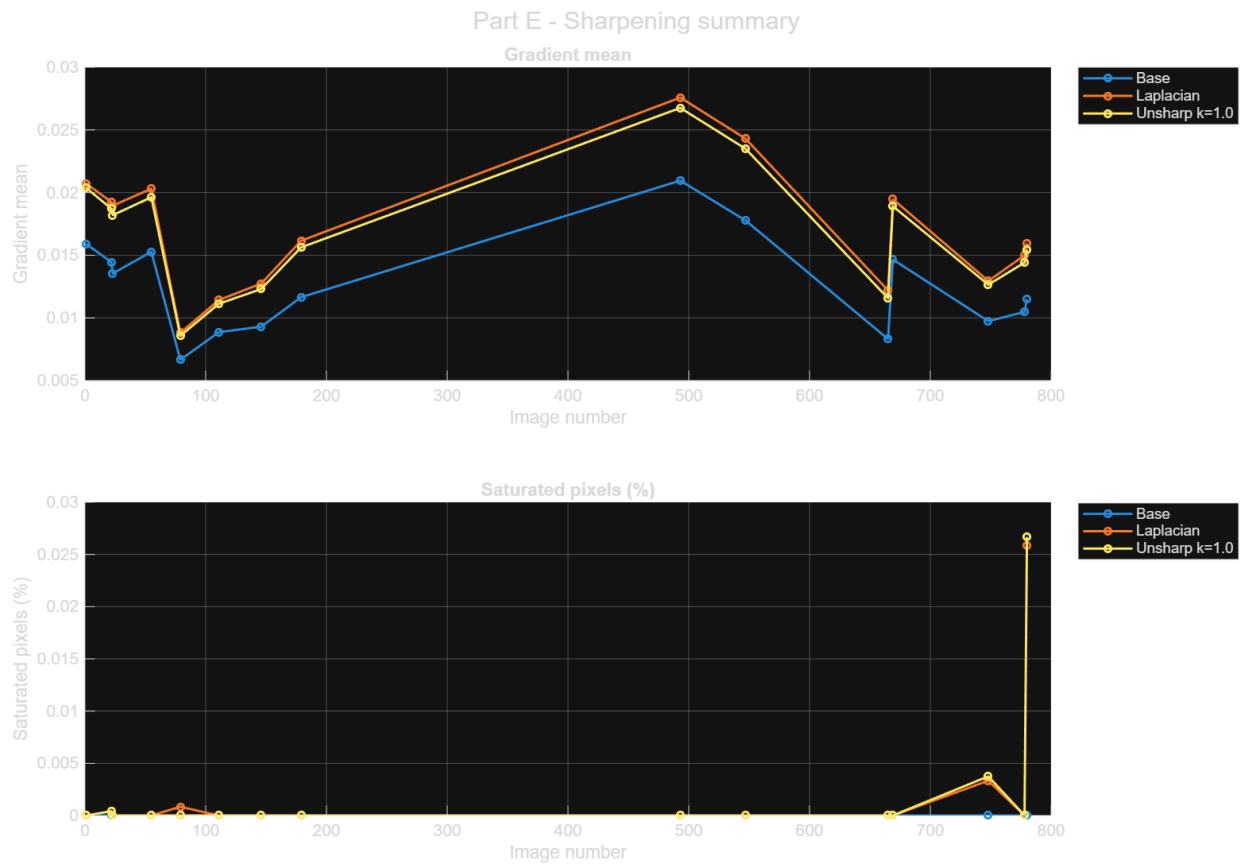
Μέθοδος	Mean	Std	Gradient	Grad. ratio	Laplacian energy	Saturated	PSNR
Original	0.0598	0.0395	0.0051	0.455	0.0192	0.00%	7.77
Base enhanced denoised	0.2756	0.1241	0.0126	1.000	0.0162	0.00%	14.67
Laplacian	0.2757	0.1267	0.0171	1.357	0.0316	0.00%	14.67
Unsharp $k=0.5$	0.2756	0.1253	0.0145	1.152	0.0215	0.00%	14.68
Unsharp $k=1.0$	0.2757	0.1265	0.0165	1.314	0.0270	0.00%	14.68
Unsharp $k=1.5$	0.2758	0.1279	0.0186	1.481	0.0325	0.00%	14.66

Το unsharp $k=0.5$ και 1 δίνουν το υψηλότερο μέσο PSNR από τις δοκιμές όξυνσης, 14.68 dB. Το $k=1.5$ αυξάνει περισσότερο το gradient ratio, περίπου 1.481, αλλά έχει μεγαλύτερο κίνδυνο υπερβολικής όξυνσης. Το Laplacian sharpening αυξάνει αισθητά την ενέργεια Laplacian, αλλά δεν βελτιώνει το PSNR σε σχέση με την ήπια unsharp επιλογή.

Part E – Sharpening: 146_low.png



Σύγκριση Laplacian και unsharp sharpening για την 146_low.png.



Συνοπτικό διάγραμμα στατιστικών του Μέρους E για όλες τις εικόνες.

Οπτικά, το Laplacian δίνει πιο έντονες και σκληρές ακμές, ενώ το unsharp masking παράγει πιο ελεγχόμενο αποτέλεσμα. Το $k = 0,5$ διατηρεί πιο φυσική εικόνα, το $k = 1$ προσφέρει καλή ισορροπία λεπτομέρειας και φυσικότητας, ενώ στο $k = 1,5$ η όξυνση γίνεται πιο εμφανής και αρχίζουν να εμφανίζονται υπερβολικά περιγράμματα γύρω από ορισμένα αντικείμενα.

8. Μέρος ΣΤ - Σύγκριση πλήρων pipelines

Ένα πλήρες pipeline συνδυάζει διαφορετικά στάδια, επειδή μία μόνο μέθοδος συνήθως δεν μπορεί να διορθώσει ταυτόχρονα τη χαμηλή φωτεινότητα, τον θόρυβο και τη μειωμένη αντίθεση. Η σειρά εφαρμογής των μεθόδων είναι σημαντική, καθώς κάθε επεξεργασία επηρεάζει την επόμενη και μπορεί είτε να βελτιώσει είτε να ενισχύσει ανεπιθύμητα χαρακτηριστικά της εικόνας.

Στο 6^ο ερώτημα δημιουργήθηκαν τρία ολοκληρωμένα pipelines, συγκεκριμένα:

To Pipeline 1 εφαρμόζει gamma 0.6 και linear stretch.

To Pipeline 2 εφαρμόζει median filtering, CLAHE και unsharp masking.

To Pipeline 3 είναι Retinex-inspired και δοκιμάζεται για sigma 15, 30 και 60.

Η Retinex-inspired προσέγγιση προσπαθεί να διαχωρίσει τον φωτισμό από τις πραγματικές λεπτομέρειες της εικόνας. Η παράμετρος 'σ' καθορίζει πόσο ομαλά εκτιμάται ο φωτισμός: μικρό 'σ' ενισχύει περισσότερο τις τοπικές λεπτομέρειες, ενώ μεγαλύτερο 'σ' δίνει πιο ομαλή αλλά και πιο έντονη συνολική μεταβολή.

Συνοπτικές μέσες τιμές πλήρων pipelines

Pipeline	Mean	Std	Entropy	Dark pixels	Saturated
Pipeline 1 - Gamma + stretch	0.2756	0.1278	6.493	49.46%	0.00%
Pipeline 2 - Median + CLAHE + unsharp	0.1387	0.0882	6.162	85.24%	0.00%
Pipeline 3 - Retinex sigma 15	0.5941	0.1758	7.093	6.00%	0.04%
Pipeline 3 - Retinex sigma 30	0.5934	0.1834	7.171	6.67%	0.04%
Pipeline 3 - Retinex sigma 60	0.5911	0.1899	7.234	7.43%	0.04%

To Pipeline 1 είναι πιο συντηρητικό και κρατά χαμηλό κορεσμό, ενώ βελτιώνει αρκετά τη φωτεινότητα. Το Pipeline 2 παραμένει πιο σκοτεινό κατά μέσο όρο, επειδή η median + CLAHE + unsharp ακολουθία είναι πιο ελεγχόμενη. Το Retinex αυξάνει πολύ τη φωτεινότητα και την entropy, ιδιαίτερα για μεγαλύτερα sigma, αλλά μπορεί να δώσει λιγότερο φυσική εικόνα ή να ενισχύσει τοπικό θόρυβο.

Μέρος ΣΤ – Pipelines: 146_low.png



Σύγκριση των pipelines και των Retinex sigma για την 146_low.png.

Όπως φαίνεται και στα αποτελέσματα στη συγκεκριμένη εικόνα ενώ το Retinex πετυχαίνει την καλύτερη φώτιση, το αποτέλεσμα είναι πολύ uncanhny και αφύσικο.

Οπτικά, το Pipeline 1 δίνει το πιο φυσικό και ισορροπημένο αποτέλεσμα, χωρίς να αλλάζει υπερβολικά την εικόνα. Το Pipeline 2 είναι πιο ήπιο και διατηρεί χαμηλότερη φωτεινότητα, ενώ το Retinex αποκαλύπτει περισσότερες λεπτομέρειες αλλά σε ορισμένες εικόνες δημιουργεί υπερβολικά φωτεινό και λιγότερο φυσικό αποτέλεσμα. Από τις τρεις τιμές 'σ', οι μεγαλύτερες τιμές δίνουν πιο ομαλή απόδοση φωτισμού, αλλά δεν εγγυώνται ότι το αποτέλεσμα θα φαίνεται πιο ρεαλιστικό.

9. Μέρος Ζ - Ποσοτική αξιολόγηση

Στο Μέρος Ζ χρησιμοποιούνται οι φωτεινές εικόνες (αριθμός)_high.png ως αναφορά, για τον υπολογισμό MSE και PSNR. Εφαρμόστηκαν στα pipelines του προηγούμενου ερωτήματος.

Το MSE και το PSNR συγκρίνουν το αποτέλεσμα με τη φωτεινή εικόνα αναφοράς, ενώ η μέση τιμή και η τυπική απόκλιση δείχνουν πόσο άλλαξαν η φωτεινότητα και η αντίθεση. Η entropy μπορεί να αυξηθεί επειδή αποκαλύφθηκαν περισσότερες λεπτομέρειες, αλλά μπορεί επίσης να αυξηθεί λόγω θορύβου.

Χαμηλότερο MSE και υψηλότερο PSNR δείχνουν μεγαλύτερη ομοιότητα με την εικόνα αναφοράς, χωρίς όμως να σημαίνουν πάντα ότι το αποτέλεσμα είναι οπτικά φυσικότερο.

Το μέσο PSNR δεν υπολογίζεται απευθείας από το μέσο MSE, επειδή το PSNR είναι λογαριθμική μετρική και οι τιμές

υπολογίζονται πρώτα για κάθε εικόνα ξεχωριστά. Επίσης, η Michelson contrast είναι περίπου ίση με 1 σε όλα τα pipelines, επειδή υπάρχουν pixels με ένταση κοντά στο μηδέν, άρα στη συγκεκριμένη περίπτωση δεν ξεχωρίζει αποτελεσματικά τις μεθόδους.

Μέσες ποσοτικές μετρικές ανά pipeline

Pipeline	MSE	PSNR	Mean	Std	Michelson	Entropy
Pipeline 1 - Gamma + stretch	0.0584	14.55	0.2757	0.1278	1.000	6.491
Pipeline 2 - Median + CLAHE + unsharp	0.1281	9.89	0.1387	0.0882	1.000	6.162
Pipeline 3 - Retinex sigma 15	0.0547	13.18	0.5941	0.1758	1.000	7.092
Pipeline 3 - Retinex sigma 30	0.0516	13.60	0.5934	0.1834	1.000	7.170
Pipeline 3 - Retinex sigma 60	0.0489	14.06	0.5911	0.1899	1.000	7.233

Καλύτερο pipeline ανά εικόνα βάσει PSNR

Εικόνα	Καλύτερο pipeline	PSNR	MSE
1_low.png	Pipeline 1 - Gamma + stretch	15.89	0.0258
22_low.png	Pipeline 1 - Gamma + stretch	16.67	0.0215
23_low.png	Pipeline 3 - Retinex sigma 60	13.38	0.0459
55_low.png	Pipeline 3 - Retinex sigma 60	15.36	0.0291
79_low.png	Pipeline 3 - Retinex sigma 60	21.08	0.0078
111_low.png	Pipeline 3 - Retinex sigma 60	18.82	0.0131
146_low.png	Pipeline 3 - Retinex sigma 60	19.77	0.0105
179_low.png	Pipeline 1 - Gamma + stretch	22.32	0.0059
493_low.png	Pipeline 1 - Gamma + stretch	20.34	0.0092
547_low.png	Pipeline 1 - Gamma + stretch	21.55	0.0070
665_low.png	Pipeline 3 - Retinex sigma 15	13.23	0.0475
669_low.png	Pipeline 1 - Gamma + stretch	18.83	0.0131
748_low.png	Pipeline 1 - Gamma + stretch	13.63	0.0434
778_low.png	Pipeline 3 - Retinex sigma 60	12.89	0.0514
780_low.png	Pipeline 1 - Gamma + stretch	19.52	0.0112

Με βάση το καλύτερο PSNR ανά εικόνα, το Pipeline 1 επιλέγεται σε 8 από τις 15 εικόνες, το Retinex sigma 60 σε 6 εικόνες και το Retinex sigma 15 σε 1 εικόνα. Συγκεντρωτικά, το Retinex sigma 60 έχει το χαμηλότερο μέσο MSE (0.0489) και υψηλότερη αντίθεση, όμως η οπτική αλλαγή είναι πιο έντονη και δεν είναι πάντα φυσική ενώ το Pipeline 1 έχει το υψηλότερο μέσο PSNR (14.55 dB). Επομένως, η καλύτερη επιλογή εξαρτάται από το αν προτεραιότητα είναι η ομοιότητα με τη reference εικόνα ή η ισχυρότερη αποκάλυψη λεπτομερειών.

10. Μέρος Η - Επίδραση των μεθόδων σε ακμές και χαρακτηριστικά

Το computer vision ασχολείται με την αυτόματη εξαγωγή χρήσιμης πληροφορίας από εικόνες, όπως η ανίχνευση ακμών, αντικειμένων και χαρακτηριστικών. Στις low-light εικόνες η χαμηλή φωτεινότητα δυσκολεύει αυτή τη διαδικασία, γι' αυτό η βελτίωσή τους μπορεί να χρησιμοποιηθεί ως στάδιο προεπεξεργασίας πριν από την εφαρμογή αλγορίθμων ανάλυσης.

Για την αξιολόγηση ως πιθανή προεπεξεργασία computer vision εφαρμόστηκαν Sobel και Canny edge detectors στην αρχική low-light εικόνα και στα βελτιωμένα αποτελέσματα.

Οι ανιχνευτές Sobel και Canny χρησιμοποιούνται για να ελεγχθεί αν η βελτίωση της εικόνας βοηθά στην ανάδειξη πραγματικών ακμών. Το Sobel βασίζεται στις τοπικές μεταβολές φωτεινότητας μεταξύ γειτονικών pixels, ενώ το Canny εφαρμόζει περισσότερα στάδια και συνήθως παράγει πιο λεπτές και συνεχόμενες ακμές.

Εξετάζονται:

- 1)πλήθος ακμών, πυκνότητα ακμών, μικρά components ως ένδειξη θορύβου
- 2)σημαντικά components ως ένδειξη συνεκτικών δομών
- 3)Jaccard similarity προς τις ακμές της φωτεινής εικόνας αναφοράς

Η αύξηση του αριθμού των ακμών δεν σημαίνει πάντα καλύτερο αποτέλεσμα, γιατί μπορεί να προέρχεται και από θόρυβο. Για αυτό εξετάζονται όχι μόνο η edge density, αλλά και τα μικρά components, τα σημαντικά components και η ομοιότητα Jaccard με την φωτεινή εικόνα αναφοράς.

Σύνοψη Canny edge metrics

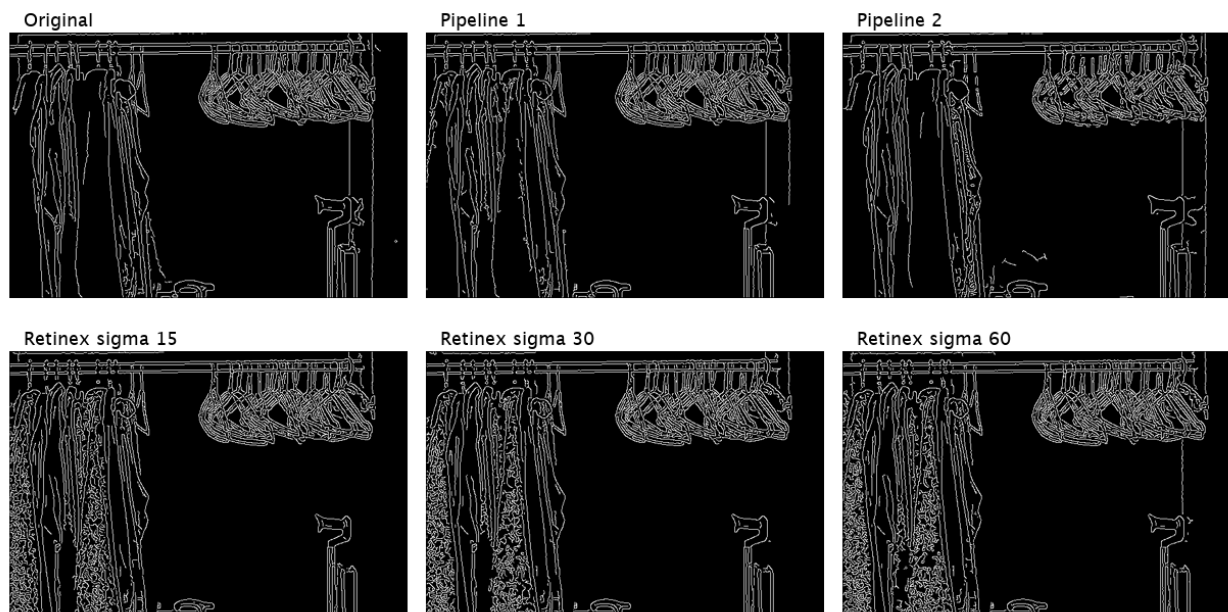
Pipeline	Edge density	Small comp.	Significant comp.	Jaccard to high
Pipeline 1 - Gamma + stretch	9.75%	20.66%	42.84%	0.518
Pipeline 2 - Median + CLAHE + unsharp	9.90%	20.62%	42.72%	0.399
Pipeline 3 - Retinex sigma 15	11.77%	23.67%	36.80%	0.379
Pipeline 3 - Retinex sigma 30	11.88%	22.67%	37.51%	0.371
Pipeline 3 - Retinex sigma 60	11.76%	23.49%	36.71%	0.371

Σύνοψη Sobel edge metrics

Pipeline	Edge density	Small comp.	Significant comp.	Jaccard to high	Ratio to original
Pipeline 1 - Gamma + stretch	3.42%	70.29%	7.58%	0.548	1.151
Pipeline 2 - Median + CLAHE + unsharp	2.93%	60.63%	11.80%	0.327	1.023
Pipeline 3 - Retinex sigma 15	4.03%	77.79%	4.81%	0.364	1.422
Pipeline 3 - Retinex sigma 30	3.94%	77.80%	5.02%	0.367	1.389
Pipeline 3 - Retinex sigma 60	3.90%	77.50%	4.92%	0.368	1.370

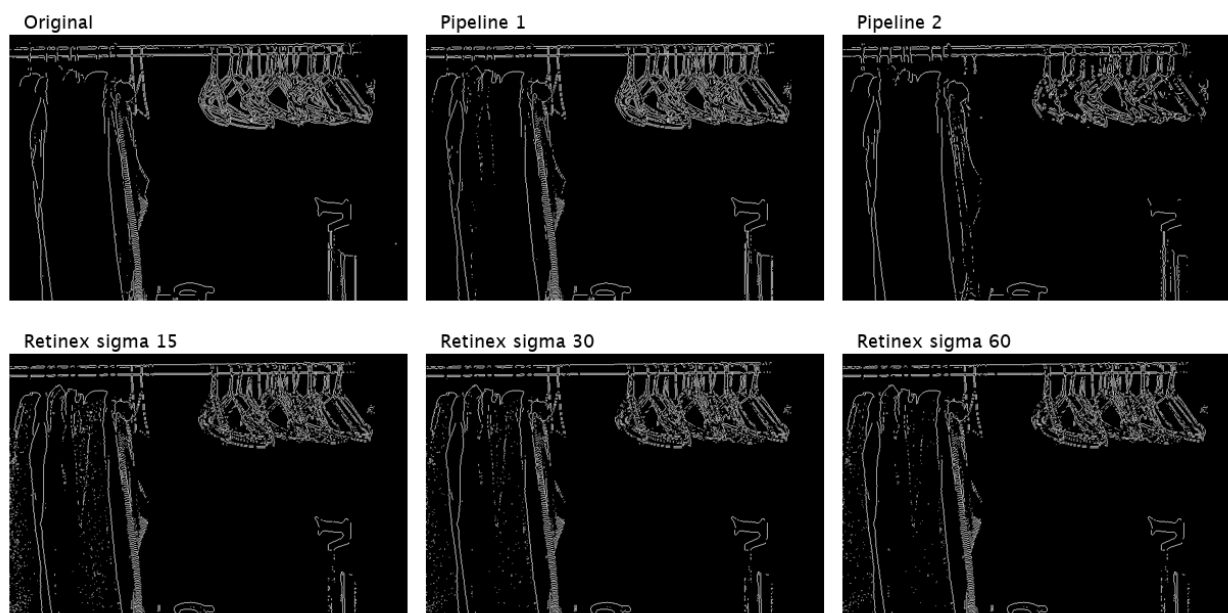
Στο Canny, το Pipeline 1 έχει το μεγαλύτερο μέσο Jaccard προς την high reference εικόνα (0.518), άρα συμφωνεί καλύτερα με τις ακμές της φωτεινής αναφοράς. Τα Retinex pipelines αυξάνουν περισσότερο την edge density, αλλά έχουν υψηλότερο ποσοστό μικρών components και χαμηλότερο Jaccard, κάτι που δείχνει ότι μέρος της επιπλέον πληροφορίας μπορεί να είναι θόρυβος ή υπερενισχυμένη υφή.

Μέρος H – Canny edges: 146_low.png



Canny edge maps για την αρχική και τις βελτιωμένες εκδοχές της 146_low.png.

Μέρος H – Sobel edges: 146_low.png



Οπτικά, το Pipeline 1 δίνει τις πιο καθαρές και συνεκτικές ακμές και συμφωνεί περισσότερο με την high reference εικόνα. Τα Retinex pipelines εμφανίζουν περισσότερες ακμές, αλλά ένα μέρος τους αντιστοιχεί σε μικρά θορυβώδη components ή σε υπερενισχυμένες υφές. Επομένως, η βελτίωση low-light εικόνων μπορεί να βοηθήσει ως preprocessing για computer vision, αρκεί να μην ενισχύει υπερβολικά τον θόρυβο

11. Γενικά συμπεράσματα

Οι αρχικές low-light εικόνες έχουν πολύ χαμηλή μέση φωτεινότητα και τα ιστογράμματα είναι συγκεντρωμένα στις χαμηλές στάθμες. Η gamma διόρθωση με τιμές μικρότερες του 1 είναι η πιο άμεση λύση για αύξηση φωτεινότητας, με τη gamma 0.4 να είναι ισχυρή και τη gamma 0.6 πιο ισορροπημένη.

Η global histogram equalization αυξάνει δυναμικά την αντίθεση αλλά μπορεί να γίνει υπερβολική. Η CLAHE είναι πιο χρήσιμη όταν μας ενδιαφέρει τοπική λεπτομέρεια και έλεγχος του θορύβου. Στην αποθορυβοποίηση, Gaussian/Wiener φίλτρα ταιριάζουν καλύτερα στον Gaussian θόρυβο, ενώ το median φίλτρο είναι σαφώς καλύτερο για Salt & Pepper θόρυβο.

Στην όξυνση, το ήπιο unsharp masking είναι πιο ασφαλές από την επιθετική αύξηση λεπτομερειών. Στα ολοκληρωμένα pipelines, το Pipeline 1 είναι το πιο σταθερό ως προς PSNR και edge similarity, ενώ το Retinex sigma 60 έχει συχνά πολύ καλή MSE αλλά πιο έντονη οπτική αλλαγή. Επομένως, δεν υπάρχει απόλυτα καλύτερη μέθοδος για όλες τις εικόνες· η επιλογή πρέπει να συνδέεται με τον στόχο, δηλαδή οπτική βελτίωση, ομοιότητα με reference ή προεπεξεργασία για ακμές.

Κώδικας MATLAB

Στο παράρτημα παρατίθεται ο τελικός κώδικας MATLAB ανά αρχείο.

Μέρος Α (Αρχείο: eikonaB3A.m)

```
clear;
clc;
close all;

% Για να αναλυθεί μια εικόνα από το script, το όνομά της πρέπει να τελειώνει σε _low.png.

greekDesktop = char([933 960 959 955 959 947 953 963 964 942 962]);
% Replace with your path
projectDir = fullfile(getenv('USERPROFILE'), 'OneDrive', greekDesktop, 'project');
resultsDir = fullfile(projectDir, 'resultsB3A');
expectedFolderName = 'resultsB3A';

[~, actualFolderName] = fileparts(resultsDir);
if isempty(resultsDir) || strcmp(resultsDir, projectDir) || ~strcmp(actualFolderName, expectedFolderName)
    error('Unsafe results folder: %s', resultsDir);
end

if ~exist(resultsDir, 'dir')
    mkdir(resultsDir);
end

oldFiles = [dir(fullfile(resultsDir, '*.png')); dir(fullfile(resultsDir, '*.csv')); dir(fullfile(resultsDir, '*.txt'))];
```

```

dir(fullfile(resultsDir, '*.mat'))];
oldWarningState = warning('off', 'all');
for oldIdx = 1:numel(oldFiles)
    oldPath = fullfile(resultsDir, oldFiles(oldIdx).name);
    if exist(oldPath, 'file') == 2
        try
            delete(oldPath);
        catch
            end
    end
end
warning(oldWarningState);

[imageFiles, imageIds] = find_low_images(projectDir);
showFigures = should_show_figures();

reportFile = fullfile(resultsDir, 'B3A_histogram_comments.txt');
reportFid = fopen(reportFile, 'w', 'n', 'UTF-8');
if reportFid == -1
    error('Δεν ήταν δυνατή η δημιουργία του αρχείου: %s', reportFile);
end
reportCleanup = onCleanup(@( ) fclose(reportFid));

fprintf(reportFid, 'Μέρος A: Ανάλυση εικόνων χαμηλού φωτισμού\n');
fprintf(reportFid, 'Φάκελος εικόνων: %s\n', projectDir);
fprintf(reportFid, 'Φάκελος αποτελεσμάτων: %s\n\n', resultsDir);
fprintf(reportFid, 'Κανόνες ονομάτων: *_low.png είναι η σκοτεινή εικόνα και *_high.png η φωτεινή αναφορά.\n');
fprintf(reportFid, 'Στο Μέρος A αναλύονται μόνο οι εικόνες *_low.png.\n\n');

imageNames = cell(numel(imageFiles), 1);
imageNumbers = zeros(numel(imageFiles), 1);
highReferenceNames = cell(numel(imageFiles), 1);
highReferenceFound = false(numel(imageFiles), 1);
meanBrightness = zeros(numel(imageFiles), 1);
intensityVariance = zeros(numel(imageFiles), 1);
stdContrast = zeros(numel(imageFiles), 1);
minIntensity = zeros(numel(imageFiles), 1);
maxIntensity = zeros(numel(imageFiles), 1);
darkPixelPercent = zeros(numel(imageFiles), 1);
brightPixelPercent = zeros(numel(imageFiles), 1);
michelsonContrast = zeros(numel(imageFiles), 1);
histogramComments = cell(numel(imageFiles), 1);

fprintf('Μέρος A: βρέθηκαν %d εικόνες χαμηλού φωτισμού.\n', numel(imageFiles));
fprintf('Τα αποτελέσματα θα αποθηκευτούν εδώ:\n%s\n\n', resultsDir);

for idx = 1:numel(imageFiles)
    imagePath = fullfile(projectDir, imageFiles(idx).name);
    [inputImage, isColorImage] = read_image_as_double(imagePath);

    if isColorImage
        grayImage = rgb2gray(inputImage);
    else
        grayImage = inputImage;
    end

    stats = compute_gray_stats(grayImage);
    commentText = make_histogram_comment(stats.Mean, stats.Std, stats.DarkPercent, stats.BrightPercent);

    highReferenceName = sprintf('%d_high.png', imageIds(idx));
    highReferencePath = fullfile(projectDir, highReferenceName);

    imageNames{idx} = imageFiles(idx).name;
    imageNumbers(idx) = imageIds(idx);
    highReferenceFound(idx) = exist(highReferencePath, 'file') == 2;
    if highReferenceFound(idx)
        highReferenceNames{idx} = highReferenceName;
    else
        highReferenceNames{idx} = '';
    end

    meanBrightness(idx) = stats.Mean;
    intensityVariance(idx) = stats.Variance;
    stdContrast(idx) = stats.Std;
    minIntensity(idx) = stats.Min;
    maxIntensity(idx) = stats.Max;
    darkPixelPercent(idx) = stats.DarkPercent;
    brightPixelPercent(idx) = stats.BrightPercent;
    michelsonContrast(idx) = stats.Michelson;
    histogramComments{idx} = commentText;

    [~, baseName] = fileparts(imageFiles(idx).name);
    safeBaseName = make_safe_filename(baseName);
    outputPrefix = sprintf('%02d_%s', idx, safeBaseName);

    imwrite(inputImage, fullfile(resultsDir, [outputPrefix, '_original.png']));
    imwrite(grayImage, fullfile(resultsDir, [outputPrefix, '_grayscale.png']));

    grayCounts = intensity_histogram(grayImage);
    if isColorImage

```

```

        redCounts = intensity_histogram(inputImage(:, :, 1));
        greenCounts = intensity_histogram(inputImage(:, :, 2));
        blueCounts = intensity_histogram(inputImage(:, :, 3));
    else
        redCounts = zeros(1, 256);
        greenCounts = zeros(1, 256);
        blueCounts = zeros(1, 256);
    end

    histogramTable = table(0:255, grayCounts(:), redCounts(:), greenCounts(:), blueCounts(:), 'VariableNames', {'IntensityLevel',
'GrayCount', 'RedCount', 'GreenCount', 'BlueCount'});
    writetable(histogramTable, fullfile(resultsDir, [outputPrefix, '_histograms.csv']));

    if showFigures
        hAnalysis = figure('Name', ['B3A - ', imageFiles(idx).name], 'Color', 'w', 'Visible', 'on', 'Position', [80 80 1250 900]);
        tiledlayout(hAnalysis, 2, 2, 'Padding', 'loose', 'TileSpacing', 'compact');

        nexttile;
        imshow(inputImage);
        title('Αρχική εικόνα', 'Interpreter', 'none');

        nexttile;
        imshow(grayImage);
        title('Grayscale');

        nexttile;
        bar(0:255, grayCounts, 1, 'FaceColor', [0.25 0.25 0.25], 'EdgeColor', 'none');
        xlim([0 255]);
        grid on;
        title('Ιστογράμμο φωτεινότητας');
        xlabel('Intensity level');
        ylabel('Pixels');

        nexttile;
        if isColorImage
            plot(0:255, redCounts, 'r', 'LineWidth', 1.1);
            hold on;
            plot(0:255, greenCounts, 'g', 'LineWidth', 1.1);
            plot(0:255, blueCounts, 'b', 'LineWidth', 1.1);
            hold off;
            legend({'R', 'G', 'B'}, 'Location', 'northeast');
            title('RGB histograms');
        else
            plot(0:255, grayCounts, 'k', 'LineWidth', 1.1);
            title('Ιστογράμμο');
        end
        xlim([0 255]);
        grid on;
        xlabel('Intensity level');
        ylabel('Pixels');

        sgtitle(sprintf('Μέσος Α - %s | μέση φωτεινότητα %.4f, διασπορά %.5f', imageFiles(idx).name, stats.Mean, stats.Variance),
'Interpreter', 'none', 'Color', 'k', 'FontSize', 12);
        save_figure_png(hAnalysis, fullfile(resultsDir, [outputPrefix, '_analysis.png']));
    end

    fprintf(reportFid, 'Εικόνα %02d: %s\n', idx, imageFiles(idx).name);
    fprintf(reportFid, 'Αντίστοιχη φωτεινή εικόνα: %s\n', value_or_none(highReferenceNames{idx}));
    fprintf(reportFid, 'Μέση φωτεινότητα: %.6f\n', stats.Mean);
    fprintf(reportFid, 'Διασπορά έντασης: %.6f\n', stats.Variance);
    fprintf(reportFid, 'Τυπική απόκλιση/εκτίμηση αντίθεσης: %.6f\n', stats.Std);
    fprintf(reportFid, 'Ελάχιστη ένταση: %.6f\n', stats.Min);
    fprintf(reportFid, 'Μέγιστη ένταση: %.6f\n', stats.Max);
    fprintf(reportFid, 'Σκοτεινά pixels <= 0.25: %.2f%%\n', stats.DarkPercent);
    fprintf(reportFid, 'Φωτεινά pixels >= 0.75: %.2f%%\n', stats.BrightPercent);
    fprintf(reportFid, 'Michelson contrast: %.6f\n', stats.Michelson);
    fprintf(reportFid, 'Σχόλιο: %s\n\n', commentText);

    fprintf('%02d/%02d %s: μέση φωτεινότητα=%.4f, διασπορά=%.5f, σκοτεινά pixels=%.1f%%\n', idx, numel(imageFiles), imageFiles(idx).name,
stats.Mean, stats.Variance, stats.DarkPercent);
    drawnow;
end

statsTable = table(imageNames, imageNumbers, highReferenceNames, highReferenceFound, meanBrightness, intensityVariance, stdContrast,
minIntensity, maxIntensity, darkPixelPercent, brightPixelPercent, michelsonContrast, histogramComments, 'VariableNames', {'Image',
'ImageNumber', 'HighReferenceImage', 'HighReferenceFound', 'MeanBrightness', 'IntensityVariance', 'StdContrast', 'MinIntensity',
'MaxIntensity', 'DarkPixelPercent', 'BrightPixelPercent', 'MichelsonContrast', 'HistogramComment'});
writetable(statsTable, fullfile(resultsDir, 'B3A_statistics.csv'));

if showFigures
    hSummary = figure('Name', 'B3A - Summary statistics', 'Color', 'w', 'Visible', 'on', 'Position', [120 120 1300 900]);
    tiledlayout(hSummary, 3, 1, 'Padding', 'loose', 'TileSpacing', 'compact');

    nexttile;
    bar(meanBrightness, 'FaceColor', [0.20 0.45 0.75]);
    ylim([0 1]);
    grid on;
    title('Mean brightness');
    ylabel('Mean');
    set_image_axis_labels(imageNames);

```

```

nexttile;
bar(intensityVariance, 'FaceColor', [0.65 0.35 0.15]);
grid on;
title('Intensity variance');
ylabel('Variance');
set_image_axis_labels(imageNames);

nexttile;
bar(darkPixelPercent, 'FaceColor', [0.25 0.25 0.25]);
ylim([0 100]);
grid on;
title('Dark pixels [0, 0.25]');
ylabel('Dark pixels (%)');
set_image_axis_labels(imageNames);

sgtitle('Μέρος Α - Συνοπτικά στατιστικά', 'Color', 'k', 'FontSize', 12);
save_figure_png(hSummary, fullfile(resultsDir, 'B3A_summary_statistics.png'));
end

fprintf('\nΤο Μέρος Α ολοκληρώθηκε.\n');
if showFigures
    fprintf('Αποθηκεύτηκαν εικόνες, ιστογράμματα, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);
else
    fprintf('Αποθηκεύτηκαν εικόνες, ιστογράμματα, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);
end

function [imageFiles, imageIds] = find_low_images(projectDir)
    lowFiles = dir(fullfile(projectDir, '*_low.png'));
    keepFile = false(numel(lowFiles), 1);
    imageIds = nan(numel(lowFiles), 1);

    for fileIdx = 1:numel(lowFiles)
        tokens = regexp(lowFiles(fileIdx).name, '^(d+)_low.png$', 'tokens', 'once');
        if ~isempty(tokens)
            keepFile(fileIdx) = true;
            imageIds(fileIdx) = str2double(tokens{1});
        end
    end

    imageFiles = lowFiles(keepFile);
    imageIds = imageIds(keepFile);

    if isempty(imageFiles)
        error('Δεν βρέθηκαν εικόνες με μορφή ονόματος number_low.png στον φάκελο: %s', projectDir);
    end

    [imageIds, sortOrder] = sort(imageIds);
    imageFiles = imageFiles(sortOrder);
end

function showFigures = should_show_figures()
    try
        showFigures = usejava('desktop') && desktop('-inuse');
    catch
        showFigures = usejava('desktop');
    end
end

function [imageDouble, isColorImage] = read_image_as_double(imagePath)
    [rawImage, colorMap] = imread(imagePath);
    if ~isempty(colorMap)
        imageDouble = ind2rgb(rawImage, colorMap);
    else
        imageDouble = im2double(rawImage);
    end
    if ndims(imageDouble) == 3 && size(imageDouble, 3) > 3
        imageDouble = imageDouble(:, :, 1:3);
    end
    isColorImage = ndims(imageDouble) == 3 && size(imageDouble, 3) == 3;
end

function stats = compute_gray_stats(grayImage)
    pixels = grayImage(:);
    stats.Mean = mean(pixels);
    stats.Variance = mean((pixels - stats.Mean) .^ 2);
    stats.Std = sqrt(stats.Variance);
    stats.Min = min(pixels);
    stats.Max = max(pixels);
    stats.DarkPercent = 100 * mean(pixels <= 0.25);
    stats.BrightPercent = 100 * mean(pixels >= 0.75);
    stats.Michelson = (stats.Max - stats.Min) / (stats.Max + stats.Min + eps);
end

function counts = intensity_histogram(channelImage)
    channelImage = min(max(channelImage, 0), 1);
    uintValues = uint8(round(255 * channelImage));
    counts = histcounts(double(uintValues(:)), -0.5:1:255.5);
end

```

```

function commentText = make_histogram_comment(mu, sigmaValue, darkPercent, brightPercent)
    if darkPercent >= 65 || mu < 0.30
        brightnessComment = 'Το ιστόγραμμα είναι συγκεντρωμένο στις χαμηλές στάθμες, άρα η εικόνα είναι έντονα υποφωτισμένη.';
    elseif darkPercent >= 40 || mu < 0.45
        brightnessComment = 'Το ιστόγραμμα μετατοπίζεται προς χαμηλές και μεσαίες στάθμες, άρα η φωτεινότητα είναι μειωμένη.';
    else
        brightnessComment = 'Το ιστόγραμμα δεν είναι συγκεντρωμένο αποκλειστικά στις σκοτεινές στάθμες.';
    end

    if sigmaValue < 0.12
        contrastComment = 'Η μικρή διασπορά δείχνει χαμηλή αντίθεση.';
    elseif sigmaValue < 0.22
        contrastComment = 'Η διασπορά δείχνει μέτρια αντίθεση.';
    else
        contrastComment = 'Η διασπορά είναι σχετικά υψηλή, πιθανώς λόγω τοπικών φωτεινών πηγών.';
    end

    if brightPercent > 8
        highlightComment = 'Υπάρχει και φωτεινή ουρά στο ιστόγραμμα, πιθανώς από έντονες πηγές φωτός.';
    else
        highlightComment = 'Λίγα pixels βρίσκονται στις υψηλές στάθμες φωτεινότητας.';
    end

    commentText = [brightnessComment, ' ', contrastComment, ' ', highlightComment];
end

function safeName = make_safe_filename(fileName)
    safeName = regexprep(fileName, ['^A-Za-z0-9]+' , '_');
    safeName = regexprep(safeName, '^+|_+$' , '');
    if isempty(safeName)
        safeName = 'image';
    end
end

function textValue = value_or_none(inputText)
    if isempty(inputText)
        textValue = 'δεν βρέθηκε';
    else
        textValue = inputText;
    end
end

function set_image_axis_labels(imageNames)
    ax = gca;
    ax.XTick = 1:numel(imageNames);
    ax.XTickLabel = imageNames;
    ax.TickLabelInterpreter = 'none';
    xtickangle(45);
end

function save_figure_png(figureHandle, outputPath)
    set(figureHandle, 'Color', 'w', 'InvertHardcopy', 'off');
    axesHandles = findall(figureHandle, 'Type', 'axes');
    for idx = 1:numel(axesHandles)
        set(axesHandles(idx), 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'ZColor', 'k');
        set(get(axesHandles(idx), 'Title'), 'Color', 'k');
        set(get(axesHandles(idx), 'XLabel'), 'Color', 'k');
        set(get(axesHandles(idx), 'YLabel'), 'Color', 'k');
        set(get(axesHandles(idx), 'ZLabel'), 'Color', 'k');
    end
    textHandles = findall(figureHandle, 'Type', 'text');
    if ~isempty(textHandles)
        set(textHandles, 'Color', 'k');
    end
    legendHandles = findall(figureHandle, 'Type', 'legend');
    if ~isempty(legendHandles)
        set(legendHandles, 'TextColor', 'k', 'Color', 'w', 'EdgeColor', 'k');
    end
    drawnow;
    print(figureHandle, outputPath, '-dpng', '-r200');
end

```

Μέρος Β (Αρχείο eikonaB3B.m)

```

clear;
clc;
close all;

```

% Για να επεξεργαστεί μια εικόνα, το όνομά της πρέπει να τελειώνει σε _low.png.

```

greekDesktop = char([933 960 959 955 959 947 953 963 964 942 962]);
% Replace with your path
projectDir = fullfile(getenv('USERPROFILE'), 'OneDrive', greekDesktop, 'project');
resultsDir = fullfile(projectDir, 'resultsB3B');
expectedFolderName = 'resultsB3B';

[~, actualFolderName] = fileparts(resultsDir);
if isempty(resultsDir) || strcmp(resultsDir, projectDir) || ~strcmp(actualFolderName, expectedFolderName)
    error('Unsafe results folder: %s', resultsDir);
end

if ~exist(resultsDir, 'dir')
    mkdir(resultsDir);
end

oldFiles = [dir(fullfile(resultsDir, '*.png')); dir(fullfile(resultsDir, '*.csv')); dir(fullfile(resultsDir, '*.txt'))];
dir(fullfile(resultsDir, '*.mat'));
oldWarningState = warning('off', 'all');
for oldIdx = 1:numel(oldFiles)
    oldPath = fullfile(resultsDir, oldFiles(oldIdx).name);
    if exist(oldPath, 'file') == 2
        try
            delete(oldPath);
        catch
            end
    end
end
warning(oldWarningState);

[imageFiles, imageIds] = find_low_images(projectDir);
showFigures = should_show_figures();
gammaValues = [0.4 0.6 0.8 1.2];

reportFile = fullfile(resultsDir, 'B3B_transform_comments.txt');
reportFid = fopen(reportFile, 'w', 'n', 'UTF-8');
if reportFid == -1
    error('Δεν ήταν δυνατή η δημιουργία του αρχείου: %s', reportFile);
end
reportCleanup = onCleanup(@() fclose(reportFid));

fprintf(reportFid, 'Μέρος B: Γραμμικοί και μη γραμμικοί μετασχηματισμοί φωτεινότητας\n');
fprintf(reportFid, 'Θάκελος εικόνων: %s\n', projectDir);
fprintf(reportFid, 'Θάκελος αποτελεσμάτων: %s\n', resultsDir);
fprintf(reportFid, '*_low.png είναι η σκοτεινή εικόνα και *_high.png η φωτεινή reference.\n');
fprintf(reportFid, 'Στο Μέρος B εφαρμόζονται μετασχηματισμοί μόνο στις εικόνες *_low.png.\n');
fprintf(reportFid, 'Γενικά σχόλια:\n');
fprintf(reportFid, 'Η γραμμική επέκταση δυναμικού εύρους βοηθά όταν η εικόνα έχει χρήσιμη πληροφορία σε μικρό εύρος εντάσεων, αλλά μπορεί να κάνει πιο εμφανή και τον θόρυβο.\n');
fprintf(reportFid, 'Για gamma < 1 οι σκοτεινές περιοχές φωτίζονται περισσότερο. Όσο μικρότερη είναι η gamma, τόσο εντονότερη είναι η ενίσχυση και τόσο μεγαλύτερος ο κίνδυνος υπερενίσχυσης θορύβου.\n');
fprintf(reportFid, 'Για gamma = 1.2 η εικόνα συνήθως σκοτεινιάζει, άρα η τιμή αυτή λειτουργεί κυρίως συγκριτικά.\n');
fprintf(reportFid, 'Ο λογαριθμικός μετασχηματισμός φωτίζει τις σκιές και συμπιέζει τις υψηλές εντάσεις, συνήθως πιο ήπια από gamma 0.4.\n');

totalRows = numel(imageFiles) * (3 + numel(gammaValues));
emptyRow = struct('Image', '', 'ImageNumber', 0, 'HighReferenceImage', '', 'Method', '', 'Parameter', '', 'MeanBrightness', 0, 'IntensityVariance', 0, 'StdContrast', 0, 'MinIntensity', 0, 'MaxIntensity', 0, 'DarkPixelPercent', 0, 'BrightPixelPercent', 0, 'SaturatedPixelPercent', 0, 'MichelsonContrast', 0, 'Comment', '');
resultRows = repmat(emptyRow, totalRows, 1);
rowIdx = 0;

fprintf('Μέρος B: βρέθηκαν %d εικόνες χαμηλού φωτισμού.\n', numel(imageFiles));
fprintf('Τα αποτελέσματα θα αποθηκευτούν εδώ:\n%s\n', resultsDir);

for idx = 1:numel(imageFiles)
    imagePath = fullfile(projectDir, imageFiles(idx).name);
    [inputImage, isColorImage] = read_image_as_double(imagePath);

    [~, baseName] = fileparts(imageFiles(idx).name);
    safeBaseName = make_safe_filename(baseName);
    outputPrefix = sprintf('%02d_%s', idx, safeBaseName);

    highReferenceName = sprintf('%d_high.png', imageIds(idx));
    if exist(fullfile(projectDir, highReferenceName), 'file') ~= 2
        highReferenceName = '';
    end

    originalImage = inputImage;
    linearImage = linear_contrast_stretch(inputImage);
    logImage = logarithmic_transform(inputImage);
    gammaImages = cell(numel(gammaValues), 1);

    for gammaIdx = 1:numel(gammaValues)
        gammaImages{gammaIdx} = gamma_transform(inputImage, gammaValues(gammaIdx));
    end

    imwrite(originalImage, fullfile(resultsDir, [outputPrefix, '_input_low.png']));
    imwrite(linearImage, fullfile(resultsDir, [outputPrefix, '_linear_stretch.png']));
    for gammaIdx = 1:numel(gammaValues)
        gammaLabel = strrep(sprintf('%.1f', gammaValues(gammaIdx)), '.', '_');

```

```

        imwrite(gammaImages{gammaIdx}, fullfile(resultsDir, [outputPrefix, '_gamma_', gammaLabel, '.png']));
    end
    imwrite(logImage, fullfile(resultsDir, [outputPrefix, '_log_transform.png']));

    rowIdx = rowIdx + 1;
    resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Original low-light', '', originalImage,
    isColorImage, 'Αρχική εικόνα χαμηλού φωτισμού πριν από τους μετασχηματισμούς.');
```

```

    rowIdx = rowIdx + 1;
    resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Linear stretch', 'full range [0,1]',
    linearImage, isColorImage, make_linear_comment(originalImage, linearImage, isColorImage));

    for gammaIdx = 1:numel(gammaValues)
        gammaValue = gammaValues(gammaIdx);
        rowIdx = rowIdx + 1;
        resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Gamma correction', sprintf('gamma =
        %.1f', gammaValue), gammaImages{gammaIdx}, isColorImage, make_gamma_comment(gammaValue, originalImage, gammaImages{gammaIdx}, isColorImage));
    end

    rowIdx = rowIdx + 1;
    resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Log transform', 'log(1+I)/log(2)',
    logImage, isColorImage, make_log_comment(originalImage, logImage, gammaImages{1}, isColorImage));

    fprintf(reportFid, 'Εικόνα %02d: %s\n', idx, imageFiles(idx).name);
    fprintf(reportFid, 'Αντιστοίχη φωτεινής εικόνας: %s\n', value_or_none(highReferenceName));
    fprintf(reportFid, 'Linear stretch: %s\n', make_linear_comment(originalImage, linearImage, isColorImage));
    for gammaIdx = 1:numel(gammaValues)
        fprintf(reportFid, 'Gamma %.1f: %s\n', gammaValues(gammaIdx), make_gamma_comment(gammaValues(gammaIdx), originalImage,
        gammaImages{gammaIdx}, isColorImage));
    end
    fprintf(reportFid, 'Log transform: %s\n\n', make_log_comment(originalImage, logImage, gammaImages{1}, isColorImage));

    if showFigures
        hComparison = figure('Name', ['B3B - ', imageFiles(idx).name], 'Color', 'w', 'Visible', 'on', 'Position', [45 45 1500 980]);
        comparisonTitle = sprintf('Μέρος B - Σύγκριση μετασχηματισμών φωτεινότητας: %s', imageFiles(idx).name);
        annotation(hComparison, 'textbox', [0.02 0.94 0.96 0.045], 'String', comparisonTitle, 'Interpreter', 'none', 'HorizontalAlignment',
        'center', 'VerticalAlignment', 'middle', 'EdgeColor', 'none', 'Color', 'k', 'FontSize', 12, 'FontWeight', 'bold');
        comparisonLayout = tiledlayout(hComparison, 2, 4, 'Padding', 'loose', 'TileSpacing', 'loose');
        comparisonLayout.Units = 'normalized';
        comparisonLayout.Position = [0.035 0.055 0.93 0.86];

        show_image_tile(originalImage, 'Original low-light');
        show_image_tile(linearImage, 'Linear stretch');
        show_image_tile(gammaImages{1}, 'Gamma 0.4');
        show_image_tile(gammaImages{2}, 'Gamma 0.6');
        show_image_tile(gammaImages{3}, 'Gamma 0.8');
        show_image_tile(gammaImages{4}, 'Gamma 1.2');
        show_image_tile(logImage, 'Log transform');

        nexttile;
        axis off;
        originalStats = compute_image_stats(originalImage, isColorImage);
        linearStats = compute_image_stats(linearImage, isColorImage);
        gamma04Stats = compute_image_stats(gammaImages{1}, isColorImage);
        logStats = compute_image_stats(logImage, isColorImage);
        text(0.02, 0.78, sprintf('Mean original: %.3f', originalStats.Mean), 'FontSize', 11);
        text(0.02, 0.60, sprintf('Mean linear: %.3f', linearStats.Mean), 'FontSize', 11);
        text(0.02, 0.42, sprintf('Mean gamma 0.4: %.3f', gamma04Stats.Mean), 'FontSize', 11);
        text(0.02, 0.24, sprintf('Mean log: %.3f', logStats.Mean), 'FontSize', 11);
        text(0.02, 0.06, sprintf('Dark pixels after linear: %.1f%%', linearStats.DarkPercent), 'FontSize', 11);

        save_figure_png(hComparison, fullfile(resultsDir, [outputPrefix, '_comparison.png']));
    end

    originalStats = compute_image_stats(originalImage, isColorImage);
    linearStats = compute_image_stats(linearImage, isColorImage);
    gamma04Stats = compute_image_stats(gammaImages{1}, isColorImage);
    logStats = compute_image_stats(logImage, isColorImage);

    fprintf('%02d/%02d %s: μέση φωτεινότητα %.4f -> linear %.4f, gamma 0.4 %.4f, log %.4f\n', idx, numel(imageFiles), imageFiles(idx).name,
    originalStats.Mean, linearStats.Mean, gamma04Stats.Mean, logStats.Mean);
    drawnow;
end

resultRows = resultRows(1:rowIdx);
statsTable = struct2table(resultRows);
writetable(statsTable, fullfile(resultsDir, 'B3B_statistics.csv'));

if showFigures
    hSummary = figure('Name', 'B3B - Summary statistics', 'Color', 'w', 'Visible', 'on', 'Position', [120 120 1350 900]);
    methodOrder = {'Original low-light', 'Linear stretch', 'Gamma correction', 'Log transform'};
    tiledlayout(hSummary, 2, 1, 'Padding', 'loose', 'TileSpacing', 'compact');

    nexttile;
    plot_summary_by_method(statsTable, methodOrder, 'MeanBrightness', 'Mean brightness');

    nexttile;
    plot_summary_by_method(statsTable, methodOrder, 'StdContrast', 'Std contrast');

    sgtitle('Μέρος B - Συνοπτική σύγκριση μεθόδων', 'Color', 'k', 'FontSize', 12, 'FontWeight', 'bold');
```

```

    save_figure_png(hSummary, fullfile(resultsDir, 'B3B_summary_statistics.png'));
end

fprintf('\nΤο Μέρος Β ολοκληρώθηκε.\n');
if showFigures
    fprintf('Αποθηκεύτηκαν μετασχηματισμένες εικόνες, figures, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);
else
    fprintf('Αποθηκεύτηκαν μετασχηματισμένες εικόνες, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);
end

function [imageFiles, imageIds] = find_low_images(projectDir)
    lowFiles = dir(fullfile(projectDir, '*_low.png'));
    keepFile = false(numel(lowFiles), 1);
    imageIds = nan(numel(lowFiles), 1);

    for fileIdx = 1:numel(lowFiles)
        tokens = regexp(lowFiles(fileIdx).name, '^(\\d+)_low\\.png$', 'tokens', 'once');
        if ~isempty(tokens)
            keepFile(fileIdx) = true;
            imageIds(fileIdx) = str2double(tokens{1});
        end
    end

    imageFiles = lowFiles(keepFile);
    imageIds = imageIds(keepFile);

    if isempty(imageFiles)
        error('Δεν βρέθηκαν εικόνες με μορφή ονόματος number_low.png στον φάκελο: %s', projectDir);
    end

    [imageIds, sortOrder] = sort(imageIds);
    imageFiles = imageFiles(sortOrder);
end

function showFigures = should_show_figures()
    try
        showFigures = usejava('desktop') && desktop('-inuse');
    catch
        showFigures = usejava('desktop');
    end
end

function [imageDouble, isColorImage] = read_image_as_double(imagePath)
    [rawImage, colorMap] = imread(imagePath);
    if ~isempty(colorMap)
        imageDouble = ind2rgb(rawImage, colorMap);
    else
        imageDouble = im2double(rawImage);
    end
    if ndims(imageDouble) == 3 && size(imageDouble, 3) > 3
        imageDouble = imageDouble(:, :, 1:3);
    end
    isColorImage = ndims(imageDouble) == 3 && size(imageDouble, 3) == 3;
end

function outputImage = linear_contrast_stretch(inputImage)
    inputMin = min(inputImage(:));
    inputMax = max(inputImage(:));
    if inputMax <= inputMin
        outputImage = zeros(size(inputImage));
    else
        outputImage = (inputImage - inputMin) / (inputMax - inputMin);
    end
end

function outputImage = gamma_transform(inputImage, gammaValue)
    outputImage = min(max(inputImage, 0), 1) .^ gammaValue;
end

function outputImage = logarithmic_transform(inputImage)
    outputImage = log(1 + min(max(inputImage, 0), 1)) / log(2);
end

function stats = compute_image_stats(inputImage, isColorImage)
    if isColorImage
        grayImage = rgb2gray(inputImage);
    else
        grayImage = inputImage;
    end
    pixels = grayImage(:);
    stats.Mean = mean(pixels);
    stats.Variance = mean((pixels - stats.Mean) .^ 2);
    stats.Std = sqrt(stats.Variance);
    stats.Min = min(pixels);
    stats.Max = max(pixels);
    stats.DarkPercent = 100 * mean(pixels <= 0.25);
    stats.BrightPercent = 100 * mean(pixels >= 0.75);
    stats.SaturatedPercent = 100 * mean(pixels >= 0.98);
    stats.Michelson = (stats.Max - stats.Min) / (stats.Max + stats.Min + eps);
end

```

```

function row = make_result_row(imageName, imageNumber, highReferenceName, methodName, parameterText, resultImage, isColorImage, commentText)
    stats = compute_image_stats(resultImage, isColorImage);
    row = struct('Image', imageName, 'ImageNumber', imageNumber, 'HighReferenceImage', highReferenceName, 'Method', methodName, 'Parameter',
parameterText, 'MeanBrightness', stats.Mean, 'IntensityVariance', stats.Variance, 'StdContrast', stats.Std, 'MinIntensity', stats.Min,
'MaxIntensity', stats.Max, 'DarkPixelPercent', stats.DarkPercent, 'BrightPixelPercent', stats.BrightPercent, 'SaturatedPixelPercent',
stats.SaturatedPercent, 'MichelsonContrast', stats.Michelson, 'Comment', commentText);
end

function commentText = make_linear_comment(originalImage, linearImage, isColorImage)
    originalStats = compute_image_stats(originalImage, isColorImage);
    linearStats = compute_image_stats(linearImage, isColorImage);
    if linearStats.SaturatedPercent > originalStats.SaturatedPercent + 2
        highlightPart = 'Παράλληλα αυξάνονται τα σχεδόν κορεσμένα pixels, άρα οι φωτεινές πηγές μπορεί να γίνουν υπερβολικές.';
    else
        highlightPart = 'Δεν δημιουργείται έντονος κορεσμός στις φωτεινές περιοχές.';
    end
    if linearStats.Std > originalStats.Std
        contrastPart = 'Η αντίθεση αυξάνεται επειδή το εύρος εντάσεων απλώνεται στο πλήρες δυναμικό εύρος.';
    else
        contrastPart = 'Η αντίθεση δεν αυξάνεται έντονα επειδή το αρχικό εύρος ήταν ήδη αρκετά μεγάλο.';
    end
    commentText = [contrastPart, ' Ο θόρυβος στις σκοτεινές περιοχές ενισχύεται μαζί με τη χρήσιμη λεπτομέρεια. ', highlightPart];
end

function commentText = make_gamma_comment(gammaValue, originalImage, gammaImage, isColorImage)
    originalStats = compute_image_stats(originalImage, isColorImage);
    gammaStats = compute_image_stats(gammaImage, isColorImage);
    meanChange = gammaStats.Mean - originalStats.Mean;
    if gammaValue < 1
        if gammaValue <= 0.4
            strengthPart = 'Η μικρή τιμή gamma δίνει ισχυρή φωτεινότητα στις σκοτεινές περιοχές.';
        else
            strengthPart = 'Η τιμή gamma φωτίζει τις σκοτεινές περιοχές πιο ήπια.';
        end
    else
        strengthPart = 'Επειδή η gamma είναι μεγαλύτερη από 1, η εικόνα σκοτεινιάζει αντί να βελτιώνεται για χαμηλό φωτισμό.';
    end
    if gammaStats.SaturatedPercent > originalStats.SaturatedPercent + 2
        riskPart = 'Η ενίσχυση μπορεί να κάνει πιο εμφανή τον θόρυβο και τα highlights.';
    else
        riskPart = 'Ο κορεσμός των φωτεινών περιοχών παραμένει περιορισμένος.';
    end
    commentText = sprintf('%s Η μέση φωτεινότητα αλλάζει κατά %.4f. %s', strengthPart, meanChange, riskPart);
end

function commentText = make_log_comment(originalImage, logImage, strongGammaImage, isColorImage)
    originalStats = compute_image_stats(originalImage, isColorImage);
    logStats = compute_image_stats(logImage, isColorImage);
    strongGammaStats = compute_image_stats(strongGammaImage, isColorImage);
    if logStats.Mean < strongGammaStats.Mean
        comparisonPart = 'Σε σύγκριση με gamma 0.4, ο λογαριθμικός μετασχηματισμός είναι πιο ήπιος.';
    else
        comparisonPart = 'Σε αυτή την εικόνα ο λογαριθμικός μετασχηματισμός πλησιάζει ή ξεπερνά τη φωτεινότητα του gamma 0.4.';
    end
    commentText = sprintf('Αveβάζει τη μέση φωτεινότητα από %.4f σε %.4f και συμπιέζει τις υψηλές εντάσεις. %s', originalStats.Mean,
logStats.Mean, comparisonPart);
end

function safeName = make_safe_filename(fileName)
    safeName = regexprep(fileName, '[^A-Za-z0-9]+', '_');
    safeName = regexprep(safeName, '^_+|_+$', '');
    if isempty(safeName)
        safeName = 'image';
    end
end

function textValue = value_or_none(inputText)
    if isempty(inputText)
        textValue = 'δεν βρέθηκε';
    else
        textValue = inputText;
    end
end

function show_image_tile(inputImage, titleText)
    nexttile;
    imshow(inputImage);
    title(titleText, 'Interpreter', 'none', 'Color', 'k');
end

function plot_summary_by_method(statsTable, methodOrder, metricName, titleText)
    hold on;
    for methodIdx = 1:numel(methodOrder)
        methodName = methodOrder{methodIdx};
        rows = strcmp(statsTable.Method, methodName);
        if strcmp(methodName, 'Gamma correction')
            rows = rows & strcmp(statsTable.Parameter, 'gamma = 0.6');
            displayName = 'Gamma 0.6';
        else

```

```

        displayName = methodName;
    end
    plot(statsTable.ImageNumber(rows), statsTable.(metricName)(rows), '-o', 'LineWidth', 1.2, 'DisplayName', displayName);
end
hold off;
grid on;
xlabel('Image number');
ylabel(metricName);
title(titleText);
legend('Location', 'best');
end

function save_figure_png(figureHandle, outputPath)
    set(figureHandle, 'Color', 'w', 'InvertHardcopy', 'off');
    axesHandles = findall(figureHandle, 'Type', 'axes');
    for idx = 1:numel(axesHandles)
        set(axesHandles(idx), 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'ZColor', 'k');
        set(get(axesHandles(idx), 'Title'), 'Color', 'k');
        set(get(axesHandles(idx), 'XLabel'), 'Color', 'k');
        set(get(axesHandles(idx), 'YLabel'), 'Color', 'k');
        set(get(axesHandles(idx), 'ZLabel'), 'Color', 'k');
    end
    textHandles = findall(figureHandle, 'Type', 'text');
    if ~isempty(textHandles)
        set(textHandles, 'Color', 'k');
    end
    legendHandles = findall(figureHandle, 'Type', 'legend');
    if ~isempty(legendHandles)
        set(legendHandles, 'TextColor', 'k', 'Color', 'w', 'EdgeColor', 'k');
    end
    drawnow;
    print(figureHandle, outputPath, '-dpng', '-r200');
end

```

Μέρος Γ (Αρχείο: eikonaB3C.m)

```

clear;
clc;
close all;

% Για να επεξεργαστεί μια εικόνα, το όνομά της πρέπει να τελειώνει σε _low.png.

greekDesktop = char([933 960 959 955 959 947 953 963 964 942 962]);
% Replace with your path
projectDir = fullfile(getenv('USERPROFILE'), 'OneDrive', greekDesktop, 'project');
resultsDir = fullfile(projectDir, 'resultsB3C');
expectedFolderName = 'resultsB3C';

[~, actualFolderName] = fileparts(resultsDir);
if isempty(resultsDir) || strcmp(resultsDir, projectDir) || ~strcmp(actualFolderName, expectedFolderName)
    error('Unsafe results folder: %s', resultsDir);
end

if ~exist(resultsDir, 'dir')
    mkdir(resultsDir);
end

oldFiles = [dir(fullfile(resultsDir, '*.png')); dir(fullfile(resultsDir, '*.csv')); dir(fullfile(resultsDir, '*.txt'))];
dir(fullfile(resultsDir, '*.mat'));
oldWarningState = warning('off', 'all');
for oldIdx = 1:numel(oldFiles)
    oldPath = fullfile(resultsDir, oldFiles(oldIdx).name);
    if exist(oldPath, 'file') == 2
        try
            delete(oldPath);
        catch
            end
    end
end
warning(oldWarningState);

[imageFiles, imageIds] = find_low_images(projectDir);
showFigures = should_show_figures();
claheSettings = [4 4 0.01; 8 8 0.01; 8 8 0.03; 16 16 0.01];

reportFile = fullfile(resultsDir, 'B3C_histogram_equalization_comments.txt');
reportFid = fopen(reportFile, 'w', 'n', 'UTF-8');
if reportFid == -1
    error('Δεν ήταν δυνατή η δημιουργία του αρχείου: %s', reportFile);
end
reportCleanup = onCleanup(@() fclose(reportFid));

fprintf(reportFid, 'Μέρος Γ: Ισοστάθμιση ιστογράμματος\n');

```

```

fprintf(reportFid, 'Θάκελος εικόνων: %s\n', projectDir);
fprintf(reportFid, 'Θάκελος αποτελεσμάτων: %s\n\n', resultsDir);
fprintf(reportFid, 'Κανόνες ονομάτων: *_low.png είναι η σκοτεινή εικόνα και *_high.png η φωτεινή αναφορά.\n');
fprintf(reportFid, 'H global ισοστάθμιση υλοποιείται από το ιστόγραμμα, την CDF και τον μετασχηματισμό  $s_k = (L-1)CDF(k)$ .\n');
fprintf(reportFid, 'H adaptive ισοστάθμιση εφαρμόζεται ως CLAHE με διαφορετικά tiles και clip limit.\n\n');

rowTemplate = struct('Image', '', 'ImageNumber', 0, 'HighReferenceImage', '', 'Method', '', 'Parameter', '', 'ApproxTileSize', '',
'MeanBrightness', 0, 'IntensityVariance', 0, 'StdContrast', 0, 'EntropyValue', 0, 'DarkPixelPercent', 0, 'BrightPixelPercent', 0,
'SaturatedPixelPercent', 0, 'MichelsonContrast', 0, 'Comment', '');
resultRows = repmat(rowTemplate, numel(imageFiles) * (2 + size(claheSettings, 1)), 1);
rowIdx = 0;

fprintf('Μέρος Γ: βρέθηκαν %d εικόνες χαμηλού φωτισμού.\n', numel(imageFiles));
fprintf('Τα αποτελέσματα θα αποθηκευτούν εδώ:\n%s\n\n', resultsDir);

for idx = 1:numel(imageFiles)
    imagePath = fullfile(projectDir, imageFiles(idx).name);
    [inputImage, isColorImage] = read_image_as_double(imagePath);
    inputBrightness = extract_brightness(inputImage, isColorImage);

    [globalBrightness, histBefore, cdfValues, histGlobal] = global_hist_equalize(inputBrightness);
    globalImage = replace_brightness(inputImage, globalBrightness, isColorImage);

    claheImages = cell(size(claheSettings, 1), 1);
    claheBrightness = cell(size(claheSettings, 1), 1);
    histClahe = cell(size(claheSettings, 1), 1);

    for settingIdx = 1:size(claheSettings, 1)
        numTiles = claheSettings(settingIdx, 1:2);
        clipLimit = claheSettings(settingIdx, 3);
        claheBrightness{settingIdx} = adapthisteq(inputBrightness, 'NumTiles', numTiles, 'ClipLimit', clipLimit, 'Distribution', 'uniform');
        claheImages{settingIdx} = replace_brightness(inputImage, claheBrightness{settingIdx}, isColorImage);
        histClahe{settingIdx} = intensity_histogram(claheBrightness{settingIdx});
    end

    histInput = histBefore;
    cdfTable = table((0:255)', histInput(:), cdfValues(:), histGlobal(:), 'VariableNames', {'IntensityLevel', 'InputHistogram', 'InputCDF',
'GlobalEqualizedHistogram'});

    [~, baseName] = fileparts(imageFiles(idx).name);
    safeBaseName = make_safe_filename(baseName);
    outputPrefix = sprintf('%02d_%s', idx, safeBaseName);

    highReferenceName = sprintf('%d_high.png', imageIds(idx));
    if exist(fullfile(projectDir, highReferenceName), 'file') ~= 2
        highReferenceName = '';
    end

    imwrite(inputImage, fullfile(resultsDir, [outputPrefix, '_input_low.png']));
    imwrite(globalImage, fullfile(resultsDir, [outputPrefix, '_global_histeq.png']));

    rowIdx = rowIdx + 1;
    resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Original low-light', '', '',
inputBrightness, 'Αρχική εικόνα χαμηλού φωτισμού πριν από την ισοστάθμιση.');
```

```

    rowIdx = rowIdx + 1;
    globalComment = make_global_comment(inputBrightness, globalBrightness);
    resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Global histogram equalization', 'CDF
mapping', '', globalBrightness, globalComment);

    for settingIdx = 1:size(claheSettings, 1)
        numTiles = claheSettings(settingIdx, 1:2);
        clipLimit = claheSettings(settingIdx, 3);
        tileSizeText = make_tile_size_text(size(inputBrightness), numTiles);
        parameterText = sprintf('NumTiles=[%d %d], ClipLimit=%.2f', numTiles(1), numTiles(2), clipLimit);
        outputName = sprintf('%s_clahe_%dx%d_clip_%s.png', outputPrefix, numTiles(1), numTiles(2), strrep(sprintf('%02f', clipLimit), '.',
'_'));
        imwrite(claheImages{settingIdx}, fullfile(resultsDir, outputName));

        cdfTable.(sprintf('CLAHE_%dx%d_clip_%s_Histogram', numTiles(1), numTiles(2), strrep(sprintf('%02f', clipLimit), '.', '_'))) =
histClahe{settingIdx}(:);
        if settingIdx == size(claheSettings, 1)
            writetable(cdfTable, fullfile(resultsDir, [outputPrefix, '_histograms_cdf.csv']));
        end

        rowIdx = rowIdx + 1;
        claheComment = make_clahe_comment(inputBrightness, globalBrightness, claheBrightness{settingIdx}, numTiles, clipLimit);
        resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Adaptive histogram equalization',
parameterText, tileSizeText, claheBrightness{settingIdx}, claheComment);
    end

    fprintf(reportFid, 'Εικόνα %02d: %s\n', idx, imageFiles(idx).name);
    fprintf(reportFid, 'Αντίστοιχη φωτεινή εικόνα: %s\n', value_or_none(highReferenceName));
    fprintf(reportFid, 'Global histogram equalization: %s\n', globalComment);
    for settingIdx = 1:size(claheSettings, 1)
        numTiles = claheSettings(settingIdx, 1:2);
        clipLimit = claheSettings(settingIdx, 3);
        fprintf(reportFid, 'CLAHE NumTiles=[%d %d], ClipLimit=%.2f: %s\n', numTiles(1), numTiles(2), clipLimit,
make_clahe_comment(inputBrightness, globalBrightness, claheBrightness{settingIdx}, numTiles, clipLimit));
    end

```

```

end
fprintf(reportFid, '\n');

if showFigures
    hGlobal = figure('Name', ['B3C Global - ', imageFiles(idx).name], 'Color', 'w', 'Visible', 'on', 'Position', [60 60 1250 900]);
    globalTitle = sprintf('Μέρος Γ1 - Global histogram equalization: %s', imageFiles(idx).name);
    annotation(hGlobal, 'textbox', [0.02 0.94 0.96 0.045], 'String', globalTitle, 'Interpreter', 'none', 'HorizontalAlignment', 'center',
'VerticalAlignment', 'middle', 'EdgeColor', 'none', 'Color', 'k', 'FontSize', 12, 'FontWeight', 'bold');
    globalLayout = tiledlayout(hGlobal, 2, 2, 'Padding', 'loose', 'TileSpacing', 'loose');
    globalLayout.Units = 'normalized';
    globalLayout.Position = [0.055 0.06 0.89 0.84];

    show_image_tile(inputImage, 'Εικόνα πριν');
    show_image_tile(globalImage, 'Εικόνα μετά');
    show_histogram_tile(histBefore, 'Ιστογράμμο πριν');
    show_histogram_tile(histGlobal, 'Ιστογράμμο μετά');
    save_figure_png(hGlobal, fullfile(resultsDir, [outputPrefix, '_global_histeq_comparison.png']));

    hClahe = figure('Name', ['B3C CLAHE - ', imageFiles(idx).name], 'Color', 'w', 'Visible', 'on', 'Position', [45 45 1500 980]);
    claheTitle = sprintf('Μέρος Γ2 - Adaptive histogram equalization: %s', imageFiles(idx).name);
    annotation(hClahe, 'textbox', [0.02 0.94 0.96 0.045], 'String', claheTitle, 'Interpreter', 'none', 'HorizontalAlignment', 'center',
'VerticalAlignment', 'middle', 'EdgeColor', 'none', 'Color', 'k', 'FontSize', 12, 'FontWeight', 'bold');
    claheLayout = tiledlayout(hClahe, 2, 3, 'Padding', 'loose', 'TileSpacing', 'loose');
    claheLayout.Units = 'normalized';
    claheLayout.Position = [0.045 0.055 0.91 0.86];

    show_image_tile(inputImage, 'Original low-light');
    show_image_tile(globalImage, 'Global histeq');
    for settingIdx = 1:size(claheSettings, 1)
        numTiles = claheSettings(settingIdx, 1:2);
        clipLimit = claheSettings(settingIdx, 3);
        show_image_tile(claheImages{settingIdx}, sprintf('CLAHE %dx%d, clip %.2f', numTiles(1), numTiles(2), clipLimit));
    end
    save_figure_png(hClahe, fullfile(resultsDir, [outputPrefix, '_clahe_comparison.png']));
end

inputStats = compute_channel_stats(inputBrightness);
globalStats = compute_channel_stats(globalBrightness);
bestClaheStats = compute_channel_stats(claheBrightness{2});

fprintf('%02d/%02d %s: mean %.4f -> global %.4f, CLAHE 8x8 %.4f | std %.4f -> global %.4f\n', idx, numel(imageFiles),
imageFiles(idx).name, inputStats.Mean, globalStats.Mean, bestClaheStats.Mean, inputStats.Std, globalStats.Std);
drawnow;
end

resultRows = resultRows(1:rowIdx);
statsTable = struct2table(resultRows);
writetable(statsTable, fullfile(resultsDir, 'B3C_statistics.csv'));

if showFigures
    hSummary = figure('Name', 'B3C - Summary statistics', 'Color', 'w', 'Visible', 'on', 'Position', [120 120 1350 900]);
    tiledlayout(hSummary, 2, 1, 'Padding', 'loose', 'TileSpacing', 'compact');

    nexttile;
    plot_summary_by_method(statsTable, 'MeanBrightness', 'Mean brightness');

    nexttile;
    plot_summary_by_method(statsTable, 'StdContrast', 'Std contrast');

    sgtitle('Μέρος Γ - Συνοπτική σύγκριση ισοστάθμισης', 'Color', 'k', 'FontSize', 12, 'FontWeight', 'bold');
    save_figure_png(hSummary, fullfile(resultsDir, 'B3C_summary_statistics.png'));
end

fprintf('\nΤο Μέρος Γ ολοκληρώθηκε.\n');
if showFigures
    fprintf('Αποθηκεύτηκαν εικόνες, figures, CDF/ιστογράμματα, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);
else
    fprintf('Αποθηκεύτηκαν εικόνες, CDF/ιστογράμματα, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);
end

function [imageFiles, imageIds] = find_low_images(projectDir)
    lowFiles = dir(fullfile(projectDir, '*_low.png'));
    keepFile = false(numel(lowFiles), 1);
    imageIds = nan(numel(lowFiles), 1);
    for fileIdx = 1:numel(lowFiles)
        tokens = regexp(lowFiles(fileIdx).name, '^(\\d+)_low\\.png$', 'tokens', 'once');
        if ~isempty(tokens)
            keepFile(fileIdx) = true;
            imageIds(fileIdx) = str2double(tokens{1});
        end
    end
    imageFiles = lowFiles(keepFile);
    imageIds = imageIds(keepFile);
    if isempty(imageFiles)
        error('Δεν βρέθηκαν εικόνες με μορφή ονόματος number_low.png στον φάκελο: %s', projectDir);
    end
    [imageIds, sortOrder] = sort(imageIds);
    imageFiles = imageFiles(sortOrder);
end

```

```

function showFigures = should_show_figures()
    try
        showFigures = usejava('desktop') && desktop('-inuse');
    catch
        showFigures = usejava('desktop');
    end
end

function [imageDouble, isColorImage] = read_image_as_double(imagePath)
    [rawImage, colorMap] = imread(imagePath);
    if ~isempty(colorMap)
        imageDouble = ind2rgb(rawImage, colorMap);
    else
        imageDouble = im2double(rawImage);
    end
    if ndims(imageDouble) == 3 && size(imageDouble, 3) > 3
        imageDouble = imageDouble(:, :, 1:3);
    end
    isColorImage = ndims(imageDouble) == 3 && size(imageDouble, 3) == 3;
end

function brightness = extract_brightness(inputImage, isColorImage)
    if isColorImage
        hsvImage = rgb2hsv(inputImage);
        brightness = hsvImage(:, :, 3);
    else
        brightness = inputImage;
    end
end

function outputImage = replace_brightness(inputImage, brightness, isColorImage)
    brightness = min(max(brightness, 0), 1);
    if isColorImage
        hsvImage = rgb2hsv(inputImage);
        hsvImage(:, :, 3) = brightness;
        outputImage = hsv2rgb(hsvImage);
    else
        outputImage = brightness;
    end
end

function [equalizedChannel, histBefore, cdfValues, histAfter] = global_hist_equalize(channelImage)
    channelImage = min(max(channelImage, 0), 1);
    uintValues = uint8(round(255 * channelImage));
    histBefore = histcounts(double(uintValues(:)), -0.5:1:255.5);
    probabilities = histBefore / sum(histBefore);
    cdfValues = cumsum(probabilities);
    equalizedChannel = reshape(cdfValues(double(uintValues(:)) + 1), size(channelImage));
    histAfter = intensity_histogram(equalizedChannel);
end

function counts = intensity_histogram(channelImage)
    channelImage = min(max(channelImage, 0), 1);
    uintValues = uint8(round(255 * channelImage));
    counts = histcounts(double(uintValues(:)), -0.5:1:255.5);
end

function stats = compute_channel_stats(channelImage)
    pixels = channelImage(:);
    stats.Mean = mean(pixels);
    stats.Variance = mean((pixels - stats.Mean) .^ 2);
    stats.Std = sqrt(stats.Variance);
    stats.Min = min(pixels);
    stats.Max = max(pixels);
    stats.DarkPercent = 100 * mean(pixels <= 0.25);
    stats.BrightPercent = 100 * mean(pixels >= 0.75);
    stats.SaturatedPercent = 100 * mean(pixels >= 0.98);
    stats.Michelson = (stats.Max - stats.Min) / (stats.Max + stats.Min + eps);
    stats.Entropy = compute_entropy(channelImage);
end

function entropyValue = compute_entropy(channelImage)
    counts = intensity_histogram(channelImage);
    probabilities = counts / sum(counts);
    probabilities = probabilities(probabilities > 0);
    entropyValue = -sum(probabilities .* log2(probabilities));
end

function row = make_result_row(imageName, imageNumber, highReferenceName, methodName, parameterText, tileSizeText, brightnessChannel, commentText)
    stats = compute_channel_stats(brightnessChannel);
    row = struct('Image', imageName, 'ImageNumber', imageNumber, 'HighReferenceImage', highReferenceName, 'Method', methodName, 'Parameter', parameterText, 'ApproxTileSize', tileSizeText, 'MeanBrightness', stats.Mean, 'IntensityVariance', stats.Variance, 'StdContrast', stats.Std, 'EntropyValue', stats.Entropy, 'DarkPixelPercent', stats.DarkPercent, 'BrightPixelPercent', stats.BrightPercent, 'SaturatedPixelPercent', stats.SaturatedPercent, 'MichelsonContrast', stats.Michelson, 'Comment', commentText);
end

function commentText = make_global_comment(inputBrightness, globalBrightness)
    inputStats = compute_channel_stats(inputBrightness);
    globalStats = compute_channel_stats(globalBrightness);

```

```

if globalStats.Std > inputStats.Std
    contrastPart = 'Η global ισοστάθμιση αυξάνει την αντίθεση επειδή απλώνει το ιστόγραμμα σε μεγαλύτερο εύρος.';
else
    contrastPart = 'Η global ισοστάθμιση δεν αυξάνει έντονα την αντίθεση σε αυτή την εικόνα.';
end
if globalStats.SaturatedPercent > inputStats.SaturatedPercent + 3
    riskPart = 'Παράλληλα εμφανίζεται αυξημένος κορεσμός, άρα υπάρχει κίνδυνος υπερενίσχυσης.';
elseif globalStats.Std > inputStats.Std * 1.8
    riskPart = 'Η ισχυρή αύξηση της αντίθεσης μπορεί να κάνει πιο ορατό τον θόρυβο στις σκοτεινές περιοχές.';
else
    riskPart = 'Η υπερενίσχυση παραμένει σχετικά περιορισμένη.';
end
commentText = [contrastPart, ' ', riskPart];
end

function commentText = make_clahe_comment(inputBrightness, globalBrightness, claheBrightness, numTiles, clipLimit)
    inputStats = compute_channel_stats(inputBrightness);
    globalStats = compute_channel_stats(globalBrightness);
    claheStats = compute_channel_stats(claheBrightness);
    if claheStats.Std > inputStats.Std
        detailPart = 'Η τοπική ισοστάθμιση αποκαλύπτει περισσότερες λεπτομέρειες στις σκοτεινές περιοχές.';
    else
        detailPart = 'Η τοπική ισοστάθμιση είναι ήπια και δεν αυξάνει πολύ την τοπική αντίθεση.';
    end
    if claheStats.SaturatedPercent < globalStats.SaturatedPercent || clipLimit <= 0.01
        clipPart = 'Το clip limit περιορίζει την υπερβολική ενίσχυση του θορύβου.';
    else
        clipPart = 'Το μεγαλύτερο clip limit δίνει πιο έντονη αντίθεση αλλά μπορεί να ενισχύσει θόρυβο.';
    end
    if max(numTiles) >= 16
        tilePart = 'Τα περισσότερα tiles αντιστοιχούν σε μικρότερα τοπικά παράθυρα και δίνουν πιο έντονη τοπική προσαρμογή.';
    elseif max(numTiles) <= 4
        tilePart = 'Τα λιγότερα tiles αντιστοιχούν σε μεγαλύτερα τοπικά παράθυρα και πιο ομαλή μεταβολή.';
    else
        tilePart = 'Η επιλογή 8x8 tiles δίνει ενδιάμεση ισορροπία ανάμεσα σε τοπική λεπτομέρεια και φυσικό αποτέλεσμα.';
    end
    commentText = [detailPart, ' ', clipPart, ' ', tilePart];
end

function tileSizeText = make_tile_size_text(imageSize, numTiles)
    tileRows = ceil(imageSize(1) / numTiles(1));
    tileCols = ceil(imageSize(2) / numTiles(2));
    tileSizeText = sprintf('about %dx%d pixels', tileRows, tileCols);
end

function safeName = make_safe_filename(fileName)
    safeName = regexprep(fileName, ['^A-Za-z0-9']+ , '_');
    safeName = regexprep(safeName, '^_+|_+$' , '');
    if isempty(safeName)
        safeName = 'image';
    end
end

function textValue = value_or_none(inputText)
    if isempty(inputText)
        textValue = 'δεν βρέθηκε';
    else
        textValue = inputText;
    end
end

function show_image_tile(inputImage, titleText)
    nexttile;
    imshow(inputImage);
    title(titleText, 'Interpreter', 'none', 'Color', 'k');
end

function show_histogram_tile(histCounts, titleText)
    nexttile;
    bar(0:255, histCounts, 1, 'FaceColor', [0.25 0.25 0.25], 'EdgeColor', 'none');
    xlim([0 255]);
    grid on;
    title(titleText, 'Color', 'k');
    xlabel('Intensity level');
    ylabel('Pixels');
end

function plot_summary_by_method(statsTable, metricName, titleText)
    hold on;
    rowsOriginal = strcmp(statsTable.Method, 'Original low-light');
    rowsGlobal = strcmp(statsTable.Method, 'Global histogram equalization');
    rowsClahe = strcmp(statsTable.Method, 'Adaptive histogram equalization') & strcmp(statsTable.Parameter, 'NumTiles=[8 8], ClipLimit=0.01');
    plot(statsTable.ImageNumber(rowsOriginal), statsTable.(metricName)(rowsOriginal), '-o', 'LineWidth', 1.2, 'DisplayName', 'Original');
    plot(statsTable.ImageNumber(rowsGlobal), statsTable.(metricName)(rowsGlobal), '-o', 'LineWidth', 1.2, 'DisplayName', 'Global histeq');
    plot(statsTable.ImageNumber(rowsClahe), statsTable.(metricName)(rowsClahe), '-o', 'LineWidth', 1.2, 'DisplayName', 'CLAHE 8x8 clip 0.01');
    hold off;
    grid on;
    xlabel('Image number');
    ylabel(metricName);
end

```

```

        title(titleText);
        legend('Location', 'best');
    end

function save_figure_png(figureHandle, outputPath)
    set(figureHandle, 'Color', 'w', 'InvertHardcopy', 'off');
    axesHandles = findall(figureHandle, 'Type', 'axes');
    for idx = 1:numel(axesHandles)
        set(axesHandles(idx), 'Color', 'w', 'XColor', 'k', 'YColor', 'k', 'ZColor', 'k');
        set(get(axesHandles(idx), 'Title'), 'Color', 'k');
        set(get(axesHandles(idx), 'XLabel'), 'Color', 'k');
        set(get(axesHandles(idx), 'YLabel'), 'Color', 'k');
        set(get(axesHandles(idx), 'ZLabel'), 'Color', 'k');
    end
    textHandles = findall(figureHandle, 'Type', 'text');
    if ~isempty(textHandles)
        set(textHandles, 'Color', 'k');
    end
    legendHandles = findall(figureHandle, 'Type', 'legend');
    if ~isempty(legendHandles)
        set(legendHandles, 'TextColor', 'k', 'Color', 'w', 'EdgeColor', 'k');
    end
    drawnow;
    print(figureHandle, outputPath, '-dpng', '-r200');
end

```

Μέρος Δ (Αρχείο: eikonaB3D.m)

```

clear;
clc;
close all;

% Για να επεξεργαστεί μια εικόνα, το όνομά της πρέπει να τελειώνει σε _low.png.

rng(1);

greekDesktop = char([933 960 959 955 959 947 953 963 964 942 962]);
% Replace with your path
projectDir = fullfile(getenv('USERPROFILE'), 'OneDrive', greekDesktop, 'project');
resultsDir = fullfile(projectDir, 'resultsB3D');
expectedFolderName = 'resultsB3D';

[~, actualFolderName] = fileparts(resultsDir);
if isempty(resultsDir) || strcmp(resultsDir, projectDir) || ~strcmp(actualFolderName, expectedFolderName)
    error('Unsafe results folder: %s', resultsDir);
end

if ~exist(resultsDir, 'dir')
    mkdir(resultsDir);
end

oldFiles = [dir(fullfile(resultsDir, '*.png')); dir(fullfile(resultsDir, '*.csv')); dir(fullfile(resultsDir, '*.txt'))];
dir(fullfile(resultsDir, '*.mat'));
oldWarningState = warning('off', 'all');
for oldIdx = 1:numel(oldFiles)
    oldPath = fullfile(resultsDir, oldFiles(oldIdx).name);
    if exist(oldPath, 'file') == 2
        try
            delete(oldPath);
        catch
        end
    end
end
warning(oldWarningState);

[imageFiles, imageIds] = find_low_images(projectDir);
saveComparisonFigures = should_save_comparison_figures();
figureVisibility = 'off';

gaussianVariance = 0.005;
saltPepperDensity = 0.05;
filterNames = {'Mean filter', 'Gaussian filter', 'Median filter', 'Wiener filter'};
pipelineFilters = {'Gaussian filter', 'Median filter'};

reportFile = fullfile(resultsDir, 'B3D_denoising_comments.txt');
reportFid = fopen(reportFile, 'w', 'n', 'UTF-8');
if reportFid == -1
    error('Δεν ήταν δυνατή η δημιουργία του αρχείου: %s', reportFile);
end
reportCleanup = onCleanup(@( ) fclose(reportFid));

fprintf(reportFid, 'Μέρος Δ: Αποθρομβοποίηση εικόνας\n');

```

```

fprintf(reportFid, 'Θάκελος εικόνων: %s\n', projectDir);
fprintf(reportFid, 'Θάκελος αποτελεσμάτων: %s\n\n', resultsDir);
fprintf(reportFid, 'Κανόνες ονομάτων: *_low.png είναι η σκοτεινή εικόνα και *_high.png η φωτεινή αναφορά.\n');
fprintf(reportFid, 'Προστίθεται τεχνητά Gaussian θόρυβος με variance %.4f και Salt & Pepper θόρυβος με density %.3f.\n', gaussianVariance, saltPepperDensity);
fprintf(reportFid, 'Η αρχική low-light εικόνα χρησιμοποιείται ως αναφορά για τη μέτρηση μείωσης θορύβου, επειδή ο θόρυβος είναι τεχνητός.\n');
fprintf(reportFid, 'Για τη σύγκριση των σειρών επεξεργασίας χρησιμοποιείται ως αναφορά η βελτιωμένη εικόνα που προκύπτει από την καθαρή low-light εικόνα.\n\n');

rowTemplate = struct('Image', '', 'ImageNumber', 0, 'HighReferenceImage', '', 'NoiseType', '', 'Stage', '', 'Method', '', 'Sequence', '', 'Parameters', '', 'MSE', 0, 'PSNR', 0, 'MAE', 0, 'StdContrast', 0, 'GradientMean', 0, 'GradientRatio', 0, 'EdgeCorrelation', 0, 'DarkPixelPercent', 0, 'SaturatedPixelPercent', 0, 'Comment', '');
resultRows = repmat(rowTemplate, numel(imageFiles) * 24, 1);
rowIdx = 0;

fprintf('Μέρος Δ: βρέθηκαν %d εικόνες χαμηλού φωτισμού.\n', numel(imageFiles));
fprintf('Τα αποτελέσματα θα αποθηκευτούν εδώ:\n%s\n\n', resultsDir);

for idx = 1:numel(imageFiles)
    imagePath = fullfile(projectDir, imageFiles(idx).name);
    [inputImage, isColorImage] = read_image_as_double(imagePath);

    [~, baseName] = fileparts(imageFiles(idx).name);
    safeBaseName = make_safe_filename(baseName);
    outputPrefix = sprintf('%02d_%s', idx, safeBaseName);

    highReferenceName = sprintf('%d_high.png', imageIds(idx));
    if exist(fullfile(projectDir, highReferenceName), 'file') ~= 2
        highReferenceName = '';
    end

    gaussianNoisy = imnoise(inputImage, 'gaussian', 0, gaussianVariance);
    saltPepperNoisy = imnoise(inputImage, 'salt & pepper', saltPepperDensity);
    idealEnhanced = enhance_image(inputImage);

    imwrite(inputImage, fullfile(resultsDir, [outputPrefix, '_input_low.png']));
    imwrite(gaussianNoisy, fullfile(resultsDir, [outputPrefix, '_gaussian_noisy.png']));
    imwrite(saltPepperNoisy, fullfile(resultsDir, [outputPrefix, '_saltpepper_noisy.png']));
    imwrite(idealEnhanced, fullfile(resultsDir, [outputPrefix, '_ideal_enhanced_clean.png']));

    noiseImages = {gaussianNoisy, saltPepperNoisy};
    noiseTypes = {'Gaussian noise', 'Salt & Pepper noise'};

    fprintf(reportFid, 'Εικόνα %02d: %s\n', idx, imageFiles(idx).name);
    fprintf(reportFid, 'Αντίστοιχη φωτεινή εικόνα: %s\n', value_or_none(highReferenceName));

    for noiseIdx = 1:numel(noiseImages)
        noisyImage = noiseImages{noiseIdx};
        noiseType = noiseTypes{noiseIdx};
        noisyMetrics = compute_metrics(inputImage, noisyImage, isColorImage);

        rowIdx = rowIdx + 1;
        resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, noiseType, 'Noisy image', 'No filtering', '', '', inputImage, noisyImage, isColorImage, 'Εικόνα μετά την τεχνητή προσθήκη θορύβου.');
```

```

        filteredImages = cell(numel(filterNames), 1);
        filteredMetrics = cell(numel(filterNames), 1);

        for filterIdx = 1:numel(filterNames)
            filterName = filterNames{filterIdx};
            filteredImages{filterIdx} = apply_filter(noisyImage, filterName);
            filteredMetrics{filterIdx} = compute_metrics(inputImage, filteredImages{filterIdx}, isColorImage);
            filterSafeName = make_safe_filename([lower(strrep(filterName, ' ', '_')), '_', lower(strrep(noiseType, ' ', '_'))]);
            imwrite(filteredImages{filterIdx}, fullfile(resultsDir, [outputPrefix, '_', filterSafeName, '.png']));

            rowIdx = rowIdx + 1;
            filterComment = make_filter_comment(noiseType, filterName, noisyMetrics, filteredMetrics{filterIdx});
            resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, noiseType, 'Denoising', filterName, 'filter only', filter_parameters(filterName), inputImage, filteredImages{filterIdx}, isColorImage, filterComment);
            fprintf(reportFid, '%s - %s: %s\n', noiseType, filterName, filterComment);
        end

        for pipelineIdx = 1:numel(pipelineFilters)
            selectedFilter = pipelineFilters{pipelineIdx};
            denoisedFirst = apply_filter(noisyImage, selectedFilter);
            pipelineDenoiseThenEnhance = enhance_image(denoisedFirst);
            enhancedFirst = enhance_image(noisyImage);
            pipelineEnhanceThenDenoise = apply_filter(enhancedFirst, selectedFilter);

            pipelineSafeName = make_safe_filename([lower(strrep(selectedFilter, ' ', '_')), '_', lower(strrep(noiseType, ' ', '_'))]);
            imwrite(pipelineDenoiseThenEnhance, fullfile(resultsDir, [outputPrefix, '_pipeline_denoise_then_enhance_', pipelineSafeName, '.png']));
            imwrite(pipelineEnhanceThenDenoise, fullfile(resultsDir, [outputPrefix, '_pipeline_enhance_then_denoise_', pipelineSafeName, '.png']));

            metricsDenoiseThenEnhance = compute_metrics(idealEnhanced, pipelineDenoiseThenEnhance, isColorImage);
            metricsEnhanceThenDenoise = compute_metrics(idealEnhanced, pipelineEnhanceThenDenoise, isColorImage);
            [commentDenoiseThenEnhance, commentEnhanceThenDenoise] = make_pipeline_comments(noiseType, selectedFilter, metricsDenoiseThenEnhance, metricsEnhanceThenDenoise);

```

```

        rowIdx = rowIdx + 1;
        resultRows(rowIdx) = make_result_row_from_metrics(imageFiles(idx).name, imageIds(idx), highReferenceName, noiseType, 'Final
enhanced image', selectedFilter, 'denoise -> enhance', 'gamma 0.6 + contrast stretch', metricsDenoiseThenEnhance, commentDenoiseThenEnhance);

        rowIdx = rowIdx + 1;
        resultRows(rowIdx) = make_result_row_from_metrics(imageFiles(idx).name, imageIds(idx), highReferenceName, noiseType, 'Final
enhanced image', selectedFilter, 'enhance -> denoise', 'gamma 0.6 + contrast stretch', metricsEnhanceThenDenoise, commentEnhanceThenDenoise);

        fprintf(reportFid, '%s - %s - denoise -> enhance: %s\n', noiseType, selectedFilter, commentDenoiseThenEnhance);
        fprintf(reportFid, '%s - %s - enhance -> denoise: %s\n', noiseType, selectedFilter, commentEnhanceThenDenoise);
    end
end

fprintf(reportFid, '\n');

if saveComparisonFigures
    save_image_grid({inputImage, gaussianNoisy, saltPepperNoisy}, {'Original low-light', 'Gaussian noise', 'Salt & Pepper noise'},
sprintf('Part D - Noise: %s', imageFiles(idx).name), fullfile(resultsDir, [outputPrefix, '_noise_comparison.png']));

    make_filter_figure(imageFiles(idx).name, outputPrefix, resultsDir, gaussianNoisy, inputImage, apply_filter(gaussianNoisy, 'Mean
filter'), apply_filter(gaussianNoisy, 'Gaussian filter'), apply_filter(gaussianNoisy, 'Median filter'), apply_filter(gaussianNoisy, 'Wiener
filter'), 'Gaussian noise', figureVisibility);
    make_filter_figure(imageFiles(idx).name, outputPrefix, resultsDir, saltPepperNoisy, inputImage, apply_filter(saltPepperNoisy, 'Mean
filter'), apply_filter(saltPepperNoisy, 'Gaussian filter'), apply_filter(saltPepperNoisy, 'Median filter'), apply_filter(saltPepperNoisy,
'Wiener filter'), 'Salt & Pepper noise', figureVisibility);
    make_pipeline_figure(imageFiles(idx).name, outputPrefix, resultsDir, idealEnhanced, gaussianNoisy, saltPepperNoisy,
figureVisibility);
end

metricsGaussianNoisy = compute_metrics(inputImage, gaussianNoisy, isColorImage);
metricsSaltPepperNoisy = compute_metrics(inputImage, saltPepperNoisy, isColorImage);
metricsGaussianFiltered = compute_metrics(inputImage, apply_filter(gaussianNoisy, 'Gaussian filter'), isColorImage);
metricsSaltPepperFiltered = compute_metrics(inputImage, apply_filter(saltPepperNoisy, 'Median filter'), isColorImage);

fprintf('%02d/%02d %s: Gaussian PSNR %.2f -> %.2f, Salt & Pepper PSNR %.2f -> %.2f\n', idx, numel(imageFiles), imageFiles(idx).name,
metricsGaussianNoisy.PSNR, metricsGaussianFiltered.PSNR, metricsSaltPepperNoisy.PSNR, metricsSaltPepperFiltered.PSNR);
drawnow;
end

resultRows = resultRows(1:rowIdx);
statsTable = struct2table(resultRows);
writetable(statsTable, fullfile(resultsDir, 'B3D_statistics.csv'));

if saveComparisonFigures
    save_d_summary_image(statsTable, fullfile(resultsDir, 'B3D_summary_statistics.png'));
end

fprintf('\nΤο Μέρος Δ ολοκληρώθηκε.\n');
if saveComparisonFigures
    fprintf('Αποθηκεύτηκαν εικόνες, figures, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);
else
    fprintf('Αποθηκεύτηκαν εικόνες, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);
end

function [imageFiles, imageIds] = find_low_images(projectDir)
    lowFiles = dir(fullfile(projectDir, '*_low.png'));
    keepFile = false(numel(lowFiles), 1);
    imageIds = nan(numel(lowFiles), 1);
    for fileIdx = 1:numel(lowFiles)
        tokens = regexp(lowFiles(fileIdx).name, '^(d+)_low.png$', 'tokens', 'once');
        if ~isempty(tokens)
            keepFile(fileIdx) = true;
            imageIds(fileIdx) = str2double(tokens{1});
        end
    end
    imageFiles = lowFiles(keepFile);
    imageIds = imageIds(keepFile);
    if isempty(imageFiles)
        error('Δεν βρέθηκαν εικόνες με μορφή ονόματος number_low.png στον φάκελο: %s', projectDir);
    end
    [imageIds, sortOrder] = sort(imageIds);
    imageFiles = imageFiles(sortOrder);
end

function saveComparisonFigures = should_save_comparison_figures()
    saveComparisonFigures = true;
end

function [imageDouble, isColorImage] = read_image_as_double(imagePath)
    [rawImage, colorMap] = imread(imagePath);
    if ~isempty(colorMap)
        imageDouble = ind2rgb(rawImage, colorMap);
    else
        imageDouble = im2double(rawImage);
    end
    if ndims(imageDouble) == 3 && size(imageDouble, 3) > 3
        imageDouble = imageDouble(:, :, 1:3);
    end
    isColorImage = ndims(imageDouble) == 3 && size(imageDouble, 3) == 3;
end

```

```

end

function filteredImage = apply_filter(inputImage, filterName)
    switch filterName
        case 'Mean filter'
            kernel = ones(3, 3) / 9;
            filteredImage = imfilter(inputImage, kernel, 'replicate');
        case 'Gaussian filter'
            filteredImage = imgaussfilt(inputImage, 1.0, 'FilterSize', 5);
        case 'Median filter'
            filteredImage = apply_channel_filter(inputImage, @(channel) medfilt2(channel, [3 3], 'symmetric'));
        case 'Wiener filter'
            filteredImage = apply_channel_filter(inputImage, @(channel) wiener2(channel, [5 5]));
        otherwise
            error('Άγνωστο φίλτρο: %s', filterName);
    end
    filteredImage = min(max(filteredImage, 0), 1);
end

function outputImage = apply_channel_filter(inputImage, filterFunction)
    if ndims(inputImage) == 3
        outputImage = zeros(size(inputImage));
        for channelId = 1:size(inputImage, 3)
            outputImage(:, :, channelId) = filterFunction(inputImage(:, :, channelId));
        end
    else
        outputImage = filterFunction(inputImage);
    end
end

function outputImage = enhance_image(inputImage)
    gammaImage = min(max(inputImage, 0), 1) .^ 0.6;
    outputImage = linear_contrast_stretch(gammaImage);
end

function outputImage = linear_contrast_stretch(inputImage)
    inputMin = min(inputImage(:));
    inputMax = max(inputImage(:));
    if inputMax <= inputMin
        outputImage = zeros(size(inputImage));
    else
        outputImage = (inputImage - inputMin) / (inputMax - inputMin);
    end
end

function metrics = compute_metrics(referenceImage, testImage, isColorImage)
    referenceImage = min(max(referenceImage, 0), 1);
    testImage = min(max(testImage, 0), 1);
    diffImage = testImage - referenceImage;
    metrics.MSE = mean(diffImage(:) .^ 2);
    if metrics.MSE == 0
        metrics.PSNR = Inf;
    else
        metrics.PSNR = 10 * log10(1 / metrics.MSE);
    end
    metrics.MAE = mean(abs(diffImage(:)));
    testGray = to_gray(testImage, isColorImage);
    referenceGray = to_gray(referenceImage, isColorImage);
    pixels = testGray(:);
    metrics.StdContrast = std(pixels);
    metrics.DarkPixelPercent = 100 * mean(pixels <= 0.25);
    metrics.SaturatedPixelPercent = 100 * mean(pixels >= 0.98);
    referenceGradient = gradient_magnitude(referenceGray);
    testGradient = gradient_magnitude(testGray);
    metrics.GradientMean = mean(testGradient(:));
    referenceGradientMean = mean(referenceGradient(:));
    metrics.GradientRatio = metrics.GradientMean / (referenceGradientMean + eps);
    metrics.EdgeCorrelation = safe_correlation(referenceGradient(:), testGradient(:));
end

function grayImage = to_gray(inputImage, isColorImage)
    if isColorImage
        grayImage = rgb2gray(inputImage);
    else
        grayImage = inputImage;
    end
end

function magnitude = gradient_magnitude(grayImage)
    sobelX = [-1 0 1; -2 0 2; -1 0 1] / 8;
    sobelY = sobelX';
    gradientX = imfilter(grayImage, sobelX, 'replicate');
    gradientY = imfilter(grayImage, sobelY, 'replicate');
    magnitude = sqrt(gradientX .^ 2 + gradientY .^ 2);
end

function value = safe_correlation(valuesA, valuesB)
    if std(valuesA) == 0 || std(valuesB) == 0
        value = 0;
    else

```

```

        correlationMatrix = corrcoef(valuesA, valuesB);
        value = correlationMatrix(1, 2);
    end
end

function row = make_result_row(imageName, imageNumber, highReferenceName, noiseType, stageName, methodName, sequenceName, parameterText,
referenceImage, resultImage, isColorImage, commentText)
    metrics = compute_metrics(referenceImage, resultImage, isColorImage);
    row = make_result_row_from_metrics(imageName, imageNumber, highReferenceName, noiseType, stageName, methodName, sequenceName,
parameterText, metrics, commentText);
end

function row = make_result_row_from_metrics(imageName, imageNumber, highReferenceName, noiseType, stageName, methodName, sequenceName,
parameterText, metrics, commentText)
    row = struct('Image', imageName, 'ImageNumber', imageNumber, 'HighReferenceImage', highReferenceName, 'NoiseType', noiseType, 'Stage',
stageName, 'Method', methodName, 'Sequence', sequenceName, 'Parameters', parameterText, 'MSE', metrics.MSE, 'PSNR', metrics.PSNR, 'MAE',
metrics.MAE, 'StdContrast', metrics.StdContrast, 'GradientMean', metrics.GradientMean, 'GradientRatio', metrics.GradientRatio,
'EdgeCorrelation', metrics.EdgeCorrelation, 'DarkPixelPercent', metrics.DarkPixelPercent, 'SaturatedPixelPercent',
metrics.SaturatedPixelPercent, 'Comment', commentText);
end

function textValue = filter_parameters(filterName)
    switch filterName
        case 'Mean filter'
            textValue = '3x3 average kernel';
        case 'Gaussian filter'
            textValue = 'sigma=1.0, filter size=5';
        case 'Median filter'
            textValue = '3x3 median';
        case 'Wiener filter'
            textValue = '5x5 adaptive Wiener';
        otherwise
            textValue = '';
    end
end

function commentText = make_filter_comment(noiseType, filterName, noisyMetrics, filteredMetrics)
    psnrGain = filteredMetrics.PSNR - noisyMetrics.PSNR;
    if psnrGain > 2
        noisePart = sprintf('Το φίλτρο μειώνει αισθητά τον θόρυβο, με κέρδος PSNR %.2f dB.', psnrGain);
    elseif psnrGain > 0
        noisePart = sprintf('Το φίλτρο μειώνει μέτρια τον θόρυβο, με κέρδος PSNR %.2f dB.', psnrGain);
    else
        noisePart = sprintf('Το φίλτρο δεν βελτιώνει τη μετρική PSNR σε αυτή την περίπτωση, με μεταβολή %.2f dB.', psnrGain);
    end
    if filteredMetrics.GradientRatio < 0.75
        detailPart = 'Παρατηρείται πιθανή θόλωση λεπτομερειών και ακμών.';
    elseif filteredMetrics.GradientRatio > 1.25
        detailPart = 'Οι ακμές παραμένουν έντονες, αλλά μπορεί να παραμένει και θόρυβος.';
    else
        detailPart = 'Η ένταση των ακμών διατηρείται σε λογικά επίπεδα.';
    end
    if strcmp(noiseType, 'Salt & Pepper noise') && strcmp(filterName, 'Median filter')
        typePart = 'Το median filter είναι ιδιαίτερα κατάλληλο για Salt & Pepper θόρυβο.';
    elseif strcmp(noiseType, 'Gaussian noise') && strcmp(filterName, 'Gaussian filter')
        typePart = 'Το Gaussian filter είναι φυσική επιλογή για Gaussian θόρυβο.';
    elseif strcmp(filterName, 'Mean filter')
        typePart = 'Το mean filter εξομαλύνει τον θόρυβο αλλά συχνά θολώνει περισσότερο τις λεπτομέρειες.';
    elseif strcmp(filterName, 'Wiener filter')
        typePart = 'Το Wiener filter προσαρμόζεται στην τοπική διακύμανση της εικόνας.';
    else
        typePart = 'Η καταλληλότητα εξαρτάται από το είδος του θορύβου και τη λεπτομέρεια της εικόνας.';
    end
    commentText = [noisePart, ' ', detailPart, ' ', typePart];
end

function [commentDenoiseThenEnhance, commentEnhanceThenDenoise] = make_pipeline_comments(noiseType, filterName, metricsDenoiseThenEnhance,
metricsEnhanceThenDenoise)
    if metricsDenoiseThenEnhance.PSNR >= metricsEnhanceThenDenoise.PSNR
        betterText = 'Η σειρά πρώτα αποθρομβοποίηση και μετά ενίσχυση δίνει καλύτερη ποσοτική προσέγγιση στην καθαρή βελτιωμένη εικόνα.';
    else
        betterText = 'Η σειρά πρώτα ενίσχυση και μετά αποθρομβοποίηση δίνει καλύτερη ποσοτική προσέγγιση στην καθαρή βελτιωμένη εικόνα.';
    end
    commentDenoiseThenEnhance = sprintf('%s Για %s με %s, το PSNR είναι %.2f dB.', betterText, noiseType, filterName,
metricsDenoiseThenEnhance.PSNR);
    commentEnhanceThenDenoise = sprintf('%s Για %s με %s, το PSNR είναι %.2f dB.', betterText, noiseType, filterName,
metricsEnhanceThenDenoise.PSNR);
end

function safeName = make_safe_filename(fileName)
    safeName = regexprep(fileName, ['^A-Za-z0-9]+' , '_');
    safeName = regexprep(safeName, '^+|_+$' , '');
    if isempty(safeName)
        safeName = 'image';
    end
end

function textValue = value_or_none(inputText)
    if isempty(inputText)
        textValue = 'δεν βρέθηκε';
    end
end

```

```

else
    textValue = inputText;
end
end

function make_filter_figure(imageName, outputPrefix, resultsDir, noisyImage, originalImage, meanImage, gaussianImage, medianImage,
wienerImage, noiseTitle, ~)
    imageList = {originalImage, noisyImage, meanImage, gaussianImage, medianImage, wienerImage};
    titleList = {'Original low-light', noiseTitle, 'Mean filter', 'Gaussian filter', 'Median filter', 'Wiener filter'};
    save_image_grid(imageList, titleList, sprintf('Part D - Filters: %s - %s', imageName, noiseTitle), fullfile(resultsDir, [outputPrefix,
'_filters_', make_safe_filename(noiseTitle), '.png']));
end

function make_pipeline_figure(imageName, outputPrefix, resultsDir, idealEnhanced, gaussianNoisy, saltPepperNoisy, ~)
    gaussianFilter = 'Gaussian filter';
    medianFilter = 'Median filter';
    gaussianDenoiseThenEnhance = enhance_image(apply_filter(gaussianNoisy, gaussianFilter));
    gaussianEnhanceThenDenoise = apply_filter(enhance_image(gaussianNoisy), gaussianFilter);
    saltDenoiseThenEnhance = enhance_image(apply_filter(saltPepperNoisy, medianFilter));
    saltEnhanceThenDenoise = apply_filter(enhance_image(saltPepperNoisy), medianFilter);
    imageList = {idealEnhanced, gaussianDenoiseThenEnhance, gaussianEnhanceThenDenoise, idealEnhanced, saltDenoiseThenEnhance,
saltEnhanceThenDenoise};
    titleList = {'Clean enhanced', 'Gaussian denoise -> enhance', 'Gaussian enhance -> denoise', 'Clean enhanced', 'S&P denoise -> enhance',
'S&P enhance -> denoise'};
    save_image_grid(imageList, titleList, sprintf('Part D - Processing order: %s', imageName), fullfile(resultsDir, [outputPrefix,
'_pipeline_comparison.png']));
end

function save_d_summary_image(statsTable, outputPath)
    methods = {'Mean filter', 'Gaussian filter', 'Median filter', 'Wiener filter'};
    noiseTypes = {'Gaussian noise', 'Salt & Pepper noise'};

    hFig = figure('Visible', 'off', 'Color', 'w', 'Position', [100, 100, 1200, 760]);
    cleanupObj = onCleanup(@( ) close(hFig));

    for noiseIdx = 1:numel(noiseTypes)
        subplot(2, 1, noiseIdx);
        hold on;
        for methodIdx = 1:numel(methods)
            rows = strcmp(statsTable.NoiseType, noiseTypes{noiseIdx}) & ...
                strcmp(statsTable.Stage, 'Denoising') & ...
                strcmp(statsTable.Method, methods{methodIdx});
            if ~any(rows)
                continue;
            end
            [xValues, sortIdx] = sort(statsTable.ImageNumber(rows));
            yValues = statsTable.PSNR(rows);
            yValues = yValues(sortIdx);
            plot(xValues, yValues, '-o', 'LineWidth', 1.4, 'MarkerSize', 4, 'DisplayName', methods{methodIdx});
        end
        hold off;
        grid on;
        title(noiseTypes{noiseIdx});
        xlabel('Image number');
        ylabel('PSNR (dB)');
        legend('Location', 'bestoutside');
    end

    try
        sgtitle('Part D - Denoising filters summary');
    catch
    end

    try
        exportgraphics(hFig, outputPath, 'Resolution', 150);
    catch
        saveas(hFig, outputPath);
    end

    clear cleanupObj;
end

function canvas = draw_filter_chart(canvas, statsTable, noiseType, methods, rect)
    xSeries = cell(numel(methods), 1);
    ySeries = cell(numel(methods), 1);
    for methodIdx = 1:numel(methods)
        rows = strcmp(statsTable.NoiseType, noiseType) & strcmp(statsTable.Stage, 'Denoising') & strcmp(statsTable.Method,
methods{methodIdx});
        [xSeries{methodIdx}, sortIdx] = sort(statsTable.ImageNumber(rows));
        values = statsTable.PSNR(rows);
        ySeries{methodIdx} = values(sortIdx);
    end
    canvas = draw_line_chart(canvas, xSeries, ySeries, methods, noiseType, 'PSNR (dB)', rect);
end

function canvas = draw_line_chart(canvas, xSeries, ySeries, labels, titleText, ylabel, rect)
    colors = [0.12 0.35 0.75; 0.85 0.33 0.10; 0.15 0.55 0.25; 0.55 0.25 0.70; 0.20 0.20 0.20];
    panelLeft = rect(1);
    panelTop = rect(2);
    panelWidth = rect(3);

```

```

panelHeight = rect(4);
plotLeft = panelLeft + 75;
plotTop = panelTop + 55;
plotWidth = panelWidth - 150;
plotHeight = panelHeight - 115;

allX = [];
allY = [];
for idx = 1:numel(ySeries)
    allX = [allX; xSeries{idx}(:)]; %#ok<AGROW>
    allY = [allY; ySeries{idx}(:)]; %#ok<AGROW>
end
allY = allY(isfinite(allY));
if isempty(allX) || isempty(allY)
    return;
end
xMin = min(allX);
xMax = max(allX);
yMin = min(allY);
yMax = max(allY);
if xMax <= xMin
    xMax = xMin + 1;
end
if yMax <= yMin
    yMax = yMin + 1;
end
yPad = 0.08 * (yMax - yMin);
yMin = yMin - yPad;
yMax = yMax + yPad;

canvas = draw_line(canvas, plotLeft, plotTop + plotHeight, plotLeft + plotWidth, plotTop + plotHeight, [0 0 0], 2);
canvas = draw_line(canvas, plotLeft, plotTop, plotLeft, plotTop + plotHeight, [0 0 0], 2);
for gridIdx = 1:4
    yGrid = round(plotTop + plotHeight * gridIdx / 5);
    canvas = draw_line(canvas, plotLeft, yGrid, plotLeft + plotWidth, yGrid, [0.85 0.85 0.85], 1);
end

for idx = 1:numel(ySeries)
    xValues = xSeries{idx};
    yValues = ySeries{idx};
    valid = isfinite(xValues) & isfinite(yValues);
    xValues = xValues(valid);
    yValues = yValues(valid);
    if isempty(xValues)
        continue;
    end
    xPix = round(plotLeft + (xValues - xMin) / (xMax - xMin) * plotWidth);
    yPix = round(plotTop + plotHeight - (yValues - yMin) / (yMax - yMin) * plotHeight);
    colorValue = colors(1 + mod(idx - 1, size(colors, 1)), :);
    for pointIdx = 1:numel(xPix) - 1
        canvas = draw_line(canvas, xPix(pointIdx), yPix(pointIdx), xPix(pointIdx + 1), yPix(pointIdx + 1), colorValue, 3);
    end
    for pointIdx = 1:numel(xPix)
        canvas = draw_square(canvas, xPix(pointIdx), yPix(pointIdx), colorValue, 4);
    end
end

if exist('insertText', 'file') == 2
    canvas = insertText(canvas, [panelLeft + 10, panelTop + 8], titleText, 'FontSize', 22, 'BoxOpacity', 0, 'TextColor', 'black');
    canvas = insertText(canvas, [plotLeft, plotTop + plotHeight + 16], 'Image number', 'FontSize', 16, 'BoxOpacity', 0, 'TextColor',
'black');
    canvas = insertText(canvas, [panelLeft + 10, plotTop + 4], yLabel, 'FontSize', 16, 'BoxOpacity', 0, 'TextColor', 'black');
    for idx = 1:numel(labels)
        legendX = panelLeft + panelWidth - 250;
        legendY = panelTop + 42 + 26 * idx;
        canvas = draw_line(canvas, legendX, legendY + 9, legendX + 28, legendY + 9, colors(1 + mod(idx - 1, size(colors, 1)), :), 4);
        canvas = insertText(canvas, [legendX + 35, legendY], labels{idx}, 'FontSize', 15, 'BoxOpacity', 0, 'TextColor', 'black');
    end
end
end

function save_image_grid(imagelist, titleList, mainTitle, outputPath)
    validIdx = find(~cellfun(@isempty, imagelist));
    if isempty(validIdx)
        return;
    end
    tileHeight = 260;
    tileWidth = 390;
    topMargin = 70;
    labelHeight = 34;
    gap = 18;
    cols = min(3, numel(validIdx));
    rows = ceil(numel(validIdx) / cols);
    canvasHeight = topMargin + rows * (labelHeight + tileHeight) + (rows + 1) * gap;
    canvasWidth = cols * tileWidth + (cols + 1) * gap;
    canvas = ones(canvasHeight, canvasWidth, 3);
    if exist('insertText', 'file') == 2
        canvas = insertText(canvas, [round(canvasWidth / 2) - 260, 18], mainTitle, 'FontSize', 24, 'BoxOpacity', 0, 'TextColor', 'black');
    end
    for tileIdx = 1:numel(validIdx)

```

```

        imageIdx = validIdx(tileIdx);
        rowIdx = floor((tileIdx - 1) / cols);
        colIdx = mod(tileIdx - 1, cols);
        xStart = gap + colIdx * (tileWidth + gap) + 1;
        yLabel = topMargin + gap + rowIdx * (labelHeight + tileHeight + gap) + 1;
        yStart = yLabel + labelHeight;
        tileImage = prepare_grid_image(imageList{imageIdx}, tileHeight, tileWidth);
        if exist('insertText', 'file') == 2
            canvas = insertText(canvas, [xStart + 8, yLabel + 4], titleList{imageIdx}, 'FontSize', 18, 'BoxOpacity', 0, 'TextColor',
'black');
        end
        canvas(yStart:yStart + tileHeight - 1, xStart:xStart + tileWidth - 1, :) = tileImage;
    end
    imwrite(canvas, outputPath);
end

function outputImage = prepare_grid_image(inputImage, targetHeight, targetWidth)
    inputImage = min(max(inputImage, 0), 1);
    if islogical(inputImage)
        inputImage = double(inputImage);
    end
    if ismatrix(inputImage)
        inputImage = repmat(inputImage, [1 1 3]);
    else
        inputImage = inputImage(:, :, 1:3);
    end
    outputImage = imresize(inputImage, [targetHeight targetWidth]);
end

function canvas = draw_line(canvas, x1, y1, x2, y2, colorValue, thickness)
    steps = max(abs(x2 - x1), abs(y2 - y1)) + 1;
    xValues = round(linspace(x1, x2, steps));
    yValues = round(linspace(y1, y2, steps));
    for idx = 1:numel(xValues)
        canvas = draw_square(canvas, xValues(idx), yValues(idx), colorValue, thickness);
    end
end

function canvas = draw_square(canvas, xCenter, yCenter, colorValue, radius)
    [height, width, ~] = size(canvas);
    xRange = max(1, xCenter - radius):min(width, xCenter + radius);
    yRange = max(1, yCenter - radius):min(height, yCenter + radius);
    for channelId = 1:3
        channel = canvas(:, :, channelId);
        channel(yRange, xRange) = colorValue(channelId);
        canvas(:, :, channelId) = channel;
    end
end

```

Μέρος Ε (Αρχείο: eikonaB3E.m)

```

clear;
clc;
close all;

% Για να επεξεργαστεί μια εικόνα, το όνομά της πρέπει να τελειώνει σε _low.png.

greekDesktop = char([933 960 959 955 959 947 953 963 964 942 962]);
% Replace with your path
projectDir = fullfile(getenv('USERPROFILE'), 'OneDrive', greekDesktop, 'project');
resultsDir = fullfile(projectDir, 'resultsB3E');
expectedFolderName = 'resultsB3E';

[~, actualFolderName] = fileparts(resultsDir);
if isempty(resultsDir) || strcmp(resultsDir, projectDir) || ~strcmp(actualFolderName, expectedFolderName)
    error('Unsafe results folder: %s', resultsDir);
end

if ~exist(resultsDir, 'dir')
    mkdir(resultsDir);
end

oldFiles = [dir(fullfile(resultsDir, '*.png')); dir(fullfile(resultsDir, '*.csv')); dir(fullfile(resultsDir, '*.txt'))];
dir(fullfile(resultsDir, '*.mat'));
oldWarningState = warning('off', 'all');
for oldIdx = 1:numel(oldFiles)
    oldPath = fullfile(resultsDir, oldFiles(oldIdx).name);
    if exist(oldPath, 'file') == 2
        try
            delete(oldPath);
        catch
        end
    end
end

```

```

end
warning(oldWarningState);

[imageFiles, imageIds] = find_low_images(projectDir);
saveComparisonFigures = should_save_comparison_figures();
figureVisibility = 'off';
unsharpKValues = [0.5 1.0 1.5];

reportFile = fullfile(resultsDir, 'B3E_sharpening_comments.txt');
reportFid = fopen(reportFile, 'w', 'n', 'UTF-8');
if reportFid == -1
    error('Δεν ήταν δυνατή η δημιουργία του αρχείου: %s', reportFile);
end
reportCleanup = onCleanup(@() fclose(reportFid));

fprintf(reportFid, 'Μέρος E: Όξυνση και ανάδειξη λεπτομερειών\n');
fprintf(reportFid, 'Θάκελος εικόνων: %s\n', projectDir);
fprintf(reportFid, 'Θάκελος αποτελεσμάτων: %s\n', resultsDir);
fprintf(reportFid, 'Κανόνας ονομάτων: *_low.png είναι η σκοτεινή εικόνα και *_high.png η φωτεινή αναφορά.\n');
fprintf(reportFid, 'Πριν την όξυνση εφαρμόζεται βασική βελτίωση φωτεινότητας και ήπια αποθρομβοποίηση, ώστε να μελετηθεί η ανάδειξη  
λεπτομερειών σε πιθανώς θολή εικόνα.\n');
fprintf(reportFid, 'Εξετάζονται Laplacian sharpening και unsharp masking με k = 0.5, 1.0, 1.5.\n\n');

rowTemplate = struct('Image', '', 'ImageNumber', 0, 'HighReferenceImage', '', 'Method', '', 'Parameter', '', 'MeanBrightness', 0,
'StdContrast', 0, 'GradientMean', 0, 'GradientRatioToBase', 0, 'LaplacianEnergy', 0, 'DarkPixelPercent', 0, 'SaturatedPixelPercent', 0,
'MSEToHigh', NaN, 'PSNRToHigh', NaN, 'Comment', '');
resultRows = repmat(rowTemplate, numel(imageFiles) * 6, 1);
rowIdx = 0;

fprintf('Μέρος E: βρέθηκαν %d εικόνες χαμηλού φωτισμού.\n', numel(imageFiles));
fprintf('Τα αποτελέσματα θα αποθηκευτούν εδώ:\n%s\n', resultsDir);

for idx = 1:numel(imageFiles)
    imagePath = fullfile(projectDir, imageFiles(idx).name);
    [inputImage, isColorImage] = read_image_as_double(imagePath);

    highReferenceName = sprintf('%d_high.png', imageIds(idx));
    highReferencePath = fullfile(projectDir, highReferenceName);
    if exist(highReferencePath, 'file') == 2
        [highReferenceImage, ~] = read_image_as_double(highReferencePath);
    else
        highReferenceName = '';
        highReferenceImage = [];
    end

    [~, baseName] = fileparts(imageFiles(idx).name);
    safeBaseName = make_safe_filename(baseName);
    outputPrefix = sprintf('%02d_%s', idx, safeBaseName);

    baseImage = create_base_enhanced_denoised_image(inputImage);
    laplacianImage = laplacian_sharpen(baseImage);
    unsharpImages = cell(numel(unsharpKValues), 1);

    for kIdx = 1:numel(unsharpKValues)
        unsharpImages{kIdx} = unsharp_mask(baseImage, unsharpKValues(kIdx));
    end

    imwrite(inputImage, fullfile(resultsDir, [outputPrefix, '_input_low.png']));
    imwrite(baseImage, fullfile(resultsDir, [outputPrefix, '_base_enhanced_denoised.png']));
    imwrite(laplacianImage, fullfile(resultsDir, [outputPrefix, '_laplacian_sharpening.png']));

    for kIdx = 1:numel(unsharpKValues)
        kLabel = strrep(sprintf('%01f', unsharpKValues(kIdx)), '.', '_');
        imwrite(unsharpImages{kIdx}, fullfile(resultsDir, [outputPrefix, '_unsharp_k_', kLabel, '.png']));
    end

    rowIdx = rowIdx + 1;
    resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Original low-light', '', inputImage,
baseImage, highReferenceImage, isColorImage, 'Αρχική εικόνα χαμηλού φωτισμού πριν από βελτίωση, αποθρομβοποίηση και όξυνση.');
```

```

    rowIdx = rowIdx + 1;
    resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Base enhanced denoised', 'gamma 0.6 +
contrast stretch + Gaussian denoise', baseImage, baseImage, highReferenceImage, isColorImage, 'Βασική βελτιωμένη και αποθρομβοποιημένη εικόνα
που χρησιμοποιείται ως είσοδος για την όξυνση.');
```

```

    rowIdx = rowIdx + 1;
    laplacianComment = make_sharpening_comment('Laplacian sharpening', baseImage, laplacianImage, isColorImage);
    resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Laplacian sharpening', '3x3 Laplacian
mask', laplacianImage, baseImage, highReferenceImage, isColorImage, laplacianComment);

    for kIdx = 1:numel(unsharpKValues)
        kValue = unsharpKValues(kIdx);
        unsharpComment = make_unsharp_comment(kValue, baseImage, unsharpImages{kIdx}, isColorImage);
        rowIdx = rowIdx + 1;
        resultRows(rowIdx) = make_result_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Unsharp masking', sprintf('k = %01f',
kValue), unsharpImages{kIdx}, baseImage, highReferenceImage, isColorImage, unsharpComment);
    end

    fprintf(reportFid, 'Εικόνα %02d: %s\n', idx, imageFiles(idx).name);

```

```

fprintf(reportFid, 'Αντιστοιχη φωτεινή εικόνα: %s\n', value_or_none(highReferenceName));
fprintf(reportFid, 'Laplacian sharpening: %s\n', laplacianComment);
for kIdx = 1:numel(unsharpKValues)
    fprintf(reportFid, 'Unsharp masking k=%.1f: %s\n', unsharpKValues(kIdx), make_unsharp_comment(unsharpKValues(kIdx), baseImage,
unsharpImages{kIdx}, isColorImage));
end
fprintf(reportFid, '\n');

if saveComparisonFigures
    imageList = {inputImage, baseImage, laplacianImage, unsharpImages{1}, unsharpImages{2}, unsharpImages{3}};
    titleList = {'Original low-light', 'Enhanced + denoised', 'Laplacian sharpening', 'Unsharp k=0.5', 'Unsharp k=1.0', 'Unsharp k=1.5'};
    save_image_grid(imageList, titleList, sprintf('Part E - Sharpening: %s', imageFiles(idx).name), fullfile(resultsDir, [outputPrefix,
'_sharpening_comparison.png']));
end

baseStats = compute_image_stats(baseImage, isColorImage);
laplacianStats = compute_image_stats(laplacianImage, isColorImage);
unsharpStats = compute_image_stats(unsharpImages{2}, isColorImage);

fprintf('%02d/%02d %s: gradient base %.4f -> Laplacian %.4f, Unsharp k=1.0 %.4f\n', idx, numel(imageFiles), imageFiles(idx).name,
baseStats.GradientMean, laplacianStats.GradientMean, unsharpStats.GradientMean);
drawnow;
end

resultRows = resultRows(1:rowIdx);
statsTable = struct2table(resultRows);
writetable(statsTable, fullfile(resultsDir, 'B3E_statistics.csv'));

if saveComparisonFigures
    save_e_summary_image(statsTable, fullfile(resultsDir, 'B3E_summary_statistics.png'));
end

fprintf('\nΤο Μέρος E ολοκληρώθηκε.\n');
if saveComparisonFigures
    fprintf('Αποθηκεύτηκαν εικόνες, figures, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);
else
    fprintf('Αποθηκεύτηκαν εικόνες, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);
end

function [imageFiles, imageIds] = find_low_images(projectDir)
    lowFiles = dir(fullfile(projectDir, '*_low.png'));
    keepFile = false(numel(lowFiles), 1);
    imageIds = nan(numel(lowFiles), 1);
    for fileIdx = 1:numel(lowFiles)
        tokens = regexp(lowFiles(fileIdx).name, '^(\d+)_low\.png$', 'tokens', 'once');
        if ~isempty(tokens)
            keepFile(fileIdx) = true;
            imageIds(fileIdx) = str2double(tokens{1});
        end
    end
    imageFiles = lowFiles(keepFile);
    imageIds = imageIds(keepFile);
    if isempty(imageFiles)
        error('Δεν βρέθηκαν εικόνες με μορφή ονόματος number_low.png στον φάκελο: %s', projectDir);
    end
    [imageIds, sortOrder] = sort(imageIds);
    imageFiles = imageFiles(sortOrder);
end

function saveComparisonFigures = should_save_comparison_figures()
    saveComparisonFigures = true;
end

function [imageDouble, isColorImage] = read_image_as_double(imagePath)
    [rawImage, colorMap] = imread(imagePath);
    if ~isempty(colorMap)
        imageDouble = ind2rgb(rawImage, colorMap);
    else
        imageDouble = im2double(rawImage);
    end
    if ndims(imageDouble) == 3 && size(imageDouble, 3) > 3
        imageDouble = imageDouble(:, :, 1:3);
    end
    isColorImage = ndims(imageDouble) == 3 && size(imageDouble, 3) == 3;
end

function baseImage = create_base_enhanced_denoised_image(inputImage)
    gammaImage = min(max(inputImage, 0), 1) .^ 0.6;
    stretchedImage = linear_contrast_stretch(gammaImage);
    baseImage = imgaussfilt(stretchedImage, 0.8, 'FilterSize', 3);
    baseImage = min(max(baseImage, 0), 1);
end

function outputImage = linear_contrast_stretch(inputImage)
    inputMin = min(inputImage(:));
    inputMax = max(inputImage(:));
    if inputMax <= inputMin
        outputImage = zeros(size(inputImage));
    else
        outputImage = (inputImage - inputMin) / (inputMax - inputMin);
    end
end

```

```

end
end

function sharpImage = laplacian_sharpen(inputImage)
    laplacianMask = [0 -1 0; -1 4 -1; 0 -1 0];
    laplacianComponent = imfilter(inputImage, laplacianMask, 'replicate');
    sharpImage = min(max(inputImage + 0.4 * laplacianComponent, 0), 1);
end

function sharpImage = unsharp_mask(inputImage, kValue)
    blurredImage = imgaussfilt(inputImage, 1.0, 'FilterSize', 5);
    detailImage = inputImage - blurredImage;
    sharpImage = min(max(inputImage + kValue * detailImage, 0), 1);
end

function stats = compute_image_stats(inputImage, isColorImage)
    grayImage = to_gray(inputImage, isColorImage);
    pixels = grayImage(:);
    stats.Mean = mean(pixels);
    stats.Std = std(pixels);
    stats.DarkPercent = 100 * mean(pixels <= 0.25);
    stats.SaturatedPercent = 100 * mean(pixels >= 0.98);
    stats.GradientMean = mean(gradient_magnitude(grayImage), 'all');
    stats.LaplacianEnergy = mean(abs(imfilter(grayImage, [0 -1 0; -1 4 -1; 0 -1 0], 'replicate')), 'all');
end

function grayImage = to_gray(inputImage, isColorImage)
    if isColorImage
        grayImage = rgb2gray(inputImage);
    else
        grayImage = inputImage;
    end
end

function magnitude = gradient_magnitude(grayImage)
    sobelX = [-1 0 1; -2 0 2; -1 0 1] / 8;
    sobelY = sobelX';
    gradientX = imfilter(grayImage, sobelX, 'replicate');
    gradientY = imfilter(grayImage, sobelY, 'replicate');
    magnitude = sqrt(gradientX.^2 + gradientY.^2);
end

function [mseValue, psnrValue] = reference_metrics(referenceImage, testImage)
    if isempty(referenceImage)
        mseValue = NaN;
        psnrValue = NaN;
        return;
    end
    if ~isequal(size(referenceImage), size(testImage))
        referenceImage = imresize(referenceImage, [size(testImage, 1), size(testImage, 2)]);
    end
    difference = min(max(testImage, 0), 1) - min(max(referenceImage, 0), 1);
    mseValue = mean(difference(:).^2);
    if mseValue == 0
        psnrValue = Inf;
    else
        psnrValue = 10 * log10(1 / mseValue);
    end
end

function row = make_result_row(imageName, imageNumber, highReferenceName, methodName, parameterText, resultImage, baseImage,
highReferenceImage, isColorImage, commentText)
    resultStats = compute_image_stats(resultImage, isColorImage);
    baseStats = compute_image_stats(baseImage, isColorImage);
    [mseToHigh, psnrToHigh] = reference_metrics(highReferenceImage, resultImage);
    gradientRatio = resultStats.GradientMean / (baseStats.GradientMean + eps);
    row = struct('Image', imageName, 'ImageNumber', imageNumber, 'HighReferenceImage', highReferenceName, 'Method', methodName, 'Parameter',
parameterText, 'MeanBrightness', resultStats.Mean, 'StdContrast', resultStats.Std, 'GradientMean', resultStats.GradientMean,
'GradientRatioToBase', gradientRatio, 'LaplacianEnergy', resultStats.LaplacianEnergy, 'DarkPixelPercent', resultStats.DarkPercent,
'SaturatedPixelPercent', resultStats.SaturatedPercent, 'MSEToHigh', mseToHigh, 'PSNRToHigh', psnrToHigh, 'Comment', commentText);
end

function commentText = make_sharpening_comment(methodName, baseImage, sharpenedImage, isColorImage)
    baseStats = compute_image_stats(baseImage, isColorImage);
    sharpStats = compute_image_stats(sharpenedImage, isColorImage);
    gradientRatio = sharpStats.GradientMean / (baseStats.GradientMean + eps);
    saturationIncrease = sharpStats.SaturatedPercent - baseStats.SaturatedPercent;
    if gradientRatio > 1.15
        detailPart = sprintf('%s αυξάνει την ένταση των ακμών και αναδεικνύει λεπτομέρειες.', methodName);
    else
        detailPart = sprintf('%s έχει ήπια επίδραση στην ανάδειξη ακμών.', methodName);
    end
    if saturationIncrease > 2
        riskPart = 'Υπάρχει κίνδυνος υπερενίσχυσης, κορεσμού ή halos γύρω από έντονες ακμές.';
    elseif gradientRatio > 1.8
        riskPart = 'Η πολύ έντονη αύξηση των ακμών μπορεί να ενισχύσει και θόρυβο.';
    else
        riskPart = 'Η υπερενίσχυση παραμένει σχετικά περιορισμένη.';
    end
    commentText = [detailPart, ' ', riskPart];
end

```

```

end

function commentText = make_unsharp_comment(kValue, baseImage, sharpenedImage, isColorImage)
    baseStats = compute_image_stats(baseImage, isColorImage);
    sharpStats = compute_image_stats(sharpenedImage, isColorImage);
    gradientRatio = sharpStats.GradientMean / (baseStats.GradientMean + eps);
    if kValue <= 0.5
        strengthPart = 'To k=0.5 δίνει ήπια όξυνση και συνήθως πιο φυσικό αποτέλεσμα.';
    elseif kValue <= 1.0
        strengthPart = 'To k=1.0 δίνει πιο καθαρή ανάδειξη λεπτομερειών.';
    else
        strengthPart = 'To k=1.5 δίνει έντονη όξυνση και χρειάζεται προσοχή.';
    end
    if sharpStats.SaturatedPercent > baseStats.SaturatedPercent + 2 || gradientRatio > 1.8
        riskPart = 'H ενίσχυση μπορεί να κάνει πιο εμφανή τον θόρυβο ή halos γύρω από ακμές.';
    else
        riskPart = 'Δεν εμφανίζεται μεγάλη αύξηση κορεσμού ή υπερβολική ενίσχυση ακμών.';
    end
    commentText = sprintf('%s O λόγος gradient προς τη βάση είναι %.3f. %s', strengthPart, gradientRatio, riskPart);
end

function safeName = make_safe_filename(fileName)
    safeName = regexprep(fileName, '[^A-Za-z0-9]+', '_');
    safeName = regexprep(safeName, '^_+|_+$', '');
    if isempty(safeName)
        safeName = 'image';
    end
end

function textValue = value_or_none(inputText)
    if isempty(inputText)
        textValue = 'δεν βρέθηκε';
    else
        textValue = inputText;
    end
end

function save_e_summary_image(statsTable, outputPath)
    methods = {'Base enhanced denoised', 'Laplacian sharpening', 'Unsharp masking'};
    labels = {'Base', 'Laplacian', 'Unsharp k=1.0'};
    metricNames = {'GradientMean', 'SaturatedPixelPercent'};
    metricTitles = {'Gradient mean', 'Saturated pixels (%)'};

    hFig = figure('Visible', 'off', 'Color', 'w', 'Position', [100, 100, 1200, 760]);
    cleanupObj = onCleanup(@() close(hFig));

    for metricIdx = 1:numel(metricNames)
        subplot(2, 1, metricIdx);
        hold on;
        for methodIdx = 1:numel(methods)
            rows = strcmp(statsTable.Method, methods{methodIdx});
            if strcmp(methods{methodIdx}, 'Unsharp masking')
                rows = rows & strcmp(statsTable.Parameter, 'k = 1.0');
            end
            if ~any(rows)
                continue;
            end
            [xValues, sortIdx] = sort(statsTable.ImageNumber(rows));
            yValues = statsTable.(metricNames{metricIdx})(rows);
            yValues = yValues(sortIdx);
            plot(xValues, yValues, '-o', 'LineWidth', 1.4, 'MarkerSize', 4, 'DisplayName', labels{methodIdx});
        end
        hold off;
        grid on;
        title(metricTitles{metricIdx});
        xlabel('Image number');
        ylabel(metricTitles{metricIdx});
        legend('Location', 'bestoutside');
    end

    try
        sgtitle('Part E - Sharpening summary');
    catch
    end

    try
        exportgraphics(hFig, outputPath, 'Resolution', 150);
    catch
        saveas(hFig, outputPath);
    end

    clear cleanupObj;
end

function canvas = draw_metric_chart(canvas, statsTable, methods, labels, metricName, titleText, rect)
    xSeries = cell(numel(methods), 1);
    ySeries = cell(numel(methods), 1);
    for methodIdx = 1:numel(methods)
        rows = strcmp(statsTable.Method, methods{methodIdx});
        if strcmp(methods{methodIdx}, 'Unsharp masking')

```

```

        rows = rows & strcmp(statsTable.Parameter, 'k = 1.0');
    end
    [xSeries{methodIdx}, sortIdx] = sort(statsTable.ImageNumber(rows));
    values = statsTable.(metricName)(rows);
    ySeries{methodIdx} = values(sortIdx);
end
canvas = draw_line_chart(canvas, xSeries, ySeries, labels, titleText, metricName, rect);
end

function canvas = draw_line_chart(canvas, xSeries, ySeries, labels, titleText, ylabel, rect)
    colors = [0.12 0.35 0.75; 0.85 0.33 0.10; 0.15 0.55 0.25; 0.55 0.25 0.70];
    panelLeft = rect(1);
    panelTop = rect(2);
    panelWidth = rect(3);
    panelHeight = rect(4);
    plotLeft = panelLeft + 75;
    plotTop = panelTop + 55;
    plotWidth = panelWidth - 150;
    plotHeight = panelHeight - 115;

    allX = [];
    allY = [];
    for idx = 1:numel(ySeries)
        allX = [allX; xSeries{idx}(:)]; %#ok<AGROW>
        allY = [allY; ySeries{idx}(:)]; %#ok<AGROW>
    end
    allY = allY(isfinite(allY));
    if isempty(allX) || isempty(allY)
        return;
    end
    xMin = min(allX);
    xMax = max(allX);
    yMin = min(allY);
    yMax = max(allY);
    if xMax <= xMin
        xMax = xMin + 1;
    end
    if yMax <= yMin
        yMax = yMin + 1;
    end
    yPad = 0.08 * (yMax - yMin);
    yMin = yMin - yPad;
    yMax = yMax + yPad;

    canvas = draw_line(canvas, plotLeft, plotTop + plotHeight, plotLeft + plotWidth, plotTop + plotHeight, [0 0 0], 2);
    canvas = draw_line(canvas, plotLeft, plotTop, plotLeft, plotTop + plotHeight, [0 0 0], 2);
    for gridIdx = 1:4
        yGrid = round(plotTop + plotHeight * gridIdx / 5);
        canvas = draw_line(canvas, plotLeft, yGrid, plotLeft + plotWidth, yGrid, [0.85 0.85 0.85], 1);
    end

    for idx = 1:numel(ySeries)
        xValues = xSeries{idx};
        yValues = ySeries{idx};
        valid = isfinite(xValues) & isfinite(yValues);
        xValues = xValues(valid);
        yValues = yValues(valid);
        if isempty(xValues)
            continue;
        end
        xPix = round(plotLeft + (xValues - xMin) / (xMax - xMin) * plotWidth);
        yPix = round(plotTop + plotHeight - (yValues - yMin) / (yMax - yMin) * plotHeight);
        colorValue = colors(1 + mod(idx - 1, size(colors, 1)), :);
        for pointIdx = 1:numel(xPix) - 1
            canvas = draw_line(canvas, xPix(pointIdx), yPix(pointIdx), xPix(pointIdx + 1), yPix(pointIdx + 1), colorValue, 3);
        end
        for pointIdx = 1:numel(xPix)
            canvas = draw_square(canvas, xPix(pointIdx), yPix(pointIdx), colorValue, 4);
        end
    end

    if exist('insertText', 'file') == 2
        canvas = insertText(canvas, [panelLeft + 10, panelTop + 8], titleText, 'FontSize', 22, 'BoxOpacity', 0, 'TextColor', 'black');
        canvas = insertText(canvas, [plotLeft, plotTop + plotHeight + 16], 'Image number', 'FontSize', 16, 'BoxOpacity', 0, 'TextColor',
'black');
        canvas = insertText(canvas, [panelLeft + 10, plotTop + 4], ylabel, 'FontSize', 16, 'BoxOpacity', 0, 'TextColor', 'black');
        for idx = 1:numel(labels)
            legendX = panelLeft + panelWidth - 230;
            legendY = panelTop + 42 + 26 * idx;
            canvas = draw_line(canvas, legendX, legendY + 9, legendX + 28, legendY + 9, colors(1 + mod(idx - 1, size(colors, 1)), :), 4);
            canvas = insertText(canvas, [legendX + 35, legendY], labels{idx}, 'FontSize', 15, 'BoxOpacity', 0, 'TextColor', 'black');
        end
    end
end

function save_image_grid(imagelist, titleList, mainTitle, outputPath)
    validIdx = find(~cellfun(@isempty, imagelist));
    if isempty(validIdx)
        return;
    end
end

```

```

tileHeight = 260;
tileWidth = 390;
topMargin = 70;
labelHeight = 34;
gap = 18;
cols = min(3, numel(validIdx));
rows = ceil(numel(validIdx) / cols);
canvasHeight = topMargin + rows * (labelHeight + tileHeight) + (rows + 1) * gap;
canvasWidth = cols * tileWidth + (cols + 1) * gap;
canvas = ones(canvasHeight, canvasWidth, 3);
if exist('insertText', 'file') == 2
    canvas = insertText(canvas, [round(canvasWidth / 2) - 260, 18], mainTitle, 'FontSize', 24, 'BoxOpacity', 0, 'TextColor', 'black');
end
for tileIdx = 1:numel(validIdx)
    imageIdx = validIdx(tileIdx);
    rowIdx = floor((tileIdx - 1) / cols);
    colIdx = mod(tileIdx - 1, cols);
    xStart = gap + colIdx * (tileWidth + gap) + 1;
    yLabel = topMargin + gap + rowIdx * (labelHeight + tileHeight + gap) + 1;
    yStart = yLabel + labelHeight;
    tileImage = prepare_grid_image(imageList{imageIdx}, tileHeight, tileWidth);
    if exist('insertText', 'file') == 2
        canvas = insertText(canvas, [xStart + 8, yLabel + 4], titleList{imageIdx}, 'FontSize', 18, 'BoxOpacity', 0, 'TextColor',
'black');
    end
    canvas(yStart:yStart + tileHeight - 1, xStart:xStart + tileWidth - 1, :) = tileImage;
end
imwrite(canvas, outputPath);
end

function outputImage = prepare_grid_image(inputImage, targetHeight, targetWidth)
inputImage = min(max(inputImage, 0), 1);
if islogical(inputImage)
    inputImage = double(inputImage);
end
if ismatrix(inputImage)
    inputImage = repmat(inputImage, [1 1 3]);
else
    inputImage = inputImage(:, :, 1:3);
end
outputImage = imresize(inputImage, [targetHeight targetWidth]);
end

function canvas = draw_line(canvas, x1, y1, x2, y2, colorValue, thickness)
steps = max(abs(x2 - x1), abs(y2 - y1)) + 1;
xValues = round(linspace(x1, x2, steps));
yValues = round(linspace(y1, y2, steps));
for idx = 1:numel(xValues)
    canvas = draw_square(canvas, xValues(idx), yValues(idx), colorValue, thickness);
end
end

function canvas = draw_square(canvas, xCenter, yCenter, colorValue, radius)
[height, width, ~] = size(canvas);
xRange = max(1, xCenter - radius):min(width, xCenter + radius);
yRange = max(1, yCenter - radius):min(height, yCenter + radius);
for channelId = 1:3
    channel = canvas(:, :, channelId);
    channel(yRange, xRange) = colorValue(channelId);
    canvas(:, :, channelId) = channel;
end
end
end

```

Μέρη ΣΤ, Ζ και Η (Αρχείο: eikonaB3FGH.m)

Ενωτικό Αρχείο κώδικα καθώς τα Μέρη Ζ και Η εργάζονται πάνω στα αποτελέσματα των pipelines του Μέρους ΣΤ. Έχουν χωριστεί τα επί μέρους μέρη στο κομμάτι κυρίως της επεξεργασίας με σχόλια.

```

% Μέρος ΣΤ
clear;
clc;
close all;

greekDesktop = char([933 960 959 955 959 947 953 963 964 942 962]);
% Replace with your path
projectDir = fullfile(getenv('USERPROFILE'), 'OneDrive', greekDesktop, 'project');
resultsDir = fullfile(projectDir, 'resultsB3FGH');
summaryDir = fullfile(resultsDir, 'summary');
detailsDir = fullfile(resultsDir, 'details');
metricsDir = fullfile(resultsDir, 'metrics');
pipelineDetailsDir = fullfile(detailsDir, 'pipeline_images');

```

```

edgeDetailsDir = fullfile(detailsDir, 'edge_maps');
expectedFolderName = 'resultsB3FGH';

[~, actualFolderName] = fileparts(resultsDir);
if isempty(resultsDir) || strcmp(resultsDir, projectDir) || ~strcmp(actualFolderName, expectedFolderName)
    error('Unsafe results folder: %s', resultsDir);
end

if ~exist(resultsDir, 'dir')
    mkdir(resultsDir);
end

oldFiles = [dir(fullfile(resultsDir, '*.png')); dir(fullfile(resultsDir, '*.csv')); dir(fullfile(resultsDir, '*.txt'))];
dir(fullfile(resultsDir, '*.mat'));
oldDirs = {summaryDir, detailsDir, metricsDir};
oldWarningState = warning('off', 'all');
for oldIdx = 1:numel(oldFiles)
    oldPath = fullfile(resultsDir, oldFiles(oldIdx).name);
    if exist(oldPath, 'file') == 2
        try
            delete(oldPath);
        catch
        end
    end
end
for oldIdx = 1:numel(oldDirs)
    if exist(oldDirs{oldIdx}, 'dir')
        try
            rmdir(oldDirs{oldIdx}, 's');
        catch
        end
    end
end
warning(oldWarningState);

mkdir(summaryDir);
mkdir(metricsDir);
mkdir(detailsDir);
mkdir(pipelineDetailsDir);
mkdir(edgeDetailsDir);

[imageFiles, imageIds] = find_low_images(projectDir);
retinexSigmas = [15 30 60];

pipelineTemplate = struct('Image', '', 'ImageNumber', 0, 'Pipeline', '', 'Parameter', '', 'MeanBrightness', 0, 'StdContrast', 0,
    'EntropyValue', 0, 'DarkPixelPercent', 0, 'SaturatedPixelPercent', 0, 'Comment', '');
pipelineRows = repmat(pipelineTemplate, numel(imageFiles) * 6, 1);
pipelineRowIdx = 0;

reportFile = fullfile(metricsDir, 'B3FGH_part_ST_comments.txt');
reportFid = fopen(reportFile, 'w', 'n', 'UTF-8');
if reportFid == -1
    error('Δεν ήταν δυνατή η δημιουργία του αρχείου: %s', reportFile);
end
reportCleanup = onCleanup(@( ) fclose(reportFid));

fprintf(reportFid, 'Μέρος ΣΤ: Σύγκριση πλήρων pipelines\n');
fprintf(reportFid, 'Pipeline 1: gamma correction και linear contrast stretching.\n');
fprintf(reportFid, 'Pipeline 2: median filtering, CLAHE και unsharp masking.\n');
fprintf(reportFid, 'Pipeline 3: Retinex-like προσέγγιση με sigma = 15, 30, 60.\n\n');

fprintf('Μέρος ΣΤ: βρέθηκαν %d εικόνες χαμηλού φωτισμού.\n', numel(imageFiles));
fprintf('Τα αποτελέσματα θα αποθηκευτούν εδώ:\n%s\n', resultsDir);

for idx = 1:numel(imageFiles)
    imagePath = fullfile(projectDir, imageFiles(idx).name);
    [lowImage, isColorImage] = read_image_as_double(imagePath);
    [~, baseName] = fileparts(imageFiles(idx).name);
    safeBaseName = make_safe_filename(baseName);
    outputPrefix = sprintf('%02d_%s', idx, safeBaseName);

    pipeline1Image = pipeline_simple_brightness(lowImage);
    pipeline2Image = pipeline_equalization_denoising(lowImage, isColorImage);
    retinexImages = cell(numel(retinexSigmas), 1);

    for sigmaIdx = 1:numel(retinexSigmas)
        retinexImages{sigmaIdx} = pipeline_retinex(lowImage, isColorImage, retinexSigmas(sigmaIdx));
    end

    pipelineImages = [{lowImage}; {pipeline1Image}; {pipeline2Image}; retinexImages(:)};
    pipelineNames = [{'Original low-light'}; {'Pipeline 1 - Gamma + stretch'}; {'Pipeline 2 - Median + CLAHE + unsharp'}; arrayfun(@(sigma)
        sprintf('Pipeline 3 - Retinex sigma %d', sigma), retinexSigmas(:), 'UniformOutput', false)];
    pipelineParameters = [{'']; {'gamma=0.6, full range stretch'}; {'median 3x3, CLAHE 8x8 clip 0.01, unsharp k=0.8'}; arrayfun(@(sigma)
        sprintf('sigma=%d', sigma), retinexSigmas(:), 'UniformOutput', false)];

    imwrite(lowImage, fullfile(pipelineDetailsDir, [outputPrefix, '_original_low.png']));
    imwrite(pipeline1Image, fullfile(pipelineDetailsDir, [outputPrefix, '_pipeline1_gamma_stretch.png']));
    imwrite(pipeline2Image, fullfile(pipelineDetailsDir, [outputPrefix, '_pipeline2_median_clahe_unsharp.png']));

    for sigmaIdx = 1:numel(retinexSigmas)

```

```

        imwrite(retinexImages{sigmaIdx}, fullfile(pipelineDetailsDir, sprintf('%s_pipeline3_retinex_sigma%d.png', outputPrefix,
retinexSigmas(sigmaIdx))));
    end

    pipelineGridTitles = {'Original', 'Pipeline 1', 'Pipeline 2', 'Retinex sigma 15', 'Retinex sigma 30', 'Retinex sigma 60'};
    save_image_grid(pipelineImages, pipelineGridTitles, sprintf('Μέρος ΣΤ - Pipelines: %s', imageFiles(idx).name), fullfile(summaryDir,
[outputPrefix, '_ST_pipeline_grid.png']));

    fprintf(reportFid, 'Εικόνα %02d: %s\n', idx, imageFiles(idx).name);
    for pipelineIdx = 1:numel(pipelineImages)
        currentImage = pipelineImages{pipelineIdx};
        currentName = pipelineNames{pipelineIdx};
        currentParameter = pipelineParameters{pipelineIdx};
        currentComment = make_pipeline_comment(currentName, currentParameter, lowImage, currentImage, isColorImage);

        pipelineRowIdx = pipelineRowIdx + 1;
        pipelineRows(pipelineRowIdx) = make_pipeline_row(imageFiles(idx).name, imageIds(idx), currentName, currentParameter, currentImage,
isColorImage, currentComment);

        if pipelineIdx > 1
            fprintf(reportFid, '%s: %s\n', currentName, currentComment);
        end
    end
    fprintf(reportFid, '\n');

    fprintf('%02d/%02d %s: ολοκληρώθηκαν τα τρία pipelines\n', idx, numel(imageFiles), imageFiles(idx).name);
    drawnow;
end

pipelineRows = pipelineRows(1:pipelineRowIdx);
pipelineTable = struct2table(pipelineRows);
pipelineSummaryTable = make_pipeline_summary_table(pipelineTable);
writetable(pipelineTable, fullfile(metricsDir, 'B3FGH_part_ST_pipeline_metrics.csv'));
writetable(pipelineSummaryTable, fullfile(metricsDir, 'B3FGH_summary_pipeline_means.csv'));
write_summary_report(reportFid, pipelineTable);

% Μέρος Ζ
zTemplate = struct('Image', '', 'ImageNumber', 0, 'HighReferenceImage', '', 'Pipeline', '', 'Parameter', '', 'MSEToHigh', NaN, 'PSNRToHigh',
NaN, 'MeanBrightness', 0, 'StdContrast', 0, 'MichelsonContrast', 0, 'EntropyValue', 0, 'Comment', '');
zRows = repmat(zTemplate, height(pipelineTable), 1);
zRowIdx = 0;

zReportFile = fullfile(metricsDir, 'B3FGH_part_Z_quantitative_evaluation.txt');
zReportFid = fopen(zReportFile, 'w', 'n', 'UTF-8');
if zReportFid == -1
    error('Δεν ήταν δυνατή η δημιουργία του αρχείου: %s', zReportFile);
end
zReportCleanup = onCleanup(@() fclose(zReportFid));

fprintf(zReportFid, 'Μέρος Ζ: Ποσοτική αξιολόγηση\n');
fprintf(zReportFid, 'Υπολογίζονται MSE, PSNR, μέση φωτεινότητα, τυπική απόκλιση ως αντίθεση, Michelson contrast και entropy.\n\n');

for idx = 1:numel(imageFiles)
    highReferenceName = sprintf('%d_high.png', imageIds(idx));
    highReferencePath = fullfile(projectDir, highReferenceName);
    if exist(highReferencePath, 'file') == 2
        [highImage, ~] = read_image_as_double(highReferencePath);
    else
        highReferenceName = '';
        highImage = [];
    end

    [~, baseName] = fileparts(imageFiles(idx).name);
    safeBaseName = make_safe_filename(baseName);
    outputPrefix = sprintf('%02d_%s', idx, safeBaseName);

    zImageFiles = [{[outputPrefix, '_original_low.png']}; {[outputPrefix, '_pipeline1_gamma_stretch.png']}; {[outputPrefix,
'_pipeline2_median_clahe_unsharp.png']}; arrayfun(@(sigma) sprintf('%s_pipeline3_retinex_sigma%d.png', outputPrefix, sigma),
retinexSigmas(:), 'UniformOutput', false)];
    zPipelineNames = [{'Original low-light'}; {'Pipeline 1 - Gamma + stretch'}; {'Pipeline 2 - Median + CLAHE + unsharp'}; arrayfun(@(sigma)
sprintf('Pipeline 3 - Retinex sigma %d', sigma), retinexSigmas(:), 'UniformOutput', false)];
    zPipelineParameters = [{'']; {'gamma=0.6, full range stretch'}; {'median 3x3, CLAHE 8x8 clip 0.01, unsharp k=0.8'}; arrayfun(@(sigma)
sprintf('sigma=%d', sigma), retinexSigmas(:), 'UniformOutput', false)];

    fprintf(zReportFid, 'Εικόνα %02d: %s\n', idx, imageFiles(idx).name);
    fprintf(zReportFid, 'Φωτεινή εικόνα αναφοράς: %s\n', value_or_none(highReferenceName));

    for pipelineIdx = 1:numel(zImageFiles)
        resultPath = fullfile(pipelineDetailsDir, zImageFiles{pipelineIdx});
        if exist(resultPath, 'file') ~= 2
            continue;
        end

        [resultImage, resultIsColorImage] = read_image_as_double(resultPath);
        currentName = zPipelineNames{pipelineIdx};
        currentParameter = zPipelineParameters{pipelineIdx};
        currentComment = make_z_comment(currentName, resultImage, highImage, resultIsColorImage);

        zRowIdx = zRowIdx + 1;

```

```

        zRows(zRowIdx) = make_z_row(imageFiles(idx).name, imageIds(idx), highReferenceName, currentName, currentParameter, resultImage,
highImage, resultIsColorImage, currentComment);

        if pipelineIdx > 1
            fprintf(zReportFid, '%s: %s\n', currentName, currentComment);
        end
    end
    fprintf(zReportFid, '\n');
end

zRows = zRows(1:zRowIdx);
zTable = struct2table(zRows);
bestPipelineTable = make_best_pipeline_summary(zTable);
writetable(zTable, fullfile(metricsDir, 'B3FGH_part_Z_quantitative_metrics.csv'));
writetable(bestPipelineTable, fullfile(metricsDir, 'B3FGH_summary_best_pipeline.csv'));
write_z_summary_report(zReportFid, zTable);

% Μέρος H
edgeMethods = {'Sobel', 'Canny'};
hTemplate = struct('Image', '', 'ImageNumber', 0, 'HighReferenceImage', '', 'Pipeline', '', 'Parameter', '', 'EdgeDetector', '', 'EdgeCount',
0, 'EdgeDensity', 0, 'ComponentCount', 0, 'SmallComponentPercent', 0, 'SignificantComponentPercent', 0, 'JaccardToHigh', NaN,
'EdgeCountRatioToOriginal', 0, 'Comment', '');
hRows = repmat(hTemplate, height(pipelineTable) * numel(edgeMethods), 1);
hRowIdx = 0;

hReportFile = fullfile(metricsDir, 'B3FGH_part_H_edge_evaluation.txt');
hReportFid = fopen(hReportFile, 'w', 'n', 'UTF-8');
if hReportFid == -1
    error('Δεν ήταν δυνατή η δημιουργία του αρχείου: %s', hReportFile);
end
hReportCleanup = onCleanup(@() fclose(hReportFid));

fprintf(hReportFid, 'Μέρος H: Επίδραση των μεθόδων σε ακμές και χαρακτηριστικά\n');
fprintf(hReportFid, 'Εφαρμόζονται Sobel και Canny στην αρχική low-light εικόνα και στις βελτιωμένες εικόνες.\n');
fprintf(hReportFid, 'Συγκρίνονται πλήθος ακμών, πυκνότητα, μικρά components ως ένδειξη θορύβου και μεγαλύτερα components ως ένδειξη ορατών
αντικειμένων.\n\n');

for idx = 1:numel(imageFiles)
    highReferenceName = sprintf('%d_high.png', imageIds(idx));
    highReferencePath = fullfile(projectDir, highReferenceName);
    if exist(highReferencePath, 'file') == 2
        [highImage, ~] = read_image_as_double(highReferencePath);
    else
        highReferenceName = '';
        highImage = [];
    end

    [~, baseName] = fileparts(imageFiles(idx).name);
    safeBaseName = make_safe_filename(baseName);
    outputPrefix = sprintf('%02d_%s', idx, safeBaseName);

    hImageFiles = [{outputPrefix, '_original_low.png'}; {outputPrefix, '_pipeline1_gamma_stretch.png'}; {outputPrefix,
'_pipeline2_median_clahe_unsharp.png'}]; arrayfun(@(sigma) sprintf('%s_pipeline3_retinex_sigma_%d.png', outputPrefix, sigma),
retinexSigmas(:), 'UniformOutput', false);
    hPipelineNames = [{'Original low-light'}; {'Pipeline 1 - Gamma + stretch'}; {'Pipeline 2 - Median + CLAHE + unsharp'}; arrayfun(@(sigma)
sprintf('Pipeline 3 - Retinex sigma %d', sigma), retinexSigmas(:), 'UniformOutput', false)];
    hPipelineParameters = [{''}; {'gamma=0.6, full range stretch'}; {'median 3x3, CLAHE 8x8 clip 0.01, unsharp k=0.8'}; arrayfun(@(sigma)
sprintf('sigma=%d', sigma), retinexSigmas(:), 'UniformOutput', false)];

    originalPath = fullfile(pipelineDetailsDir, hImageFiles{1});
    [originalImage, originalIsColorImage] = read_image_as_double(originalPath);
    originalGray = to_gray(originalImage, originalIsColorImage);
    if isempty(highImage)
        highGray = [];
    else
        highForEdges = resize_to_match(highImage, originalImage);
        highForEdgesIsColor = ndims(highForEdges) == 3 && size(highForEdges, 3) == 3;
        highGray = to_gray(highForEdges, highForEdgesIsColor);
    end

    fprintf(hReportFid, 'Εικόνα %02d: %s\n', idx, imageFiles(idx).name);
    fprintf(hReportFid, 'Φωτεινή εικόνα αναφοράς: %s\n', value_or_none(highReferenceName));

    for edgeIdx = 1:numel(edgeMethods)
        edgeMethod = edgeMethods{edgeIdx};
        originalEdge = edge(originalGray, lower(edgeMethod));
        if isempty(highGray)
            highEdge = [];
        else
            highEdge = edge(highGray, lower(edgeMethod));
        end

        edgeGridImages = cell(numel(hImageFiles), 1);
        edgeGridTitles = {'Original', 'Pipeline 1', 'Pipeline 2', 'Retinex sigma 15', 'Retinex sigma 30', 'Retinex sigma 60'};
        edgeGridImages{1} = originalEdge;

        edgeOutputName = sprintf('%s_original_low_edges_%s.png', outputPrefix, lower(edgeMethod));
        imwrite(originalEdge, fullfile(edgeDetailsDir, edgeOutputName));

        hRowIdx = hRowIdx + 1;
    end
end

```

```

        edgeComment = make_h_comment('Original low-light', originalEdge, originalEdge, highEdge);
        hRows(hRowIdx) = make_h_row(imageFiles(idx).name, imageIds(idx), highReferenceName, 'Original low-light', '', edgeMethod,
originalEdge, originalEdge, highEdge, edgeComment);

        for pipelineIdx = 2:numel(hImageFiles)
            resultPath = fullfile(pipelineDetailsDir, hImageFiles(pipelineIdx));
            if exist(resultPath, 'file') ~= 2
                continue;
            end

            [resultImage, resultIsColorImage] = read_image_as_double(resultPath);
            resultGray = to_gray(resultImage, resultIsColorImage);
            resultEdge = edge(resultGray, lower(edgeMethod));
            currentName = hPipelineNames(pipelineIdx);
            currentParameter = hPipelineParameters(pipelineIdx);
            edgeOutputName = sprintf('%s_%s_edges_%s.png', outputPrefix, make_safe_filename(currentName), lower(edgeMethod));
            imwrite(resultEdge, fullfile(edgeDetailsDir, edgeOutputName));
            edgeGridImages{pipelineIdx} = resultEdge;

            hRowIdx = hRowIdx + 1;
            edgeComment = make_h_comment(currentName, resultEdge, originalEdge, highEdge);
            hRows(hRowIdx) = make_h_row(imageFiles(idx).name, imageIds(idx), highReferenceName, currentName, currentParameter, edgeMethod,
resultEdge, originalEdge, highEdge, edgeComment);
        end

        save_image_grid(edgeGridImages, edgeGridTitles, sprintf('Μέρος H - %s edges: %s', edgeMethod, imageFiles(idx).name),
fullfile(summaryDir, sprintf('%s_H_edges_%s_grid.png', outputPrefix, lower(edgeMethod))));
    end

    fprintf(hReportFid, 'Σχόλιο: %s\n\n', make_h_image_summary(hRows(1:hRowIdx), imageFiles(idx).name));
end

hRows = hRows(1:hRowIdx);
hTable = struct2table(hRows);
edgeSummaryTable = make_edge_summary_table(hTable);
writetable(hTable, fullfile(metricsDir, 'B3FGH_part_H_edge_metrics.csv'));
writetable(edgeSummaryTable, fullfile(metricsDir, 'B3FGH_summary_edge_canny.csv'));
write_h_summary_report(hReportFid, hTable);

fprintf('\nΤα Μέρη ΣΤ, Ζ και Η ολοκληρώθηκαν.\n');
fprintf('Αποθηκεύτηκαν εικόνες pipelines, edge maps, συγκριτικά πλαίσια, στατιστικά και σχόλια εδώ:\n%s\n', resultsDir);

function [imageFiles, imageIds] = find_low_images(projectDir)
    lowFiles = dir(fullfile(projectDir, '*_low.png'));
    keepFile = false(numel(lowFiles), 1);
    imageIds = nan(numel(lowFiles), 1);
    for fileIdx = 1:numel(lowFiles)
        tokens = regexp(lowFiles(fileIdx).name, '^(\\d+)_low\\.png$', 'tokens', 'once');
        if ~isempty(tokens)
            keepFile(fileIdx) = true;
            imageIds(fileIdx) = str2double(tokens{1});
        end
    end
    imageFiles = lowFiles(keepFile);
    imageIds = imageIds(keepFile);
    if isempty(imageFiles)
        error('Δεν βρέθηκαν εικόνες με μορφή ονόματος number_low.png στον φάκελο: %s', projectDir);
    end
    [imageIds, sortOrder] = sort(imageIds);
    imageFiles = imageFiles(sortOrder);
end

function [imageDouble, isColorImage] = read_image_as_double(imagePath)
    [rawImage, colorMap] = imread(imagePath);
    if ~isempty(colorMap)
        imageDouble = ind2rgb(rawImage, colorMap);
    else
        imageDouble = im2double(rawImage);
    end
    if ndims(imageDouble) == 3 && size(imageDouble, 3) > 3
        imageDouble = imageDouble(:, :, 1:3);
    end
    isColorImage = ndims(imageDouble) == 3 && size(imageDouble, 3) == 3;
end

function outputImage = pipeline_simple_brightness(inputImage)
    gammaImage = min(max(inputImage, 0), 1) .^ 0.6;
    outputImage = linear_contrast_stretch(gammaImage);
end

function outputImage = pipeline_equalization_denoising(inputImage, isColorImage)
    denoisedImage = apply_median_filter(inputImage);
    brightness = extract_brightness(denoisedImage, isColorImage);
    enhancedBrightness = adapthisteq(brightness, 'NumTiles', [8 8], 'ClipLimit', 0.01, 'Distribution', 'uniform');
    claheImage = replace_brightness(denoisedImage, enhancedBrightness, isColorImage);
    outputImage = unsharp_mask(claheImage, 0.8);
end

function outputImage = pipeline_retinex(inputImage, isColorImage, sigmaValue)
    brightness = extract_brightness(inputImage, isColorImage);

```

```

    epsilonValue = 1e-3;
    illumination = imgaussfilt(brightness, sigmaValue);
    retinexImage = log(brightness + epsilonValue) - log(illumination + epsilonValue);
    retinexImage = robust_rescale(retinexImage, 1, 99);
    retinexImage = retinexImage .^ 0.85;
    outputImage = replace_brightness(inputImage, retinexImage, isColorImage);
end

function outputImage = apply_median_filter(inputImage)
    if ndims(inputImage) == 3
        outputImage = zeros(size(inputImage));
        for channelIdx = 1:size(inputImage, 3)
            outputImage(:, :, channelIdx) = medfilt2(inputImage(:, :, channelIdx), [3 3], 'symmetric');
        end
    else
        outputImage = medfilt2(inputImage, [3 3], 'symmetric');
    end
end

function outputImage = unsharp_mask(inputImage, kValue)
    blurredImage = imgaussfilt(inputImage, 1.0, 'FilterSize', 5);
    detailImage = inputImage - blurredImage;
    outputImage = min(max(inputImage + kValue * detailImage, 0), 1);
end

function outputImage = linear_contrast_stretch(inputImage)
    inputMin = min(inputImage(:));
    inputMax = max(inputImage(:));
    if inputMax <= inputMin
        outputImage = zeros(size(inputImage));
    else
        outputImage = (inputImage - inputMin) / (inputMax - inputMin);
    end
end

function brightness = extract_brightness(inputImage, isColorImage)
    if isColorImage
        hsvImage = rgb2hsv(inputImage);
        brightness = hsvImage(:, :, 3);
    else
        brightness = inputImage;
    end
end

function outputImage = replace_brightness(inputImage, brightness, isColorImage)
    brightness = min(max(brightness, 0), 1);
    if isColorImage
        hsvImage = rgb2hsv(inputImage);
        hsvImage(:, :, 3) = brightness;
        outputImage = hsv2rgb(hsvImage);
    else
        outputImage = brightness;
    end
end

function outputImage = robust_rescale(inputImage, lowPercentile, highPercentile)
    lowValue = percentile_value(inputImage(:), lowPercentile);
    highValue = percentile_value(inputImage(:), highPercentile);
    if highValue <= lowValue
        outputImage = linear_contrast_stretch(inputImage);
    else
        outputImage = (inputImage - lowValue) / (highValue - lowValue);
        outputImage = min(max(outputImage, 0), 1);
    end
end

function value = percentile_value(values, percentile)
    values = sort(values(:));
    if isempty(values)
        value = 0;
        return;
    end
    position = 1 + (numel(values) - 1) * percentile / 100;
    lowerIndex = floor(position);
    upperIndex = ceil(position);
    if lowerIndex == upperIndex
        value = values(lowerIndex);
    else
        value = values(lowerIndex) + (position - lowerIndex) * (values(upperIndex) - values(lowerIndex));
    end
end

function row = make_pipeline_row(imageName, imageNumber, pipelineName, parameterText, resultImage, isColorImage, commentText)
    stats = compute_image_stats(resultImage, isColorImage);
    row = struct('Image', imageName, 'ImageNumber', imageNumber, 'Pipeline', pipelineName, 'Parameter', parameterText, 'MeanBrightness',
stats.Mean, 'StdContrast', stats.Std, 'EntropyValue', stats.Entropy, 'DarkPixelPercent', stats.DarkPercent, 'SaturatedPixelPercent',
stats.SaturatedPercent, 'Comment', commentText);
end

function stats = compute_image_stats(inputImage, isColorImage)

```

```

    grayImage = to_gray(inputImage, isColorImage);
    pixels = grayImage(:);
    stats.Mean = mean(pixels);
    stats.Std = std(pixels);
    stats.DarkPercent = 100 * mean(pixels <= 0.25);
    stats.SaturatedPercent = 100 * mean(pixels >= 0.98);
    stats.Michelson = (max(pixels) - min(pixels)) / (max(pixels) + min(pixels) + eps);
    stats.Entropy = compute_entropy(grayImage);
end

function entropyValue = compute_entropy(grayImage)
    uintValues = uint8(round(255 * min(max(grayImage, 0), 1)));
    counts = histcounts(double(uintValues(:)), -0.5:1:255.5);
    probabilities = counts / sum(counts);
    probabilities = probabilities(probabilities > 0);
    entropyValue = -sum(probabilities .* log2(probabilities));
end

function grayImage = to_gray(inputImage, isColorImage)
    if isColorImage
        grayImage = rgb2gray(inputImage);
    else
        grayImage = inputImage;
    end
end

function save_image_grid(imageList, titleList, mainTitle, outputPath)
    validIdx = find(~cellfun(@isempty, imageList));
    if isempty(validIdx)
        return;
    end
    tileHeight = 260;
    tileWidth = 390;
    topMargin = 70;
    labelHeight = 34;
    gap = 18;
    rows = 2;
    cols = 3;
    canvasHeight = topMargin + rows * (labelHeight + tileHeight) + (rows + 1) * gap;
    canvasWidth = cols * tileWidth + (cols + 1) * gap;
    canvas = ones(canvasHeight, canvasWidth, 3);
    if exist('insertText', 'file') == 2
        canvas = insertText(canvas, [round(canvasWidth / 2) - 330, 18], mainTitle, 'FontSize', 24, 'BoxOpacity', 0, 'TextColor', 'black');
    end
    for tileIdx = 1:numel(validIdx)
        imageIdx = validIdx(tileIdx);
        rowIdx = floor((tileIdx - 1) / cols);
        colIdx = mod(tileIdx - 1, cols);
        xStart = gap + colIdx * (tileWidth + gap) + 1;
        yLabel = topMargin + gap + rowIdx * (labelHeight + tileHeight + gap) + 1;
        yStart = yLabel + labelHeight;
        tileImage = prepare_grid_image(imageList{imageIdx}, tileHeight, tileWidth);
        if exist('insertText', 'file') == 2
            canvas = insertText(canvas, [xStart + 8, yLabel + 4], titleList{imageIdx}, 'FontSize', 18, 'BoxOpacity', 0, 'TextColor',
'black');
        end
        canvas(yStart:yStart + tileHeight - 1, xStart:xStart + tileWidth - 1, :) = tileImage;
    end
    imwrite(canvas, outputPath);
end

function outputImage = prepare_grid_image(inputImage, targetHeight, targetWidth)
    inputImage = min(max(inputImage, 0), 1);
    if islogical(inputImage)
        inputImage = double(inputImage);
    end
    if ismatrix(inputImage)
        inputImage = repmat(inputImage, [1 1 3]);
    else
        inputImage = inputImage(:, :, 1:3);
    end
    outputImage = imresize(inputImage, [targetHeight targetWidth]);
end

function commentText = make_pipeline_comment(pipelineName, parameterText, lowImage, resultImage, isColorImage)
    lowStats = compute_image_stats(lowImage, isColorImage);
    resultStats = compute_image_stats(resultImage, isColorImage);
    brightnessGain = resultStats.Mean - lowStats.Mean;
    contrastGain = resultStats.Std - lowStats.Std;

    if contains(pipelineName, 'Pipeline 1')
        methodPart = 'Απλή και γρήγορη βελτίωση φωτεινότητας με gamma correction και άνοιγμα του δυναμικού εύρους.';
    elseif contains(pipelineName, 'Pipeline 2')
        methodPart = 'Πιο ισορροπημένη προσέγγιση, επειδή μειώνει θόρυβο, αυξάνει την τοπική αντίθεση και επαναφέρει λεπτομέρειες με unsharp
masking.';
    elseif contains(pipelineName, 'Retinex')
        methodPart = sprintf('H Retinex-like μέθοδος με %s εκτιμά τον φωτισμό με Gaussian smoothing και ενισχύει την τοπική αντίθεση.',
parameterText);
    else
        methodPart = 'Η αρχική εικόνα χρησιμοποιείται ως βάση σύγκρισης.';
    end
end

```

```

end

if resultStats.SaturatedPercent > lowStats.SaturatedPercent + 5
    riskPart = 'Υπάρχει αυξημένος κορεσμός, άρα μπορεί να εμφανίζεται υπερενίσχυση.';
elseif resultStats.Entropy > lowStats.Entropy + 0.5
    riskPart = 'Η μεγαλύτερη entropy δείχνει περισσότερη ορατή πληροφορία, αλλά μπορεί να περιλαμβάνει και θόρυβο.';
else
    riskPart = 'Η ενίσχυση παραμένει σχετικά ελεγχόμενη.';
end

commentText = sprintf('%s Η μέση φωτεινότητα αλλάζει κατά %.4f και η αντίθεση κατά %.4f. %s', methodPart, brightnessGain, contrastGain, riskPart);
end

function row = make_z_row(imageName, imageNumber, highReferenceName, pipelineName, parameterText, resultImage, highImage, isColorImage, commentText)
    stats = compute_image_stats(resultImage, isColorImage);
    [mseValue, psnrValue] = reference_metrics(highImage, resultImage);
    row = struct('Image', imageName, 'ImageNumber', imageNumber, 'HighReferenceImage', highReferenceName, 'Pipeline', pipelineName, 'Parameter', parameterText, 'MSEToHigh', mseValue, 'PSNRToHigh', psnrValue, 'MeanBrightness', stats.Mean, 'StdContrast', stats.Std, 'MichelsonContrast', stats.Michelson, 'EntropyValue', stats.Entropy, 'Comment', commentText);
end

function [mseValue, psnrValue] = reference_metrics(referenceImage, testImage)
    if isempty(referenceImage)
        mseValue = NaN;
        psnrValue = NaN;
        return;
    end
    referenceImage = resize_to_match(referenceImage, testImage);
    referenceImage = min(max(referenceImage, 0), 1);
    testImage = min(max(testImage, 0), 1);
    if ndims(referenceImage) ~= ndims(testImage) || size(referenceImage, 3) ~= size(testImage, 3)
        referenceImage = to_gray(referenceImage, ndims(referenceImage) == 3 && size(referenceImage, 3) == 3);
        testImage = to_gray(testImage, ndims(testImage) == 3 && size(testImage, 3) == 3);
    end
    difference = testImage - referenceImage;
    mseValue = mean(difference(:) .^ 2);
    if mseValue == 0
        psnrValue = Inf;
    else
        psnrValue = 10 * log10(1 / mseValue);
    end
end

function resizedImage = resize_to_match(referenceImage, targetImage)
    if size(referenceImage, 1) ~= size(targetImage, 1) || size(referenceImage, 2) ~= size(targetImage, 2)
        resizedImage = imresize(referenceImage, [size(targetImage, 1), size(targetImage, 2)]);
    else
        resizedImage = referenceImage;
    end
    if ismatrix(targetImage) && ndims(resizedImage) == 3 && size(resizedImage, 3) == 3
        resizedImage = rgb2gray(resizedImage);
    elseif ndims(targetImage) == 3 && size(targetImage, 3) == 3 && ismatrix(resizedImage)
        resizedImage = repmat(resizedImage, [1 1 3]);
    end
end

function commentText = make_z_comment(pipelineName, resultImage, highImage, isColorImage)
    stats = compute_image_stats(resultImage, isColorImage);
    [~, psnrValue] = reference_metrics(highImage, resultImage);
    if isempty(highImage)
        referencePart = 'Δεν υπάρχει φωτεινή εικόνα αναφοράς, οπότε η αξιολόγηση βασίζεται σε no-reference μεγέθη και οπτική σύγκριση.';
    elseif psnrValue >= 20
        referencePart = sprintf('Το PSNR είναι σχετικά καλό (%.2f dB), άρα το αποτέλεσμα βρίσκεται κοντά στη φωτεινή εικόνα αναφοράς.', psnrValue);
    elseif psnrValue >= 15
        referencePart = sprintf('Το PSNR είναι μέτριο (%.2f dB), άρα υπάρχει βελτίωση αλλά και διαφορά από τη φωτεινή αναφορά.', psnrValue);
    else
        referencePart = sprintf('Το PSNR είναι χαμηλό (%.2f dB), άρα το αποτέλεσμα απέχει αρκετά από τη φωτεινή αναφορά.', psnrValue);
    end
    if stats.Michelson > 0.85
        contrastPart = 'Η αντίθεση είναι υψηλή και μπορεί να κάνει ορατές περισσότερες λεπτομέρειες.';
    elseif stats.Michelson > 0.55
        contrastPart = 'Η αντίθεση είναι μέτρια και το αποτέλεσμα παραμένει σχετικά ισορροπημένο.';
    else
        contrastPart = 'Η αντίθεση παραμένει χαμηλή, επομένως αρκετές περιοχές μπορεί να φαίνονται επίπεδες.';
    end
    commentText = sprintf('%s Μέση φωτεινότητα %.4f, τυπική απόκλιση %.4f, Michelson %.4f, entropy %.4f. %s', referencePart, stats.Mean, stats.Std, stats.Michelson, stats.Entropy, contrastPart);
    if strcmp(pipelineName, 'Original low-light')
        commentText = ['Αρχική low-light εικόνα για βάση σύγκρισης. ', commentText];
    end
end

function row = make_h_row(imageName, imageNumber, highReferenceName, pipelineName, parameterText, edgeMethod, edgeMap, originalEdgeMap, highEdgeMap, commentText)
    edgeCount = nnz(edgeMap);
    edgeDensity = 100 * edgeCount / numel(edgeMap);
    componentInfo = edge_component_info(edgeMap);

```

```

if isempty(highEdgeMap)
    jaccardValue = NaN;
else
    jaccardValue = edge_jaccard(edgeMap, highEdgeMap);
end
originalEdgeCount = nnz(originalEdgeMap);
edgeCountRatio = edgeCount / (originalEdgeCount + eps);
row = struct('Image', imageName, 'ImageNumber', imageNumber, 'HighReferenceImage', highReferenceName, 'Pipeline', pipelineName,
'Parameter', parameterText, 'EdgeDetector', edgeMethod, 'EdgeCount', edgeCount, 'EdgeDensity', edgeDensity, 'ComponentCount',
componentInfo.ComponentCount, 'SmallComponentPercent', componentInfo.SmallComponentPercent, 'SignificantComponentPercent',
componentInfo.SignificantComponentPercent, 'JaccardToHigh', jaccardValue, 'EdgeCountRatioToOriginal', edgeCountRatio, 'Comment',
commentText);
end

function info = edge_component_info(edgeMap)
    components = bwconncomp(edgeMap);
    info.ComponentCount = components.NumObjects;
    if components.NumObjects == 0
        info.SmallComponentPercent = 0;
        info.SignificantComponentPercent = 0;
    else
        componentSizes = cellfun(@numel, components.PixelIdxList);
        info.SmallComponentPercent = 100 * mean(componentSizes <= 5);
        info.SignificantComponentPercent = 100 * mean(componentSizes >= 25);
    end
end

function value = edge_jaccard(edgeMapA, edgeMapB)
    edgeMapA = logical(edgeMapA);
    edgeMapB = logical(edgeMapB);
    intersectionCount = nnz(edgeMapA & edgeMapB);
    unionCount = nnz(edgeMapA | edgeMapB);
    value = intersectionCount / (unionCount + eps);
end

function commentText = make_h_comment(pipelineName, edgeMap, originalEdgeMap, highEdgeMap)
    edgeRatio = nnz(edgeMap) / (nnz(originalEdgeMap) + eps);
    componentInfo = edge_component_info(edgeMap);
    if edgeRatio > 1.6
        countPart = 'Το πλήθος ακμών αυξάνεται πολύ, οπότε αποκαλύπτονται λεπτομέρειες αλλά πιθανόν ενισχύεται και θόρυβος.';
    elseif edgeRatio > 1.1
        countPart = 'Το πλήθος ακμών αυξάνεται ήπια και δείχνει καλύτερη ανάδειξη δομών.';
    elseif edgeRatio < 0.75
        countPart = 'Το πλήθος ακμών μειώνεται, πιθανώς επειδή η επεξεργασία εξομαλύνει λεπτομέρειες.';
    else
        countPart = 'Το πλήθος ακμών μένει κοντά στην αρχική low-light εικόνα.';
    end
    if componentInfo.SmallComponentPercent > 60
        noisePart = 'Τα πολλά μικρά components δείχνουν θορυβώδες edge map.';
    elseif componentInfo.SmallComponentPercent > 35
        noisePart = 'Υπάρχουν αρκετά μικρά components, άρα χρειάζεται προσοχή στην ερμηνεία.';
    else
        noisePart = 'Το edge map έχει λιγότερα μικρά θραύσματα και φαίνεται πιο καθαρό.';
    end
    if componentInfo.SignificantComponentPercent > 25
        objectPart = 'Οι μεγαλύτερες συνδεδεμένες ακμές βοηθούν στην ορατότητα σημαντικών αντικειμένων.';
    else
        objectPart = 'Οι σημαντικές συνεχόμενες ακμές παραμένουν περιορισμένες.';
    end
    if isempty(highEdgeMap)
        referencePart = 'Δεν υπάρχει φωτεινή αναφορά για σύγκριση ακμών.';
    else
        jaccardValue = edge_jaccard(edgeMap, highEdgeMap);
        if jaccardValue > 0.35
            referencePart = sprintf('Η συμφωνία με τις ακμές της φωτεινής αναφοράς είναι σχετικά καλή (Jaccard %.3f).', jaccardValue);
        elseif jaccardValue > 0.20
            referencePart = sprintf('Η συμφωνία με τις ακμές της φωτεινής αναφοράς είναι μέτρια (Jaccard %.3f).', jaccardValue);
        else
            referencePart = sprintf('Η συμφωνία με τις ακμές της φωτεινής αναφοράς είναι χαμηλή (Jaccard %.3f).', jaccardValue);
        end
    end
    commentText = [pipelineName, ': ', countPart, ' ', noisePart, ' ', objectPart, ' ', referencePart];
end

function summaryText = make_h_image_summary(hRows, imageName)
    rows = strcmp({hRows.Image}', imageName) & strcmp({hRows.EdgeDetector}', 'Canny') & ~strcmp({hRows.Pipeline}', 'Original low-light');
    selectedRows = hRows(rows);
    if isempty(selectedRows)
        summaryText = 'Δεν υπολογίστηκαν Canny ακμές στις βελτιωμένες εικόνες.';
        return;
    end
    jaccardValues = [selectedRows.JaccardToHigh]';
    if all(isnan(jaccardValues))
        scoreValues = [selectedRows.SignificantComponentPercent]' - [selectedRows.SmallComponentPercent]';
        [~, bestIdx] = max(scoreValues);
        summaryText = sprintf('Χωρίς φωτεινή αναφορά, πιο χρήσιμο για επόμενο vision στάδιο φαίνεται το %s, επειδή κρατά περισσότερες σημαντικές ακμές με λιγότερο μικρό θόρυβο.', selectedRows(bestIdx).Pipeline);
    else
        [~, bestIdx] = max(jaccardValues);
        summaryText = sprintf('Με βάση το Canny και τη φωτεινή αναφορά, καλύτερη συμφωνία ακμών δίνει το %s.',

```

```

selectedRows(bestIdx).Pipeline);
end
end

function summaryTable = make_pipeline_summary_table(pipelineTable)
    enhancedRows = ~strcmp(pipelineTable.Pipeline, 'Original low-light');
    pipelines = unique(pipelineTable.Pipeline(enhancedRows), 'stable');
    rowTemplate = struct('Pipeline', '', 'MeanBrightness', 0, 'MeanStdContrast', 0, 'MeanEntropyValue', 0, 'MeanDarkPixelPercent', 0,
'MeanSaturatedPixelPercent', 0);
    summaryRows = repmat(rowTemplate, numel(pipelines), 1);
    for pipelineIdx = 1:numel(pipelines)
        currentPipeline = pipelines{pipelineIdx};
        rows = enhancedRows & strcmp(pipelineTable.Pipeline, currentPipeline);
        summaryRows(pipelineIdx) = struct('Pipeline', currentPipeline, 'MeanBrightness', mean(pipelineTable.MeanBrightness(rows), 'omitnan'),
'MeanStdContrast', mean(pipelineTable.StdContrast(rows), 'omitnan'), 'MeanEntropyValue', mean(pipelineTable.EntropyValue(rows), 'omitnan'),
'MeanDarkPixelPercent', mean(pipelineTable.DarkPixelPercent(rows), 'omitnan'), 'MeanSaturatedPixelPercent',
mean(pipelineTable.SaturatedPixelPercent(rows), 'omitnan'));
    end
    summaryTable = struct2table(summaryRows);
end

function summaryTable = make_best_pipeline_summary(zTable)
    imageNames = unique(zTable.Image, 'stable');
    rowTemplate = struct('Image', '', 'ImageNumber', 0, 'HighReferenceImage', '', 'BestPipeline', '', 'PSNRToHigh', NaN, 'MSEToHigh', NaN,
'MeanBrightness', 0, 'StdContrast', 0, 'EntropyValue', 0);
    summaryRows = repmat(rowTemplate, numel(imageNames), 1);
    for imageIdx = 1:numel(imageNames)
        currentImage = imageNames{imageIdx};
        rows = strcmp(zTable.Image, currentImage) & ~strcmp(zTable.Pipeline, 'Original low-light');
        selectedRows = zTable(rows, :);
        if any(~isnan(selectedRows.PSNRToHigh))
            [~, bestIdx] = max(selectedRows.PSNRToHigh);
        else
            [~, bestIdx] = max(selectedRows.EntropyValue);
        end
        bestRow = selectedRows(bestIdx, :);
        summaryRows(imageIdx) = struct('Image', currentImage, 'ImageNumber', bestRow.ImageNumber, 'HighReferenceImage',
bestRow.HighReferenceImage{1}, 'BestPipeline', bestRow.Pipeline{1}, 'PSNRToHigh', bestRow.PSNRToHigh, 'MSEToHigh', bestRow.MSEToHigh,
'MeanBrightness', bestRow.MeanBrightness, 'StdContrast', bestRow.StdContrast, 'EntropyValue', bestRow.EntropyValue);
    end
    summaryTable = struct2table(summaryRows);
end

function summaryTable = make_edge_summary_table(hTable)
    cannyRows = strcmp(hTable.EdgeDetector, 'Canny') & ~strcmp(hTable.Pipeline, 'Original low-light');
    pipelines = unique(hTable.Pipeline(cannyRows), 'stable');
    rowTemplate = struct('Pipeline', '', 'MeanEdgeDensity', 0, 'MeanSmallComponentPercent', 0, 'MeanSignificantComponentPercent', 0,
'MeanJaccardToHigh', NaN);
    summaryRows = repmat(rowTemplate, numel(pipelines), 1);
    for pipelineIdx = 1:numel(pipelines)
        currentPipeline = pipelines{pipelineIdx};
        rows = cannyRows & strcmp(hTable.Pipeline, currentPipeline);
        summaryRows(pipelineIdx) = struct('Pipeline', currentPipeline, 'MeanEdgeDensity', mean(hTable.EdgeDensity(rows), 'omitnan'),
'MeanSmallComponentPercent', mean(hTable.SmallComponentPercent(rows), 'omitnan'), 'MeanSignificantComponentPercent',
mean(hTable.SignificantComponentPercent(rows), 'omitnan'), 'MeanJaccardToHigh', mean(hTable.JaccardToHigh(rows), 'omitnan'));
    end
    summaryTable = struct2table(summaryRows);
end

function write_summary_report(reportFid, pipelineTable)
    fprintf(reportFid, 'Συνοπτική παρατήρηση:\n');
    enhancedRows = ~strcmp(pipelineTable.Pipeline, 'Original low-light');
    pipelines = unique(pipelineTable.Pipeline(enhancedRows), 'stable');
    for pipelineIdx = 1:numel(pipelines)
        rows = enhancedRows & strcmp(pipelineTable.Pipeline, pipelines{pipelineIdx});
        meanBrightness = mean(pipelineTable.MeanBrightness(rows), 'omitnan');
        meanContrast = mean(pipelineTable.StdContrast(rows), 'omitnan');
        meanEntropy = mean(pipelineTable.EntropyValue(rows), 'omitnan');
        fprintf(reportFid, '%s: μέση φωτεινότητα %.4f, μέση αντίθεση %.4f, μέση entropy %.4f.\n', pipelines{pipelineIdx}, meanBrightness,
meanContrast, meanEntropy);
    end
    fprintf(reportFid, 'Συνήθως μικρό sigma στο Retinex δίνει πιο έντονη τοπική αντίθεση αλλά μπορεί να τονίσει θόρυβο, ενώ μεγαλύτερο sigma
δίνει πιο ομαλό αποτέλεσμα.\n');
end

function write_z_summary_report(zReportFid, zTable)
    fprintf(zReportFid, 'Συνοπτική ποσοτική αξιολόγηση:\n');
    enhancedRows = ~strcmp(zTable.Pipeline, 'Original low-light');
    pipelines = unique(zTable.Pipeline(enhancedRows), 'stable');
    for pipelineIdx = 1:numel(pipelines)
        rows = enhancedRows & strcmp(zTable.Pipeline, pipelines{pipelineIdx});
        meanMse = mean(zTable.MSEToHigh(rows), 'omitnan');
        meanPsnr = mean(zTable.PSNRToHigh(rows), 'omitnan');
        meanBrightness = mean(zTable.MeanBrightness(rows), 'omitnan');
        meanContrast = mean(zTable.StdContrast(rows), 'omitnan');
        meanEntropy = mean(zTable.EntropyValue(rows), 'omitnan');
        fprintf(zReportFid, '%s: μέσο MSE %.5f, μέσο PSNR %.2f dB, μέση φωτεινότητα %.4f, αντίθεση %.4f, entropy %.4f.\n',
pipelines{pipelineIdx}, meanMse, meanPsnr, meanBrightness, meanContrast, meanEntropy);
    end
    fprintf(zReportFid, 'Με εικόνα αναφοράς προτιμάται υψηλότερο PSNR και χαμηλότερο MSE. Χωρίς αναφορά, κοιτάμε αν η φωτεινότητα, η αντίθεση

```

```

και η entropy αυξάνονται χωρίς υπερβολικό κορεσμό ή θόρυβο.\n');
end

function write_h_summary_report(hReportFid, hTable)
fprintf(hReportFid, 'Συνοπτική αξιολόγηση ακμών:\n');
cannyRows = strcmp(hTable.EdgeDetector, 'Canny') & ~strcmp(hTable.Pipeline, 'Original low-light');
if any(cannyRows)
    pipelines = unique(hTable.Pipeline(cannyRows), 'stable');
    for pipelineIdx = 1:numel(pipelines)
        rows = cannyRows & strcmp(hTable.Pipeline, pipelines{pipelineIdx});
        meanDensity = mean(hTable.EdgeDensity(rows), 'omitnan');
        meanSmall = mean(hTable.SmallComponentPercent(rows), 'omitnan');
        meanSignificant = mean(hTable.SignificantComponentPercent(rows), 'omitnan');
        meanJaccard = mean(hTable.JaccardToHigh(rows), 'omitnan');
        fprintf(hReportFid, '%s: μέση πυκνότητα ακμών %.3f%%, μικρά components %.2f%%, σημαντικά components %.2f%%, Jaccard προς high
%.3f.\n', pipelines{pipelineIdx}, meanDensity, meanSmall, meanSignificant, meanJaccard);
    end
end
fprintf(hReportFid, 'Γενικά, μια βελτίωση εικόνας βοηθά ως προεπεξεργασία όταν αυξάνει τις συνεχόμενες και σημαντικές ακμές χωρίς να
γεμίζει το edge map με πολλά μικρά θορυβώδη components.\n');
end

function safeName = make_safe_filename(fileName)
safeName = regexprep(fileName, '^[^A-Za-z0-9]+', '_');
safeName = regexprep(safeName, '^+|_+$', '');
if isempty(safeName)
    safeName = 'image';
end
end

function textValue = value_or_none(inputText)
if isempty(inputText)
    textValue = 'δεν βρέθηκε';
else
    textValue = inputText;
end
end

```